Rust Embedded Hardware Support

A Complete Guide Using the Silicon Labs EFR32MG24

Marcelo Correa[1]

2026
Version 1.0.0

[1]With contributions from the embedded Rust community

Copyright © 2026 Marcelo Correa
Licensed under Apache-2.0 or MIT (at your option)

Source code and examples:
https://github.com/bitscrafts/EFR32MG2X-RS

ISBN: 978-1-XXXX-XXXX-X

Typeset in LaTeX (pdfLaTeX, TeX Live)
Document version: 1.0.0 (2026-11-11)
First printing: January 2026

Contents

List of Code Listings vii

Preface ix

I PAC Generation and Foundation 1

1 Introduction to Embedded Rust and Project Architecture 3
1.1 Rust's Value Proposition for Embedded . . . 4
1.1.1 Memory Safety Without Garbage Collection . . . 4
1.1.2 Zero-Cost Abstractions . . . 4
1.1.3 Fearless Concurrency . . . 4
1.1.4 Comparison with Traditional Embedded Languages . . . 5
1.1.5 Rust vs C . . . 5
1.1.6 Real-World Impact . . . 5
1.1.7 Rust's Embedded Ecosystem . . . 5
1.1.8 Core Crates . . . 5
1.1.9 Tooling Integration . . . 5
1.1.10 Addressing Embedded Development Challenges . . . 6
1.1.11 Resource Constraints . . . 6
1.1.12 Deterministic Behavior . . . 6
1.1.13 Hardware Interface Safety . . . 6
1.1.14 The EFR32MG24: Ideal for Rust . . . 6
1.1.15 Summary . . . 7
1.2 Your First Bare-Metal Rust Program . . . 8
1.2.1 Creating a Minimal Embedded Project . . . 8
1.2.2 Configuring for bare_metal: `no_std` . . . 8
1.2.3 What is no_std? . . . 8
1.2.4 The Entry Point . . . 8
1.2.5 Cross-Compilation . . . 9
1.2.6 Building the Project . . . 9
1.2.7 Analyzing the Binary . . . 9
1.2.8 What This Demonstrates . . . 9
1.3 The PAC/HAL/BSP Architecture Pattern . . . 11
1.3.1 The Three-Layer Architecture . . . 11
1.3.2 Peripheral Access Crate (PAC) . . . 11
1.3.3 Hardware Abstraction Layer (HAL) . . . 12
1.3.4 Board Support Package (BSP) . . . 12
1.3.5 The Flow of Abstractions . . . 13
1.3.6 Why This Separation Matters . . . 13
1.3.7 PAC Regeneration . . . 13
1.3.8 Multiple HAL Implementations . . . 13

1.3.9 Ecosystem Compatibility . . . 13
1.3.10 Our Approach in This Book . . . 14
1.4 Project Organization and Repository Structure . . . 15
1.4.1 Top-Level Directory Structure . . . 15
1.4.2 Crate Organization . . . 16
1.4.3 Peripheral Access Crate (efr32mg24-pac) . . . 16
1.4.4 Hardware Abstraction Layer (efr32mg24-hal) . . . 16
1.4.5 Radio System Crates (efr32mg24-rail-sys) . . . 16
1.4.6 Protocol System Crates (bluetooth-sys, etc.) . . . 16
1.4.7 Board Support Package (xiao-mg24-bsp) . . . 16
1.4.8 Why This Structure Works . . . 16
1.4.9 Clear Separation of Concerns . . . 16
1.4.10 Testing Strategy . . . 17
1.4.11 Version Management . . . 17
1.5 Development Prerequisites and Toolchain Setup . . . 18
1.5.1 Required Tools . . . 18
1.5.2 Installation on Linux (Ubuntu/Debian) . . . 18
1.5.3 Installation on macOS . . . 18
1.5.4 Installation on Windows . . . 18
1.5.5 Hardware Debug Probe Setup . . . 18
1.5.6 Installing probe-rs . . . 18
1.5.7 Flashing: one sanctioned path . . . 19
1.5.8 Updating Debug Probe Firmware . . . 19
1.5.9 Development Environment Setup . . . 19
1.5.10 VS Code Configuration . . . 19
1.5.11 Cargo Configuration for Embedded . . . 19
1.5.12 Git Configuration . . . 19
1.5.13 Verifying Your Setup . . . 19
1.6 Building Your First Embedded Binary . . . 21
1.6.1 Preparing a Complete Example . . . 21
1.6.2 Reference: Complete Project Example . . . 21
1.6.3 Understanding the Build Process . . . 21
1.6.4 Compile Step . . . 21
1.6.5 Binary Analysis . . . 21
1.6.6 Size Analysis . . . 21
1.6.7 Disassembly . . . 22
1.6.8 Listing Sections . . . 22
1.6.9 Build Optimization Comparison . . . 22
1.6.10 Linking and Linker Scripts . . . 23
1.6.11 Memory Layout . . . 23
1.6.12 Two Linker Flags That Are Not Optional . . . 23
1.6.13 Examining the Linker Output . . . 23
1.6.14 Creating a Flashable Binary . . . 23
1.6.15 Convert to Binary Format . . . 23
1.6.16 Hex Format for Some Tools . . . 24
1.6.17 Automated Build Script . . . 24
1.6.18 Build Automation with Justfile . . . 24
1.6.19 Key Takeaways . . . 24
1.7 What Goes In Each Crate, and Why . . . 25
1.7.1 The Question, Stated Sharply . . . 25
1.7.2 Three Candidate Layouts . . . 25

1.7.3 What Four Ecosystems Actually Do . . . 26
1.7.4 The Decision, and the Rule Behind Each Part . . . 26
1.7.5 Where the Algorithm Goes . . . 27
1.7.6 And Where the Examples Go . . . 27
1.7.7 A Decision Procedure for Hardware Not In This Book . . . 27
1.7.8 Adding Support for a New Chip or Board: the Order of Work . . . 28
1.7.9 What This Costs . . . 29
1.8 The Boundaries the Compiler Cannot See . . . 29
1.8.1 The Question That Finds Them . . . 29
1.8.2 The Same Defect, One Layer Down . . . 30
1.8.3 A Layer That Exports the Layer Below . . . 30
1.8.4 Why It Kept Happening: The Table . . . 30
1.8.5 Writing the Boundary Down as a Check . . . 30
1.8.6 What Is Not a Violation . . . 31
1.8.7 The Procedure, for the Next Project . . . 31
1.9 The Silicon Labs Toolchain, From the Outside . . . 32
1.9.1 The Pieces, and What They Are For . . . 32
1.9.2 JTAG, SWD, and the Debugger Already on the Board . . . 32
1.9.3 The Bootloader Is Not Optional, and It Is Why Linker Scripts Matter . . . 33
1.9.4 Simplicity Commander: Convenient, and the Wrong Tool Here . . . 33
1.9.5 The Radio Stack: Two Licences in One Directory . . . 33
1.9.6 The Radio Configurator, and Why RAIL Needs a Generated File . . . 34
1.9.7 What To Take Away . . . 34

2 Understanding SVD Files and the PAC Generation Process 37
2.1 What an SVD Actually Contains . . . 38
2.1.1 The Five Things That Matter . . . 38
2.1.2 A Register, in the SVD and in Rust . . . 38
2.1.3 Where the SVD Comes From, and Why It Is Trustworthy . . . 38
2.1.4 The Practical Consequence: One File, One Crate . . . 38
2.2 What svd2rust Emits, and How to Read It . . . 38
2.2.1 Shape 1: The Peripheral . . . 39
2.2.2 Shape 2: The Register Block . . . 39
2.2.3 Shape 3: Read, Write, Modify . . . 39
2.2.4 Shape 4: The Field Writer . . . 39
2.2.5 Unsafe, and Where It Is Honest . . . 39
2.3 The SVD as an Oracle . . . 39
2.3.1 Case 1: The Accelerator That Never Ran . . . 39
2.3.2 Case 2: A Register That Is Not There . . . 40
2.3.3 The Habit . . . 40
2.4 Why PAC-Level Examples Exist Anyway . . . 40
2.4.1 The Ladder . . . 40
2.4.2 What the PAC Level Proves . . . 41
2.4.3 The Rule That Keeps It Honest . . . 41
2.4.4 When to Delete One . . . 41
2.5 What the SVD Cannot Tell You . . . 41
2.5.1 Class 1: The Board . . . 42
2.5.2 Class 2: The Aliases . . . 42
2.5.3 Class 3: Timing and Sequencing . . . 42
2.5.4 The Order to Read Things In . . . 42

3 Build Systems and Cross-Compilation Setup **45**
3.1 The Target Triple, Decoded . . . 46
3.1.1 Hard Float, and Why It Matters Here . . . 46
3.1.2 The Toolchain, in Files . . . 46
3.1.3 Per-Crate Features, Not Per-Crate Flags . . . 46
3.2 The Bootloader Decides Your Memory Layout . . . 46
3.2.1 What Is at the Base of Flash . . . 46
3.2.2 The Wrong memory.x, and What It Does . . . 47
3.2.3 The Version in Use . . . 47
3.2.4 The Rule, Enforced in a Script . . . 47
3.3 –nmagic, or the Linker's Idea of a Segment . . . 47
3.3.1 What the Linker Does by Default . . . 47
3.3.2 The Symptom, Measured . . . 47
3.3.3 The Fix, and Why It Is One Flag . . . 48
3.3.4 Checking the Invariant Instead of Trusting the Flag . . . 48
3.4 Builds That Lie . . . 48
3.4.1 Case 1: A build.rs That Did Not Watch Its Own C File . . . 48
3.4.2 Case 2: A Book That Compiled Yesterday's Sources . . . 48
3.4.3 The Pattern Behind Both . . . 48
3.5 The Gate: What Runs Without a Board . . . 48
3.5.1 What It Checks . . . 49
3.5.2 The Discovered-vs-Listed Rule . . . 49
3.5.3 The Rule Nobody Wrote Down . . . 49
3.5.4 What a Gate Cannot Do . . . 49

II Hardware Abstraction Layer (HAL) **51**

4 Phase A: GPIO Implementation **53**
4.1 The Registers, and How Few There Are . . . 54
4.1.1 The Clock Gate Comes First . . . 54
4.1.2 Not Every Port Has Every Register . . . 54
4.2 A Pin Is Three Numbers . . . 54
4.2.1 What the Small Version Looks Like . . . 54
4.2.2 Why the Larger Version Was Retired . . . 55
4.2.3 What Replaced the Type System . . . 55
4.3 Read-Modify-Write, or Why the Bus Stopped Working . . . 55
4.3.1 The Defect . . . 55
4.3.2 How It Was Found . . . 55
4.3.3 The Rule . . . 56
4.3.4 A Second, Quieter Version of the Same Bug . . . 56
4.4 Open Drain, and the Line That Has Two Masters . . . 56
4.4.1 What WIRED_AND Does . . . 56
4.4.2 The Measurement That Proved the Pull-Ups . . . 56
4.4.3 Bit-Banging an Open-Drain Bus . . . 56
4.4.4 Half a Bit Period, and Why It Is a Number in Cycles . . . 57
4.5 What the Vendor's Own Documents Get Wrong . . . 57
4.5.1 The User LED . . . 57
4.5.2 The IMU's Two Wires . . . 57
4.5.3 The Lesson, Which Is Not "the Vendor Is Wrong" . . . 57
4.6 Proving a Pin Without Trusting Your Eyes . . . 58
4.6.1 The Case That Made This Necessary . . . 58

4.6.2 Reading the Latch Over SWD . . . 58
4.6.3 Three Instruments, Three Questions . . . 58
4.6.4 The Mailbox, Which Is the Real Lesson . . . 58

5 Phase A: Clock Management and System Configuration 61
5.1 The Tree, in the Order You Meet It . . . 62
5.1.1 Question 1: Which Oscillator . . . 62
5.1.2 Question 2: Which Branch . . . 62
5.1.3 Question 3: Which Gate . . . 62
5.1.4 The Three Questions, Together . . . 62
5.2 The Clock You Forgot You Were Using . . . 62
5.2.1 19 MHz, Not 78 . . . 63
5.2.2 Why the Error Is Invisible . . . 63
5.2.3 The Right Shape for the Constant . . . 63
5.2.4 Measuring It Properly . . . 63
5.3 Selecting an Oscillator Is Not Starting It . . . 63
5.3.1 The Two-Step That Is Actually Three . . . 64
5.3.2 The Converter That Hung on FSRCO . . . 64
5.3.3 The Rule That Came Out of It . . . 64
5.4 Delay Without Lying . . . 64
5.4.1 asm::delay Is Not a Cycle Count . . . 64
5.4.2 SysTick: Real Milliseconds . . . 65
5.4.3 Ever the Millisecond: Do Not Divide Early . . . 65
5.4.4 The Cycle Counter: Measurement, Not Delay . . . 65
5.5 What to Write Down, and Where . . . 65
5.5.1 The Three Places . . . 65
5.5.2 How the Numbers Here Are Written . . . 65
5.5.3 The Table That Belongs in Every Project . . . 66
5.5.4 The One That Generalises . . . 66

6 Phase B: Communication Peripherals - USART 69

7 I2C Master Mode 71
7.1 I2C Protocol Fundamentals . . . 72
7.1.1 The Transaction, in Voltage . . . 72
7.1.2 The Electrical Reality on This Board . . . 72
7.1.3 Open Drain in the Registers . . . 73
7.2 The Controller That Could Not Read . . . 73
7.2.1 The Symptom . . . 73
7.2.2 What Was Ruled Out, and How . . . 73
7.2.3 What To Do About It . . . 74
7.2.4 Why This Belongs in a Book . . . 74
7.3 A Transport That Works: The Bit-Banged Master . . . 74
7.3.1 The Whole Protocol in Four Primitives . . . 74
7.3.2 Timing Without a Timer . . . 75
7.3.3 The `embedded-hal` Face . . . 75
7.3.4 Two Buses on One Board . . . 75
7.4 Two Lines, Swapped: A Case Study in Not Trusting a Pin Map . . . 75
7.4.1 What the Documentation Said . . . 75
7.4.2 How It Was Found . . . 76
7.4.3 Verification . . . 76
7.4.4 What This Teaches . . . 77

7.5 What To Take Away . . . 77
7.5.1 Exercises . . . 77

8 Phase B: SPI Master Mode (Three Variants) 79

9 Phase C: Advanced Peripherals - DMA, ADC, and Power Management 81

III Wireless Protocol Stacks 83

10 Phase D: Wireless Protocol Stacks - RAIL Foundation 85

11 Phase D2: Bluetooth 5.3 Low Energy 87
11.1 What the Stack Actually Is . . . 88
11.1.1 Three Libraries, One Protocol . . . 88
11.1.2 BGAPI: Commands In, Events Out . . . 88
11.1.3 What the Stack Assumes You Have Done . . . 88
11.2 The Bridge: bindgen Over a Vendor Header . . . 89
11.2.1 Surprise 1: The Union Is Emulated . . . 89
11.2.2 Surprise 2: A Flexible Array Member Is Not a Slice . . . 89
11.2.3 Surprise 3: One Field, Two Names . . . 89
11.2.4 What the Generated Code Is Good For . . . 89
11.3 The Symbols Nobody Provides . . . 89
11.3.1 The CMSIS Helpers . . . 90
11.3.2 Provided Once, for Every Stack . . . 90
11.3.3 The #[link] Directive That Is Not Obvious . . . 90
11.3.4 The Rule That Generalises . . . 90
11.4 A Beacon and a GATT Peripheral . . . 91
11.4.1 The Beacon: Nine Lines and a Phone . . . 91
11.4.2 The GATT Peripheral: Four Steps and Four Traps . . . 91
11.4.3 Counting Refusals Is the Whole Discipline . . . 91
11.4.4 The Beacon Is the Debugging Instrument . . . 91
11.4.5 What a Client Can Prove, and What It Cannot . . . 92
11.4.6 A Connectable Advertiser Stops When Somebody Connects . . . 92
11.5 What It Costs, and What Is Still Missing . . . 92
11.5.1 The Numbers . . . 92
11.5.2 The Gap That Must Be Named: PSA Crypto . . . 92
11.5.3 The Three Things to Verify Before Believing a BLE Build . . . 93
11.5.4 Where This Book Stands . . . 93
11.5.5 The Four Symbols Between Here and GATT . . . 93
11.6 The API Is Not the Air . . . 93
11.6.1 Four Successes, One Silence . . . 93
11.6.2 The Crystal Has To Be Selected, Not Merely Started . . . 94
11.6.3 The Amplifier Needs Its Curves, and the Call Is One Line . . . 94
11.6.4 The Vector Table Had No Radio In It . . . 94
11.6.5 The Instrument Matters More Than the Hypothesis . . . 94
11.6.6 What the Working Sequence Is . . . 95
11.6.7 Proof Has Layers, and the Highest One Is the Stack . . . 95
11.6.8 The Rule This Chapter Adds to the Book . . . 95
11.7 One Radio, Three Protocols . . . 95
11.7.1 The rules, written while Bluetooth was being brought up . . . 95
11.7.2 What the radio does not decide . . . 96

12 Phase D3: Zigbee 3.0 Protocol Stack **99**

13 Phase D4: Thread Protocol and Matter Integration **101**

IV Testing, BSP, and Production **103**

14 Test-Driven Development **105**
14.1 What Can Be Tested Off Target . . . 106
14.1.1 Split the Policy From the Peripheral . . . 106
14.1.2 The API That Makes Testing Possible . . . 106
14.1.3 The Shape of the Whole Test Suite . . . 106
14.2 Tests That Pin a Threshold to a Measurement . . . 107
14.2.1 The Near-Miss Rule . . . 107
14.2.2 Writing the Test From the Recording, Not From the Idea . . . 107
14.2.3 Embed the Recording in the Image . . . 108
14.3 Nine Real Drops . . . 108
14.3.1 The Protocol . . . 108
14.3.2 What the Recordings Showed . . . 109
14.3.3 The Three Assumptions the Data Destroyed . . . 109
14.3.4 What Is Still Missing . . . 109
14.4 The Instrument That Lied . . . 110
14.4.1 The Sampler That Claimed 400 Hz and Ran at 158 . . . 110
14.4.2 What Gave It Away . . . 110
14.4.3 Three Fixes, and One of Them Was Deleted by the Compiler . . . 110
14.4.4 The Instrument Rules . . . 111
14.5 What To Take Away . . . 111
14.5.1 Exercises . . . 111

15 Board Support Packages and Hardware Abstraction **113**
15.1 Two Pieces of Hardware, Not One . . . 114
15.1.1 What Each Piece Owns . . . 114
15.1.2 The Same PCB With More Parts . . . 114
15.1.3 The Carrier Is a Different Crate, and It Points One Way . . . 114
15.1.4 What the Direction Buys . . . 114
15.2 One Crate per Board, and the Arrow Between Them . . . 115
15.2.1 The Two Schools . . . 115
15.2.2 The Four Crate Questions . . . 115
15.2.3 What Ends Up in the Workspace . . . 115
15.2.4 The Defect the Layout Prevented . . . 115
15.3 What a BSP Should Expose . . . 116
15.3.1 The Right Shape . . . 116
15.3.2 What the Signature Hides, and What It Must Not . . . 116
15.3.3 What It Must Not Hide: Structure . . . 116
15.3.4 Conflicts Should Be Refused, Not Resolved . . . 117
15.4 Examples Belong to the Hardware They Need . . . 117
15.4.1 The Rule . . . 117
15.4.2 The Three Ways to Get It Wrong . . . 117
15.4.3 The Check That Keeps It True . . . 117
15.4.4 The Half-Example That Taught the Rule . . . 117

16 Documentation Strategies and Project Management **119**

17 Production Deployment and Ecosystem Integration **121**

18 Complete Project Walkthrough: From Zero to Production **123**

A EFR32MG24 Reference Manual Quick Guide **125**

B Rust Embedded Ecosystem Overview **127**

C Troubleshooting Common Issues **129**

D Complete Code Examples Repository **131**

E Wireless Protocol Comparison **133**

Glossary **135**

Bibliography **136**

List of Figures

1.1 The PAC/HAL/BSP layered architecture 11
1.2 EFR32MG24 project repository structure 15
1.3 Rust embedded build pipeline 21

List of Code Listings

Preface

What This Book Is

This book documents the complete process of creating Rust support for a new embedded microcontroller family, using the Silicon Labs EFR32MG24 as a real-world case study. It covers everything from generating a Peripheral Access Crate (PAC) from vendor System View Description (SVD) files, through implementing a Hardware Abstraction Layer (HAL), to integrating complex wireless protocol stacks.

What This Book Is Not

This is not an introduction to Rust programming—it assumes basic Rust knowledge. It's also not a replacement for the EFR32MG24 Reference Manual or Rust Embedded Book, which it complements.

Who This Book Is For

- Embedded developers wanting to learn Rust
- Rust developers interested in embedded systems
- Engineers bringing Rust support to new hardware
- Technical leads evaluating Rust for embedded projects
- Computer science students studying systems programming

Structure of the Book

The book is organized into four parts:

Part I (Chapters 1–3): Foundation and PAC Generation
Foundation concepts, SVD files, and build system setup.

Part II (Chapters 4–9): Hardware Abstraction Layer
Peripheral implementations (GPIO, clocks, communication, advanced features).

Part III (Chapters 11–14): Wireless Protocol Stacks
RAIL foundation, Bluetooth LE, Zigbee, Thread, and Matter integration.

Part IV (Chapters 10, 15–18): Testing, BSP, and Production
Test-driven development, board support packages, documentation, and deployment.

Prerequisites

Required knowledge:

- Basic Rust programming (ownership, traits, modules)

- Familiarity with command-line tools
- Understanding of binary/hexadecimal
- Basic electronics knowledge (helpful but not required)

Required hardware:

- Seeed Studio XIAO MG24 Sense development board
- USB-C cable (data-capable, not just power)
- Computer running Linux, macOS, or Windows

Optional but recommended:

- Logic analyzer (for debugging communication protocols)
- Oscilloscope (for signal quality analysis)
- Multiple XIAO MG24 boards (for testing wireless protocols)

How to Read This Book

This book can be read in two ways:

Linear Approach: Read cover-to-cover for complete understanding. Each chapter builds on previous ones.

Reference Approach: Jump to specific chapters for:
- SVD generation → Chapter 2
- GPIO implementation → Chapter 4
- I2C protocol details → Chapter 7
- Bluetooth LE → Chapter 12
- Testing strategies → Chapter 10

All code examples are available in the accompanying repository.

Acknowledgments

This book and the accompanying EFR32MG24 implementation would not be possible without:
- The Rust Embedded Working Group and their excellent tools
- The stm32-rs, nrf-rs, and esp-rs communities for pioneering work
- Silicon Labs for the excellent EFR32MG24 documentation
- The embedded Rust community contributors

Feedback and Contributions

Please report issues, suggest improvements, or contribute at:
https://github.com/bitscrafts/EFR32MG2X-RS

Marcelo Correa
January 2026

Part I

PAC Generation and Foundation

1. Introduction to Embedded Rust and Project Architecture

> *"The biggest memory safety vulnerability is the assumption that memory safety isn't important for embedded systems."*
> – Embedded Systems Security Research, 2023

This chapter introduces embedded Rust development and establishes the foundation for understanding the PAC/HAL/BSP architecture pattern. We'll explore why Rust is compelling for embedded systems, create a bare-metal program, and set up the complete development environment.

Complex software systems, especially those dealing with direct hardware manipulation, memory management, and concurrent interrupts, are notoriously difficult to write correctly. Traditional embedded development often relies on C or C++, languages that provide raw power and direct hardware access but lack modern safety guarantees. Memory safety bugs—buffer overflows, use-after-free, null pointer dereferences, and data races—account for the majority of security vulnerabilities and system crashes in embedded systems.

1.1 Rust's Value Proposition for Embedded

Rust addresses these challenges through a unique combination of features that make it exceptionally well-suited for embedded systems programming:

1.1.1 Memory Safety Without Garbage Collection

Traditional memory-safe languages rely on garbage collection, which introduces unpredictable pauses and significant runtime overhead—unacceptable constraints for resource-constrained microcontrollers. Rust achieves memory safety through its ownership system, enforced at compile time with zero runtime cost.

Key Insight: Rust's borrow checker prevents memory safety bugs at compilation, eliminating entire classes of vulnerabilities before the code ever reaches the hardware.

Consider this common C bug pattern:

```c
void process_data(uint8_t *buffer, size_t len) {
    uint8_t local_buffer[16];
    // Buffer overflow if len > 16!
    memcpy(local_buffer, buffer, len);
    // ... process data
}
```

The same logic in Rust:

```rust
fn process_data(buffer: &[u8]) {
    // Compile error: attempt to copy 32 bytes into 16-byte array
    let mut local_buffer = [0u8; 16];
    local_buffer.copy_from_slice(buffer); // Panics if buffer.len() != 16
    // Or use safe slice copying:
    let count = buffer.len().min(16);
    local_buffer[..count].copy_from_slice(&buffer[..count]);
}
```

The Rust compiler enforces bounds checking at compile time or generates explicit panics for runtime bounds violations, preventing undefined behavior.

1.1.2 Zero-Cost Abstractions

Rust's high-level abstractions compile down to the same efficient machine code as hand-written C. This is critical for embedded systems where every byte and CPU cycle matters.

A GPIO pin abstraction in Rust:

```rust
let pin = gpio.pin0.into_output();
pin.set_high();
```

This compiles to a single register write instruction, identical to:

```c
GPIO->DOUT |= (1 << 0);
```

The Rust code provides type safety, compile-time mode checking, and clear semantics while maintaining identical performance to the raw C version.

1.1.3 Fearless Concurrency

Embedded systems increasingly require concurrent operation: interrupt handlers, DMA transfers, and multiple sensors polling simultaneously. Rust's ownership model prevents data races at compile time.

```rust
// This compiles safely
static mut COUNTER: u32 = 0;

#[interrupt]
fn TIMER0() {
    // Safe because we use atomic operations
    cortex_m::interrupt::free(|_| {
        unsafe { COUNTER += 1; }
    });
}

// This fails to compile!
fn increment_counter() {
    COUNTER += 1; // Error: cannot access mutable static without unsafe
}
```

The compiler forces developers to explicitly handle synchronization, making race conditions nearly impossible.

1.1.4 Comparison with Traditional Embedded Languages

1.1.5 Rust vs C

Feature	C	Rust
Memory Safety	Manual (runtime errors)	Compile-time guaranteed
Concurrency Safety	Manual (locks, careful design)	Compile-time guaranteed
Type Safety	Weak (void*, casts)	Strong (no implicit casts)
Null Pointers	Always possible	Option<T> type
Error Handling	Error codes	Result<T, E> type
Abstraction Cost	Potentially high	Zero-cost
Learning Curve	Moderate	Steep (ownership concepts)
Ecosystem	Mature, established	Growing rapidly
Real-time Suitability	Excellent (predictable)	Excellent (no GC)

Table 1.1: Rust vs C for embedded development

1.1.6 Real-World Impact

The Linux kernel, which has historically been written entirely in C, began accepting Rust code in version 6.1 (2022). This demonstrates Rust's maturity for systems programming even at the highest levels.

In the embedded space, companies are reporting:

- 40-50
- Near elimination of memory corruption bugs

- Improved team productivity through better compiler diagnostics
- Easier onboarding for new developers

1.1.7 Rust's Embedded Ecosystem

1.1.8 Core Crates

The embedded Rust ecosystem provides standardized interfaces:

- `embedded-hal`: Hardware abstraction layer traits for portability
- `cortex-m`: Low-level Cortex-M register access
- `cortex-m-rt`: Startup code and interrupt handling
- `cortex-m-semihosting`: Debug output via debugger
- `svd2rust`: PAC generation from SVD files
- `panic-halt`: Panic handler for minimal footprint

1.1.9 Tooling Integration

Rust integrates seamlessly with embedded development workflows:

- **cargo**: Built-in package manager and build system
- **probe-rs**: Flash programming and debugging
- **defmt**: Efficient logging for resource-constrained targets
- **flip-link**: Linker wrapper for stack overflow protection

1.1.10 Addressing Embedded Development Challenges

1.1.11 Resource Constraints

Rust's ownership system enables compile-time memory management without runtime overhead. The `no_std` environment runs without heap allocation, relying entirely on stack allocation and static data.

```rust
#![no_std]  // No standard library (no heap allocation)
#![no_main] // Disabled main wrapping

// All memory is statically allocated
static BUFFER: [u8; 1024] = [0; 1024];

#[no_mangle]
pub extern "C" fn main() -> ! {
    loop {
        // Application logic
    }
}
```

1.1.12 Deterministic Behavior

Embedded systems require predictable timing. Rust provides:

- No hidden allocations or deallocations
- No garbage collection pauses
- Explicit control over memory layout
- `const fn` for compile-time computation

1.1.13 Hardware Interface Safety

Direct hardware register manipulation is inherently unsafe. Rust forces developers to mark unsafe operations explicitly:

```
pub fn enable_gpio_clock() {
    unsafe {
        (*CMU::ptr()).hfbusclken0.modify(|_, w|
            w.gpio().set_bit()
        );
    }
}
```

The `unsafe` block signals to developers and reviewers that manual verification is required.

1.1.14 The EFR32MG24: Ideal for Rust

The Silicon Labs EFR32MG24 is particularly well-suited for Rust development:

- **ARM Cortex-M33**: Modern architecture with TrustZone-M
- **Memory Protection Unit**: Hardware-enforced memory isolation
- **Deterministic Architecture**: No unpredictable timing behavior
- **Rich Peripherals**: 133 peripherals across multiple domains
- **Unified Radio**: Multi-protocol stack support
- **Hardware Cryptography**: Secure element integration

1.1.15 Summary

Rust brings modern software engineering practices to embedded development without sacrificing performance or control. Its compile-time safety guarantees eliminate entire classes of bugs that plague C-based embedded systems, while its zero-cost abstractions enable expressive, maintainable code that compiles to efficient machine instructions.

The journey through this book will demonstrate:

1. How to generate a Peripheral Access Crate (PAC) from vendor SVD files
2. Building safe, ergonomic HAL abstractions on top of raw registers
3. Implementing standard embedded-hal traits for ecosystem compatibility
4. Creating board support packages for specific hardware
5. Integrating complex protocol stacks (Bluetooth, Zigbee, Thread)

6. Testing strategies for embedded systems
7. Production deployment and maintenance

Along the way, we'll explore the unique challenges and solutions that emerged during the real-world implementation of EFR32MG24 support in Rust.

1.2 Your First Bare-Metal Rust Program

After understanding why Rust is compelling for embedded systems, let's create our first bare-metal Rust program. This example will run without an operating system, directly on the microcontroller hardware.

1.2.1 Creating a Minimal Embedded Project

First, create a new directory for our project:

```
$ mkdir blink_bare_metal
$ cd blink_bare_metal
$ cargo init --name blink_bare_metal
```

This creates a standard Rust project structure. However, for embedded development, we need to modify it significantly.

1.2.2 Configuring for bare_metal: `no_std`

Before we begin, let's understand what a bare-metal embedded project looks like. Edit `Cargo.toml`:

```toml
[package]
name = "blink_bare_metal"
version = "0.1.0"
edition = "2021"

[profile.dev]
opt-level = "z"          # Optimize for size
debug = true

[profile.release]
opt-level = "z"          # Optimize for size
panic = "abort"          # Simplest panic handler

[dependencies]
# We'll add dependencies as needed
```

Now modify `src/main.rs`:

```rust
#![no_std]    // Don't link the standard library (no heap, no OS)
#![no_main]   // We define our own entry point

// Prevent Rust from generating main() wrapper
use core::panic::PanicInfo;

/// Our entry point - Cortex-M calls this on reset
#[no_mangle]
pub extern "C" fn main() -> ! {
    // Application logic will go here

    loop {
        // Infinite loop to prevent return
    }
}

/// Simple panic handler - just halt
#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}
```

1.2.3 What is no_std?

no_std means "no standard library." Our program cannot use:

- Heap allocation (Box, Vec, String)
- File I/O
- Threads
- Networking

We can still use: `core`, slices, static arrays, and direct hardware access.

The standard library requires an operating system for features like heap allocation and threads. In bare-metal embedded, we have no OS, so we use only `core`, which provides fundamental types and operations that don't require an OS.

1.2.4 The Entry Point

In hosted environments (with an OS), Rust generates a C main() function that calls your main(). In bare metal:

- `#[no_mangle]` prevents Rust from changing the function name
- `pub extern "C" fn` ensures the correct calling convention
- `-> !` means "never returns" (diverging function)
- We must provide an infinite loop or halt instruction

1.2.5 Cross-Compilation

Our development machine (likely x86_64) can't execute ARM Cortex-M code. We need to cross-compile:

```
$ rustup target add thumbv8m.main-none-eabihf
```

This installs the Rust standard library for the ARM Cortex-M33 architecture without an OS.

Create `.cargo/config.toml`:

```
[build]
target = "thumbv8m.main-none-eabihf"

[target.thumbv8m.main-none-eabihf]
# No `runner` on purpose: `cargo run` would program the chip the moment it builds,
# and on this board that has to go through the guarded script (see the note below).
rustflags = [
  "-C", "link-arg=-Tlink.x",
  "-C", "link-arg=--nmagic",
]

[env]
DEFMT_LOG = "debug"
```

Why this book has no cargo run. The first 24 KB of flash belong to the Gecko bootloader. A 'runner' in '.cargo/config.toml' turns every `cargo run` into a write, and a single wrong linker script silently relocates the vector table into that region. This project flashes through `scripts/flash.sh`: dry run by default, `--confirm` to write, a hard refusal when the image's lowest address is below `0x08006000`, and a read-back verification afterwards. The same script is what re-flashes the board after a mistake — which matters more than the convenience of `cargo run`.

1.2.6 Building the Project

```
$ cargo build --release
   Compiling blink_bare_metal v0.1.0
    Finished release [optimized] target(s) in 0.5s
```

Check the result:

```
$ cargo size --release
   text    data     bss     dec     hex filename
     64       0       0      64      40 blink_bare_metal
```

Our minimal program is only 64 bytes!

1.2.7 Analyzing the Binary

Use `objdump` to examine the machine code:

```
$ cargo objdump --release -- -d
```

This shows the ARM assembly instructions. You'll see something like:

```
bare_metal:     file format elf32-littlearm

Disassembly of section .text:

main:
    b #-4 <main>
    b #-4 <main>
```

The two `b #-4` instructions are branches to themselves—infinite loops. One is our main() function, the other is the panic handler.

1.2.8 What This Demonstrates

Even at this minimal stage, we've established:

- Cross-compilation toolchain setup
- Bare-metal runtime configuration
- Minimal program that builds successfully
- Linker script and startup code (provided by cortex-m-rt)

The program won't do anything visible yet (no LEDs blinking), but it represents a working foundation. In the next section, we'll add actual hardware interaction.

Exercise (Easy): Modify the panic handler to increment a counter in an infinite loop. Use a static mut variable and examine how the compiler warns about unsafe code.

Exercise (Medium): Research what happens when main() (or the reset handler) returns on a Cortex-M microcontroller. Why is `-> !` important?

Exercise (Hard): Create a minimal linker script (`linker.ld`) and build script (`build.rs`) to understand how the linker allocates memory for your program.

1.3 The PAC/HAL/BSP Architecture Pattern

Embedded Rust development follows a layered architecture that promotes code reuse, safety, and separation of concerns. This pattern is not arbitrary—it reflects years of lessons learned from implementing hardware support across multiple vendors and microcontroller families.

1.3.1 The Three-Layer Architecture

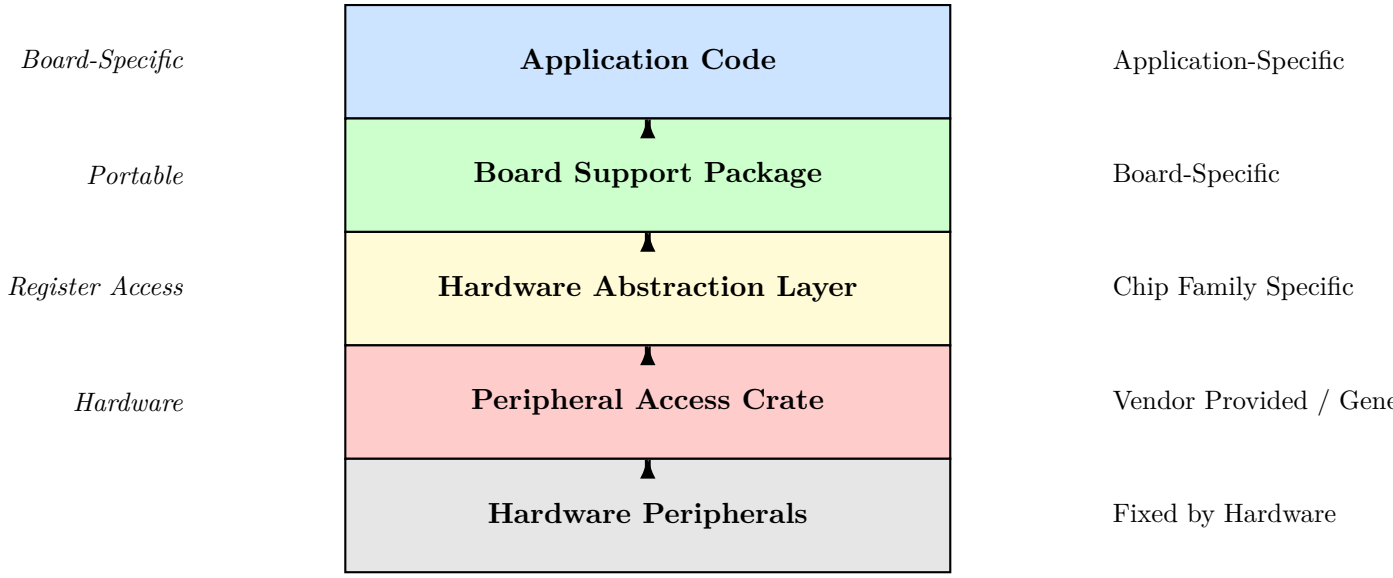


Figure 1.1: The PAC/HAL/BSP layered architecture

Each layer has distinct responsibilities and portability characteristics.

1.3.2 Peripheral Access Crate (PAC)

The PAC provides the lowest-level, register-accurate representation of the microcontroller's peripherals. It is typically generated automatically from the vendor's System View Description (SVD) file using `svd2rust`.

Key characteristics:

- One-to-one mapping with hardware registers
- No abstractions on top of hardware
- All operations are `unsafe` (by design)
- Generated automatically—never hand-edited
- Chip variant specific (EFR32MG24B220 vs A020)

```
// Example: Direct register access via PAC
unsafe {
    (*efr32mg24_pac::GPIO_S::ptr())
        .port_a_dout
        .modify(|r, w| w.bits(r.bits() | (1 << 5)));
}
```

This writes directly to GPIO Port A, bit 5. The `unsafe` keyword forces you to acknowledge that you're bypassing Rust's safety guarantees.

PAC code should be wrapped in HAL abstractions. Direct PAC usage in application code is an anti-pattern that sacrifices safety for no benefit.

1.3.3 Hardware Abstraction Layer (HAL)

The HAL provides safe, ergonomic APIs that wrap the raw PAC access. It implements standard traits for portability and adds compile-time safety guarantees.

Key characteristics:

- Safe APIs (no `unsafe` required by users)
- Type-safe state machines (e.g., pin modes encoded in types)
- Implements `embedded-hal` traits
- Chip family specific (works across all EFR32MG24 variants)
- Medium abstraction level
- Written and maintained by community

```
// Example: GPIO via HAL
let pin = gpio.pin5.into_output();
pin.set_high();
```

Compare this to the PAC version above. The HAL provides:

- Type-safe pin selection (can't use non-existent pins)
- Compile-time mode tracking (can't read from output pin)
- Clear, readable method names
- No unsafe required
- Identical performance (zero-cost abstraction)

1.3.4 Board Support Package (BSP)

The BSP provides board-specific abstractions and convenience APIs. It's the only layer that "knows" about your specific PCB design.

Key characteristics:

- Maps logical functions to physical pins (e.g., LED1 → PA5)
- Provides board-specific initialization sequences
- Combines peripherals for board features
- Board specific (each PCB design has its own BSP)
- Highest abstraction level
- Minimal code (thin wrappers over HAL)

```
// Example: BSP for XIAO MG24 Sense
use xiao_mg24_bsp::{Board, LED};

let board = Board::new();
board.led_green.on();
delay_ms(500);
board.led_green.off();
```

The BSP knows that the green LED connects to PA5 on this particular board and provides a convenient LED abstraction.

1.3.5 The Flow of Abstractions

Here's how a simple operation (turning on an LED) flows through the layers:

1. **Application**: Calls `board.led_red.on()`
2. **BSP**: Translates to `hal.gpio.pin5.set_high()`
3. **HAL**: Checks pin mode, then calls PAC
4. **PAC**: Executes `GPIO->DOUT |= (1 << 5)`
5. **Hardware**: Physical pin voltage goes high, LED illuminates

This layered approach provides benefits:

Portability: Switch to a different EFR32MG24 board? Only rewrite the BSP. HAL and PAC remain unchanged.

Maintainability: Bug in GPIO handling? Fix it in the HAL. All boards using that HAL benefit.

Composability: Libraries written for HAL traits work across all boards using that HAL.

> **Architecture Rule**: Application code should never directly use PAC. Always go through HAL and BSP.

1.3.6 Why This Separation Matters

The embedded Rust community settled on this architecture through experience:

1.3.7 PAC Regeneration

When vendors release updated SVD files (fixing errors or documenting new features), you regenerate the PAC without affecting HAL or BSP code.

1.3.8 Multiple HAL Implementations

Different use cases may need different HAL trade-offs:

- High-performance HAL for real-time applications
- Small-footprint HAL for resource-constrained devices
- Feature-rich HAL for development boards

All can use the same PAC.

1.3.9 Ecosystem Compatibility

The `embedded-hal` trait system enables an ecosystem of drivers:

- Display drivers (SSD1306, ST7789) work with any SPI implementation
- Sensor drivers (BMP280, MPU6050) work with any I2C implementation
- No code changes needed when switching microcontrollers

1.3.10 Our Approach in This Book

In this book, we'll implement the stack bottom-up:

1. **Chapter 2**: Generate the PAC from Silicon Labs SVD
2. **Chapters 4-9**: Build HAL for all peripherals
3. **Chapter 11**: RAIL radio abstraction
4. **Chapters 12-14**: Protocol stacks (Bluetooth, Zigbee, Thread)
5. **Chapter 15**: BSP for XIAO MG24 Sense

This order ensures:

- Each layer has a strong foundation
- Clear dependency management
- Comprehensive testing at each level
- Real hardware validation

Exercise (Easy): Look at a simple development board (like Arduino Uno) and list:

1. Which features belong in PAC vs HAL vs BSP?
2. Why does Arduino's "digitalWrite()" combine all three layers?

Exercise (Medium): Design a type-safe state machine for a USART that:

1. Only allows baud rate configuration when USART is disabled
2. Only allows transmission when USART is enabled
3. Prevents mode changes during active transmission

How would you encode these states in Rust's type system?

Exercise (Hard): Research other embedded HAL implementations:

1. Compare `stm32-hal` vs `nrf-hal`
2. Note differences in how they handle:
    - Clock configuration
    - Error types
    - Feature flags
    - Documentation approach

1.4 Project Organization and Repository Structure

A well-organized repository structure is essential for maintainability, especially in embedded projects where you may support multiple chip variants, boards, and protocol stacks. In this section, we'll examine the repository structure used throughout this book and understand the rationale behind each decision.

1.4.1 Top-Level Directory Structure

Here's the complete structure of our EFR32MG24 project:

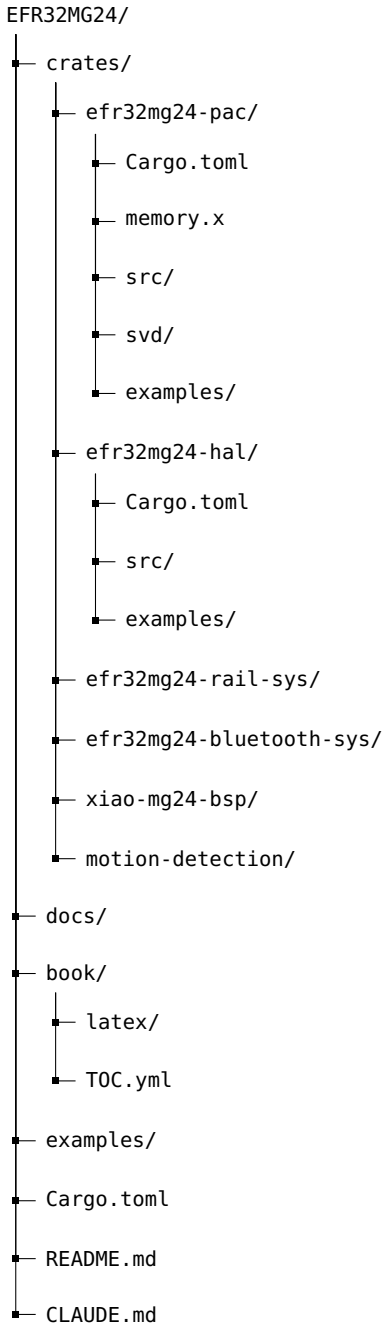


Figure 1.2: EFR32MG24 project repository structure

1.4.2 Crate Organization

1.4.3 Peripheral Access Crate (efr32mg24-pac)

```
crates/efr32mg24-pac/
├── Cargo.toml          # PAC crate metadata
├── memory.x            # Flash at 0x0800_6000 (after the bootloader), RAM 252K
├── svd/
│   └── EFR32MG24B220F1536IM48.svd  # SVD file (one variant, not forty)
├── src/
│   ├── lib.rs          # generated: peripheral singletons, 48.8 KB / 1,529 lines
│   ├── generic.rs      # generated: register access machinery, 28.7 KB
│   └── <periph>_s/     # 103 peripherals, each `<periph>_s.rs` + a directory
└── examples/           # 19 examples, each with a measured result
```

Key Points:

- Generated by `svd2rust 0.37.1`; there is no `mod.rs` in any peripheral — the tool emits `<periph>.rs` beside a directory of register modules, and that file declares them
- Contains ONLY auto-generated code from svd2rust
- Never edited manually after generation
- Version locked to specific SVD file version
- Re-exported by HAL, users never depend on PAC directly

1.4.4 Hardware Abstraction Layer (efr32mg24-hal)

The HAL workspace contains multiple peripheral crates:

```
crates/efr32mg24-hal/
├── Cargo.toml              # HAL crate metadata
├── src/
│   ├── lib.rs              # Main HAL entry point
│   ├── clock/              # Clock management (CMU)
│   │   ├── mod.rs
│   │   ├── types.rs
│   │   ├── traits.rs
│   │   └── README.md
│   ├── gpio/               # GPIO peripheral
│   │   ├── mod.rs
│   │   ├── types.rs
│   │   ├── traits.rs
│   │   └── README.md
│   ├── usart/              # USART communication
│   │   ├── mod.rs
│   │   ├── types.rs
│   │   ├── traits.rs
│   │   └── README.md
│   ├── i2c/                # I2C peripheral
│   │   ├── mod.rs
│   │   ├── types.rs
│   │   ├── traits.rs
│   │   └── README.md
│   ├── spi/                # SPI peripheral
│   │   ├── mod.rs
│   │   ├── types.rs
│   │   ├── traits.rs
│   │   └── README.md
│   └── prelude.rs          # Common imports
├── examples/               # HAL-level examples
│   ├── 01_clock.rs
│   ├── 02_gpio_blink.rs
```

```
│   ├── 03_usart_echo.rs
│   └── ...
└── docs/                  # HAL-specific documentation
    ├── README.md
    └── BUILDING.md
```

File Pattern: Each peripheral follows the same structure with four files: mod.rs, types.rs, traits.rs, and README.md.

```
peripheral-name/
├── mod.rs       // Main implementation
├── types.rs     // Type definitions
├── traits.rs    // Trait implementations
└── README.md    // Per-peripheral documentation
```

1.4.5 Radio System Crates (efr32mg24-rail-sys)

FFI bindings to Silicon Labs RAIL (Radio Abstraction Interface Layer):

```
efr32mg24-rail-sys/
├── Cargo.toml          // Link to vendor libraries
├── build.rs
├── wrapper.h          // bindgen header
└── src/
    └── lib.rs         # Generated FFI bindings
```

Purpose: Enables multi-protocol radio operation (Bluetooth, Zigbee, Thread, proprietary).

1.4.6 Protocol System Crates (bluetooth-sys, etc.)

FFI bindings to higher-level protocol stacks:

```
efr32mg24-bluetooth-sys/
├── Cargo.toml         # Bluetooth library linking
├── build.rs
├── wrapper.h
└── src/
    ├── lib.rs        # FFI bindings
    └── events/       # Event handling code
```

1.4.7 Board Support Package (xiao-mg24-bsp)

Board-specific abstractions:

```
xiao-mg24-bsp/
├── Cargo.toml          # Depends on efr32mg24-hal
├── src/
│   ├── lib.rs         # Board initialization
│   ├── board.rs       # Board struct
│   ├── sensors.rs     # Onboard sensors
│   └── radio.rs       # Radio utilities
└── examples/
    ├── blink_onboard_led.rs
    ├── ble_beacon.rs
    └── zigbee_coordinator.rs
```

1.4.8 Why This Structure Works

1.4.9 Clear Separation of Concerns

- PAC: Hardware truth (auto-generated)
- HAL: Safe abstractions (manually written)
- BSP: Board integration (thin wrappers)
- Application: User logic (no hardware details)

1.4.10 Testing Strategy

Each layer can be tested independently:

- PAC: Verify register addresses match datasheet
- HAL: Unit tests with mocked registers
- BSP: Integration tests on real hardware
- Application: Standard Rust testing

1.4.11 Version Management

Crates can be versioned independently:

```
efr32mg24-pac = "0.1.0"      # Hardware revision specific
efr32mg24-hal = "0.4.2"      # Feature updates
xiao-mg24-bsp = "0.2.1"      # Board revision specific
```

Exercise (Easy): Sketch a project structure for supporting three boards:

- Seeed XIAO MG24 (covered here)
- Silicon Labs xG24-DK2601B dev kit
- Custom PCB with external sensors

What crates would be shared? Which would be board-specific?

Exercise (Medium): Consider a startup shipping products using this architecture:

1. How would you version the crates when releasing new firmware?
2. What testing strategy ensures HAL changes don't break BSP?
3. How would you handle multiple hardware revisions?

Exercise (Hard): Compare this structure to Linux kernel architecture:

1. Map PAC/HAL/BSP to Linux equivalents
2. How does Linux handle different architectures? (arch/ directory)
3. How does Linux handle different boards? (Device Tree)
4. What lessons can we apply to our Rust embedded architecture?

1.5 Development Prerequisites and Toolchain Setup

Before we can build and flash embedded Rust code to the EFR32MG24, we need to install and configure several tools. This section provides a complete setup guide for Linux, macOS, and Windows.

1.5.1 Required Tools

Rust Toolchain:

- **Rust 1.70+** with support for the thumbv8m.main-none-eabihf target
- **cargo-binutils** for examining compiled binaries
- **cargo-size** for checking binary size
- **cargo-objdump** for disassembly

1.5.2 Installation on Linux (Ubuntu/Debian)

```
# Install Rust via rustup
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

source $HOME/.cargo/env

# Add ARM Cortex-M33 target
rustup target add thumbv8m.main-none-eabihf

# Install cargo tools
cargo install cargo-binutils
rustup component add llvm-tools-preview

# Install system dependencies
sudo apt-get update
sudo apt-get install -y \
    build-essential \
    pkg-config \
    libssl-dev \
    git \
    wget
```

1.5.3 Installation on macOS

```
# Install Rust via rustup
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

source $HOME/.cargo/env

# Add ARM Cortex-M33 target
rustup target add thumbv8m.main-none-eabihf

# Install cargo tools
cargo install cargo-binutils
rustup component add llvm-tools-preview

# Install Homebrew if not already installed
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install system dependencies
brew install git wget
```

1.5.4 Installation on Windows

1. Download and install from https://rustup.rs/
2. Run the installer with default settings
3. After installation, open a new PowerShell/Command Prompt
4. Install the ARM target:

```
rustup target add thumbv8m.main-none-eabihf
cargo install cargo-binutils
rustup component add llvm-tools-preview
```

1.5.5 Hardware Debug Probe Setup

The EFR32MG24 requires a debug probe for flashing and debugging. We'll use probe-rs, a Rust-based tool.

1.5.6 Installing probe-rs

```
# Install probe-rs
cargo install probe-rs --features cli

# Verify installation
probe-rs list
```

Expected output (when hardware is connected):

```
Probing OpenUSB... Done
[0]: J-Link Silicon Labs (<serial-number>) (J-Link)
```

1.5.7 Flashing: one sanctioned path

Silicon Labs' Simplicity Commander exists, and it is the right tool for one job: the authorisation that unlocks a secured part. It is *not* the flashing path this book uses, because a bare `commander flash` will happily write wherever the image says — including over the bootloader.

What this project uses instead, and why it is worth the extra script:

```
# 1. show where the image would land, without touching the board
python3 scripts/elf-load-range.py target/thumbv8m.main-none-eabihf/release/examples/pac_imu
# load_min=0x08006000 load_max=0x08006813 size=2068 segments=3

# 2. dry run: prints the plan and refuses anything with load_min < 0x08006000
bash scripts/flash.sh "$ELF"

# 3. write, then verify by reading the flash back
bash scripts/flash.sh "$ELF" --confirm
```

The script assumes the board may hang off a *different* machine from the one you build on, reachable over SSH, and it takes that machine from `FLASH_HOST` (`FLASH_USER` alongside it). If your board is plugged into the machine you build on, leave both alone and it drives the programmer locally — the guards, not the transport, are the point.

> **Measured caveat.** `probe-rs` works for the first attach and then desynchronises on this board's onboard CMSIS-DAP bridge (`Command ID in response (0x14) does not match sent command ID`); a USB reset does not clear it and only a physical replug does. OpenOCD talks to the same probe without the problem, so the script uses OpenOCD for reads and `probe-rs` for the write-and-verify step.

1.5.8 Updating Debug Probe Firmware

If using a J-Link probe:

```
# On Linux/macOS
JLinkExe -CommanderScript <(echo "exit")

# Follow prompts to update firmware
```

1.5.9 Development Environment Setup

1.5.10 VS Code Configuration

Install these extensions:

- rust-analyzer (official Rust support)
- Even Better TOML (for Cargo.toml files)
- CodeLLDB (for debugging)

Create `.vscode/settings.json`:

```
{
    "rust-analyzer.cargo.target": "thumbv8m.main-none-eabihf",
    "rust-analyzer.checkOnSave.allTargets": false,
    "rust-analyzer.cargo.features": ["rt"],
    "rust-analyzer.procMacro.ignored": {
        "cortex-m-rt": ["interrupt"]
    }
}
```

1.5.11 Cargo Configuration for Embedded

Create `.cargo/config.toml`:

```
[target.thumbv8m.main-none-eabihf]
rustflags = [
    "-C", "link-arg=-Tlink.x",
    "-C", "link-arg=-Tdefmt.x",  # If using defmt
]

runner = "probe-rs run --chip EFR32MG24B220F1536IM48"

[build]
target = "thumbv8m.main-none-eabihf"

[env]
DEFMT_LOG = "debug"
```

1.5.12 Git Configuration

Set up Git for embedded development:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Set up LFS for large files (like SVD files)
git lfs install
```

Create `.gitattributes`:

```
# Track large files with Git LFS
*.svd filter=lfs diff=lfs merge=lfs -text
*.pack filter=lfs diff=lfs merge=lfs -text
*.a filter=lfs diff=lfs merge=lfs -text
```

1.5.13 Verifying Your Setup

Run this verification script to ensure everything is correctly installed:

```
#!/bin/bash

echo "=== Checking Rust Installation ==="
rustc --version
cargo --version

echo ""
echo "=== Checking Targets ==="
rustup target list | grep "thumbv8m.main-none-eabihf (installed)"

echo ""
echo "=== Checking Cargo Tools ==="
cargo size --version 2>/dev/null && echo "✓ cargo-size installed"

echo ""
echo "=== Checking probe-rs ==="
probe-rs --version

echo ""
echo "=== Checking debug probe ==="
probe-rs list
```

Expected output:

```
=== Checking Rust Installation ===
rustc 1.70.0 (90c541806 2023-05-31)
cargo 1.70.0 (90c541806 2023-05-31)

=== Checking Targets ===
thumbv8m.main-none-eabihf (installed)

=== Checking Cargo Tools ===
✓ cargo-size installed

=== Checking probe-rs ===
probe-rs 0.18.0

=== Checking debug probe ===
[0]: J-Link Silicon Labs (xxxx) (J-Link)
```

If all checks pass, you're ready to start development!

Common Issues:
- probe-rs list shows nothing: Check USB cable, ensure probe has power
- Target not installed: Run rustup target add thumbv8m.main-none-eabihf
- cargo-size not found: Run cargo install cargo-binutils
- Permission denied (Linux): Add your user to dialout or plugdev group

Exercise (Easy): Create a shell script that automates the entire setup verification process for your team.

Exercise (Medium): Write a cargo subcommand that checks for required tools and provides helpful installation instructions if anything is missing.

Exercise (Hard): Using Docker, create a container with the complete development environment pre-installed so new team members can start immediately.

1.6 Building Your First Embedded Binary

Now that we have our toolchain set up and understand the project structure, let's build and analyze a real embedded binary. This section will guide you through the complete build process with detailed explanations of each step.

1.6.1 Preparing a Complete Example

Let's create a minimal but complete embedded project that we can actually build and analyze:

```
$ cargo new --lib efr32mg24-minimal
$ cd efr32mg24-minimal
$ mkdir examples
$ cat > examples/minimal.rs << 'EOF'
#![no_std]
#![no_main]

use core::panic::PanicInfo;

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}

#[no_mangle]
pub extern "C" fn main() -> ! {
    loop {
        // Main application loop
        cortex_m::asm::nop();
    }
}
EOF
```

Add dependencies to `Cargo.toml`:

```
[package]
name = "efr32mg24-minimal"
version = "0.1.0"
edition = "2021"

[profile.dev]
opt-level = "z"
debug = true
overflow-checks = false  # Critical for embedded

[profile.release]
opt-level = "z"
debug = true
overflow-checks = false
panic = "abort"
lto = true

codegen-units = 1

[dependencies]
cortex-m = "0.7"
cortex-m-rt = "0.7"

# Development dependencies
[dev-dependencies]
cortex-m-semihosting = "0.5"
```

1.6.2 Reference: Complete Project Example

For a comprehensive step-by-step guide with all project files (build.rs, Cargo.toml, memory.x, etc.), see Section **??**.

This section focuses on understanding the build process and analyzing the resulting binary.

1.6.3 Understanding the Build Process

When you run `cargo build --release`, here's what happens:

This diagram shows the build pipeline for embedded Rust:

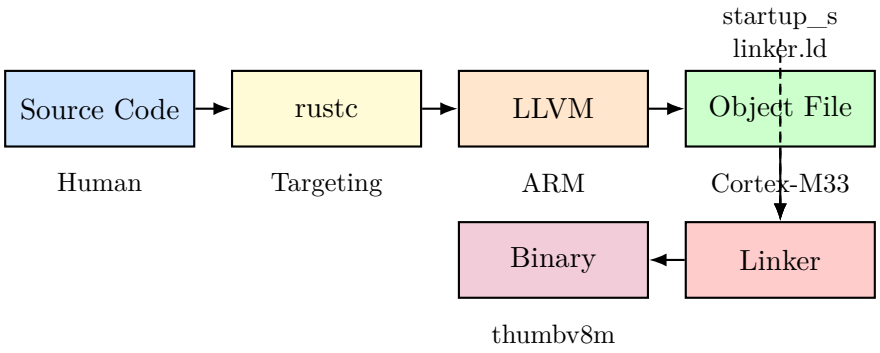


Figure 1.3: Rust embedded build pipeline

1.6.4 Compile Step

```
$ cargo build --release --example minimal
   Compiling autocfg v1.1.0
   Compiling proc-macro2 v1.0.56
   Compiling cortex-m v0.7.7
   Compiling quote v1.0.27
   Compiling cortex-m-rt v0.7.3
   Compiling efr32mg24-minimal v0.1.0
    Finished release [optimized] target(s) in 1.24s
```

Notice that cargo automatically downloads and compiles dependencies. For embedded, this includes startup code and linker scripts via `cortex-m-rt`.

1.6.5 Binary Analysis

Once built, let's examine the binary:

1.6.6 Size Analysis

```
$ cargo size --release --example minimal
   text    data     bss     dec     hex filename
     80       0       0      80      50 minimal
```

What do these sections mean?

- **text**: The actual machine code (read-only)
- **data**: Initialized static variables
- **bss**: Uninitialized static variables (zeroed at startup)
- **dec**: Total bytes (decimal)
- **hex**: Total bytes (hexadecimal)

Our minimal program is 80 bytes total. Let's see what's inside:

1.6.7 Disassembly

```
$ cargo objdump --release --example minimal -- -d
```

Output (simplified):

```
00000000 <main>:
       0: b580            push    {r7, lr}
       2: f000 f802       bl      0xa <_ZN4core4...
       6: bf00            nop
       8: e7fe            b.n     0x8 <main+0x8>

0000000a <HardFault>:
       a: e7fe            b.n     0xa <HardFault>
```

Interpretation:

- `push {r7, lr}`: Save registers
- `bl`: Branch with link (call function)
- `nop`: No operation (our loop body)
- `b.n`: Branch to self (infinite loop)
- `HardFault`: Default exception handler

1.6.8 Listing Sections

```
$ cargo objdump --release --example minimal -- -h
```

Shows memory layout:

```
Idx Name          Size      VMA       LMA       File off  Algn
  0 .text         00000014  00000000  00000000  00000034  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .data         00000000  20000000  20000000  00000034  2**0
                  CONTENTS, ALLOC, LOAD, DATA
  2 .bss          00000000  20000000  20000000  00000034  2**0
                  ALLOC
```

Explanation:

- `.text` starts at 0x00000000 (flash memory)
- `.data/.bss` start at 0x20000000 (SRAM)
- Flash: 1,536 KB on EFR32MG24
- SRAM: 256 KB on EFR32MG24

1.6.9 Build Optimization Comparison

Build with different optimization levels:

Profile	text	data	bss	Total
dev (opt-level='z')	124	0	0	124
release (opt-level='z')	80	0	0	80
release (opt-level='s')	76	0	0	76
release (opt-level=3)	68	0	0	68

Table 1.2: Size comparison by optimization level

Optimization levels:

- `'z'`: Optimize for size aggressively
- `'s'`: Optimize for size
- `'3'`: Optimize for speed (may reduce size too)

Even the unoptimized debug build is only 124 bytes!

1.6.10 Linking and Linker Scripts

The linker script is crucial in embedded systems. It tells the linker where to place code and data in memory.

1.6.11 Memory Layout

Create `memory.x`. The numbers below are the ones this board actually needs, and two of them are safety-critical:

```
MEMORY
{
  /* The Gecko bootloader owns the first 24 KB: an application must not start
     at 0x0800_0000, however tempting the round number looks. */
  FLASH : ORIGIN = 0x08006000, LENGTH = 1512K

  /* SRAM is 256 KB, but the last 4 KB page is the SWD mailbox this project
     reads its results from, so the linker gets 252 KB. */
  RAM : ORIGIN = 0x20000000, LENGTH = 252K
}

/* cortex-m-rt places the stack here. The Arduino core's stack pointer
   (0x20001208) is NOT a memory budget: copying it leaves the linker 4.5 KB and
   any buffer larger than a few kilobytes fails to link with
   "stack end address is not below stack start". */
_stack_start = 0x2003F000;
```

This matches the EFR32MG24B220 memory map:

- Flash: 1,536 KB at 0x0800_0000, of which 0x0800_0000–0x0800_5FFF is the bootloader and 0x0800_6000–0x0817_FFFF is the application
- SRAM: 256 KB at 0x2000_0000, the last 4 KB page reserved as a mailbox

1.6.12 Two Linker Flags That Are Not Optional

```
[target.thumbv8m.main-none-eabihf]
rustflags = [
  "-C", "link-arg=-Tlink.x",
  # Without --nmagic, LLD aligns the first segment to a 64 KB page and emits a
  # PT_LOAD at 0x0800_0000 -- inside the bootloader. Measured on this board:
  # 372 bytes written below the application's own start address.
  "-C", "link-arg=--nmagic",
]
```

The failure mode is worth stating plainly: the ELF looks fine, `cargo build` succeeds, and the only sign is a segment the loader would place where the bootloader lives. `scripts/elf-load-range.py` exists to print exactly that range before anything is flashed:

```
$ python3 scripts/elf-load-range.py target/thumbv8m.main-none-eabihf/release/examples/pac_imu
load_min=0x08006000 load_max=0x08006813 size=2068 segments=3
```

1.6.13 Examining the Linker Output

```
$ cargo build --release --example minimal --verbose 2>&1 | grep "LINK"
```

Shows the linker command:

```
"rust-lld" "-flavor" "gnu"
  "-Tlink.x"
  "-Tmemory.x"
  "-o" ".../target/release/examples/minimal"
  ...
```

Key flags:

- `rust-lld`: LLVM linker for embedded targets
- `-Tlink.x`: Standard Cortex-M linker script
- `-Tmemory.x`: Our custom memory layout

1.6.14 Creating a Flashable Binary

The ELF binary contains debug symbols and metadata. For flashing, we need a raw binary:

1.6.15 Convert to Binary Format

```
$ cargo objcopy --release --example minimal -- -O binary minimal.bin
```

Check sizes:

```
$ ls -lh target/thumbv8m.main-none-eabihf/release/
-rwxr-xr-x  80K minimal          # ELF with symbols
-rwxr-xr-x  80B minimal.bin     # Raw binary for flash
```

The binary is much smaller because it contains only the raw machine code without debug symbols, section headers, etc.

1.6.16 Hex Format for Some Tools

Some tools prefer Intel hex format:

```
$ cargo objcopy --release --example minimal -- -O ihex minimal.hex
```

1.6.17 Automated Build Script

Create a rust file, `build.rs`, in your project root:

```rust
use std::env;
use std::fs;
use std::path::PathBuf;

fn main() {
    // Tell cargo to rerun build script if memory.x changes
    println!("cargo:rerun-if-changed=memory.x");

    // Copy memory.x to output directory
    let out = &PathBuf::from(env::var_os("OUT_DIR").unwrap());
    fs::copy("memory.x", out.join("memory.x")).unwrap();

    println!("cargo:rustc-link-search={}", out.display());
}
```

This ensures the linker script is available during linking.

1.6.18 Build Automation with Justfile

For complex projects, use *just* (a command runner):

```
# Justfile - Build automation

# Build release binary
build:
    cargo build --release --example minimal

# Show binary size
size:
    cargo size --release --example minimal

# Disassemble
disasm:
    cargo objdump --release --example minimal -- -d

# Show section info
sections:
    cargo objdump --release --example minimal -- -h

# Convert to binary
binary:
    cargo objcopy --release --example minimal -- -O binary minimal.bin

# Run all analysis
analyze: build size disasm sections binary
    @echo "Analysis complete!"
```

Usage:

```
$ just build
$ just size
$ just analyze
```

1.6.19 Key Takeaways

- Embedded Rust binaries are incredibly small (our minimal example is 80 bytes)
- Optimization settings significantly impact size (from 124 to 68 bytes)
- The build process involves: compile → link → optionally convert format
- Linker scripts control memory layout and are critical for embedded
- Cargo provides powerful tools for analyzing binaries (size, objdump, objcopy)
- The ELF format contains metadata; for flashing, use binary or hex format
- Scripting your build workflow improves productivity

Exercise (Easy): Create a project template with a complete build script that:

- Builds release binary
- Shows size information
- Creates both .bin and .hex files
- Runs objdump analysis

Exercise (Medium): Compare binaries built with different optimization levels:

- What differences do you see in objdump output?
- How much code can the linker remove from our minimal example?
- Where does libc start inserting startup code?

Exercise (Hard): Write a custom linker script that:

- Places the vector table at a specific address
- Creates a separate section for time-critical code
- Reserves space at the end of RAM for a stack canary
- Adds checksums for firmware verification

Then verify it works with objdump.

1.7 What Goes In Each Crate, and Why

The PAC/HAL/BSP pattern from Section 1.3 says the layers exist. It does not say where one ends and the next begins, and every project fills that gap differently. That gap is what you face the first time you add support for a new MCU or a new board — which is the situation this book is written for, and the one it should leave you equipped to handle.

This section argues the choice, compares it with what four mature Rust embedded ecosystems do, gives a decision procedure you can apply to hardware that is not in this book, and states the rules that keep the answer from drifting.

1.7.1 The Question, Stated Sharply

Three pieces of code make the ambiguity visible. All three are real, from this project:

1. **A pin pair.** The IMU on this board answers on `SDA = PB2`, `SCL = PB3` — the opposite of what the vendor's pin map declares. Whose knowledge is that?
2. **A power gate.** The same sensor does not answer until `PD5` is driven high. Is that a driver detail or a board detail?
3. **An algorithm.** A fall classifier takes acceleration samples and decides whether the device fell. It has no registers, no pins and no clock. Where does it live so that it stays testable?

A HAL that answers (1) and (2) knows too much: every new board becomes a fork of the HAL. An example that answers them re-derives them, and the same investigation gets repeated by the next person — the failure this project has already paid for once.

1.7.2 Three Candidate Layouts

Layout A — no BSP at all; wiring lives in the examples. This is the smallest possible structure: the HAL offers drivers, and each example says which pins to use. It is what the STM32 and ESP32 ecosystems do, and it is defensible when the point of the example is to teach the peripheral.

Its cost appears the moment the wiring is not obvious. This board's IMU needed a swapped `SDA`/`SCL` and a power gate before it answered anything; putting that in every example means every example can get it wrong independently, and the knowledge is only as good as the most recently written one.

Layout B — one BSP crate, features for the optional hardware. A single `xiao-mg24-bsp` with features `sense` and `expansion-base`. It looks like the cheapest correct answer, and it has a specific trap that the nRF ecosystem documents in its own repository: *cargo unifies features across a workspace*, so a build that enables the base for one target can silently enable it for another. More importantly, the API lies: a `ExpansionBase` type that exists in a build without the base has to answer "no OLED" at runtime, and every method that could have been a compile error becomes an `Option`.

Layout C — one BSP crate per board, and a feature for a variant of one board. The module gets its own crate (LED on both variants; IMU and microphone behind `sense`), the Expansion Base gets its own crate that depends on the module's, and the dependency arrow points one way. A user who does not own the base never compiles it, and never sees an `Option` that only exists because two boards were welded together.

1.7.3 What Four Ecosystems Actually Do

Read from the repositories, not from memory:

ecosystem	chip selection	BSP crates	where examples live	where wiring is written
rp-hal (RP2040/RP2350)	one HAL per chip	**yes**, one per board, in a *separate repository* (`rp-hal-boards/boards/`)	in each BSP, plus separate example crates	in the BSP, once
nrf-hal	one HAL per chip (`nrf52840-hal`, `nrf9160-hal`, …) + `nrf-hal-common`	no	in the HAL repo, board chosen by feature	in the examples
stm32f4xx-hal	one HAL per family	no	in the HAL crate	in the examples (pin numbers inline)
esp-hal	one crate, chip by feature	no	one `examples/` grouped by capability	in the examples

Table 1.3: How four Rust embedded ecosystems split PAC, HAL, BSP and examples. Verified from the repositories in November 2026.

Two details in that table are worth more than the table.

First, **rp-hal puts its BSPs in a different repository from the HAL**. That is stronger than a folder: the HAL's continuous integration cannot depend on a board, because the board is not present. The BSP crates describe themselves as a HAL for one board that re-exports the chip HAL — a facade, exactly as this book calls it.

Second, **nrf-hal ships a workspace template instead of a workspace**, and says why: *"Since Cargo resolves all features for all crates in the workspace together, it is not possible to have a workspace with all the crates together, since they enable incompatible features."* That sentence is the argument against Layout B in the ecosystem's own words.

1.7.4 The Decision, and the Rule Behind Each Part

crate / folder	what goes in	what must never go in
`efr32mg24-pac`	registers, generated from the SVD	anything hand-written that a generator could produce
`efr32mg24-hal`	drivers: which bytes, in which order, with which timing	pins, addresses, power gates, board names
`xiao-mg24-bsp`	the module: LED; behind `sense`, the IMU bus, its power gate and the microphone	any device that is not on the module
`xiao-expansion-base-bsp`	the base: OLED, BME68x, RTC, buzzer, button, Grove, microSD, and the base's own bus	anything that exists without the base
`libraries/`	domain policy with no hardware at all	any dependency on the hardware stack, direct or transitive

Table 1.4: Ownership. The right-hand column is what the gate checks; the left two are what a review checks.

Three rules follow, and each one exists because of a specific failure:

- **A BSP describes one piece of plastic.** Two boards that stack are two crates, because a user may own only one of them.
- **A variant of one board is a feature, not a crate.** The Sense module is the same PCB with more parts fitted; a separate crate would duplicate the board init and the LED to add two sensors.
- **A dependency arrow never points at a board.** The base depends on the module; the module does not know the base exists. This is the property that makes "does it work without the base?" answerable by the compiler instead of by a test matrix.

1.7.5 Where the Algorithm Goes

The classifier deserves its own sentence because it is where layering usually breaks. It has no hardware, but the temptation is to put it "near" the driver that produces its input — in the same crate as the IMU, or in the example that uses it.

Both are wrong for the same reason: the moment it shares a crate with anything that needs a target triple, its tests need a target triple, and the tests are what keep it honest. In this project it is a crate with **zero dependencies**, and the gate fails if its dependency tree ever contains a hardware crate:

```
$ cargo tree -p motion-detection | grep efr32mg24
$ echo $?
1        # nothing from the hardware stack: the rule holds
```

The contract is worth quoting, because it is the whole testability argument in one line:

```
// [i32; 3] of milligravity, and the interval between samples in microseconds.
pub fn update_us(&mut self, accel_mg: [i32; 3], dt_us: u32) -> Option<Event>
```

No sensor type, no bus, no clock. Which is why nine tests run in under a second on a laptop, and why the same code can be replayed against a recording at ten times real speed.

1.7.6 And Where the Examples Go

The layout of the examples follows the same principle as the crates, with one addition for hardware that stacks: **an example lives in the crate of the largest hardware requirement it has, and states that requirement in its first comment.**

```
crates/xiao-mg24-bsp/examples/            runs with the module alone on the desk
crates/xiao-expansion-base-bsp/examples/  needs the module (Sense) and the base
crates/efr32mg24-pac/examples/            chip level: any pin is legitimate
```

The alternative — every example in one folder — looks tidier and is worse: the crate of the module would have no examples, and an API that nothing builds is an API that nothing checks. A crate with hardware features and zero examples should fail a gate, not pass it quietly.

1.7.7 A Decision Procedure for Hardware Not In This Book

The four questions below decide the layout. They take minutes and they are the difference between a structure that survives the second board and one that has to be refactored.

1. **How many chips?** If more than one, the HAL is per chip (or per family, when the parts are close enough that one driver set serves them). This is what `nrf-hal` does per chip and `stm32f4xx-hal` does per family; `esp-hal` instead keeps one crate and selects the chip by feature, which works when the vendor's crates already share their register code.

2. **How many boards, and can one sit on top of another?** Boards that are alternatives become one BSP crate each. Boards that *stack* become one crate each *plus* a one-way dependency: the carrier depends on the module. A single crate that knows both cannot answer "does this work without the carrier?".

3. **Are there variants of the same board?** A variant — same PCB, different parts fitted — is a *feature*, not a crate. A separate crate duplicates the board init and everything the variants share, in exchange for a name that repeats what the feature flag already says.

4. **Is the wiring obvious or hard-won?** Obvious wiring (an LED on a documented pin) may live in an example. Hard-won wiring — a swapped bus, a power gate, a pin that the vendor's own map gets wrong — belongs in a BSP, once, with the measurement recorded next to it. This is the question that decides between Layout A and Layout C for most hobby-scale projects.

The same questions decide where an algorithm goes, with one addition: **if it can be tested on a host, it must be**, so it cannot share a crate with anything that needs a target triple.

1.7.8 Adding Support for a New Chip or Board: the Order of Work

Having decided the layout, the order below is what this project would follow again. Each step produces something checkable before the next one starts.

1. **Registers first.** Obtain the SVD, generate the PAC, and build one example that only writes a register and reads it back. If the SVD is wrong, everything downstream inherits the error, and a five-line example finds that in minutes.

2. **Clock and pins.** Prove that the peripheral clock gate is on and that a GPIO changes state. This is the step where a missing `CMU\_CLKEN0` bit appears as a bus fault at the first register access.

3. **One transport.** Bring up the simplest bus the board has, and read a device identifier through it. A chip identifier or a WHO_AM_I is a free proof that addressing, timing and the byte order are all correct.

4. **Drivers, in the HAL, with pins as arguments.** Nothing in this layer may name a port. If a driver needs a pin, take it as a parameter; the BSP will supply it.

5. **The BSP for one board, and its examples.** Put the wiring in one place, in the crate named after the board whose documentation describes the feature, and write the example that proves the board works end to end.

6. **A second board, if there is one, and the dependency direction.** Add the carrier crate only after the module crate stands alone, and make the dependency point one way. If it cannot, the split is wrong.

7. **The gate.** Before more examples are written, make the machine check what the review would check: zero warnings, every example builds, every domain crate tests on the host, and no domain crate depends on the hardware stack.

The step people skip is the first one. Generating the PAC and immediately writing drivers feels productive, and it hides an SVD error until the driver is complicated enough that the error looks like a driver bug. The five-line register read-back is the cheapest test in the whole project.

1.7.9 What This Costs

Honesty about a design decision includes its price:

- **More manifests.** Two BSP crates mean two `Cargo.toml` files, two example folders and a dependency between them that has to be kept one-way.
- **Some knowledge is duplicated on purpose.** The clock setup lives in the module crate and the base depends on the module rather than re-initialising; a project with two unrelated boards could not share it this way.
- **The split is a judgement, not a theorem.** "Which plastic is this pin on?" is unambiguous for an LED and less so for a Grove connector that reaches the base through the module's castellated pads. The rule that resolves it: the BSP is named after the board whose *documentation* describes the feature.

The test to apply to your own board. Ask which of your examples would still make sense to someone holding only the chip. If the answer is "none of them", the board has swallowed the crate and a split is due. If the answer is "all of them", you may not need a BSP at all — Layout A is not a mistake, it is a choice with a cost that is sometimes worth paying.

1.8 The Boundaries the Compiler Cannot See

The previous section said what belongs in each crate. This section is about what happens when the answer is written down, agreed, and then quietly violated — because *the compiler has no idea that a layer boundary exists.* Every example in this section compiled. Every test passed. Every lint was clean. That is what makes this class of defect expensive: there is no error message, only a slow loss of the property the layering was supposed to provide.

1.8.1 The Question That Finds Them

There is a single question that finds these defects, and it is cheap to ask:

> **For this public signature, what does it force the caller to own?**

A function that takes a register block forces the caller to have a handle to that register block. A function that takes the whole peripheral singleton forces the caller to hold the singleton. Both compile. Both are wrong the moment the caller is a layer above: the caller now has to reach for something the layer below was supposed to keep to itself.

A real example, from this project. The HAL's buzzer and PWM constructors took the peripheral singleton:

```rust
// Before: the caller has to own the PAC to start a tone.
pub fn new(dp: &pac::Peripherals, port: Port, pin: u8) -> Option<Self>
pub fn take(dp: &pac::Peripherals, port: Port, pin: u8) -> Option<Self>
```

The signature looks innocent — it is the idiom of every Rust HAL, and it is how exclusivity is proven at compile time. The consequence only showed up two layers up, in an example that had already called `Board::take()`:

```rust
// The example could not compile without this line, and nobody wrote down why.
let pac = unsafe { efr32mg24_pac::Peripherals::steal() };
let timer = Timer0::take(&pac, Port::A, 7);
```

An example that has to steal the singleton is not an example with a problem; it is a HAL that failed to hand out what it already owned. The fix is not to teach the example to steal more politely. The fix is a door on the layer that owns the thing:

```rust
// After: the board owns the PAC once, and hands out what a caller needs.
let mut board = Board::take().expect("peripherals taken twice");
let mut led = board.pwm_on(Port::A, 7, 1_000)?;   // no PAC in sight
let mut mic = board.converter().microphone()?;    // a handle, not a register block
```

The two constructs that make this work are worth naming, because they generalise:

- **A constructor on the owner.** `Board::pwm_on` is one line: it calls the private `Timer0::take` with the singleton the board already holds. The information did not have to travel; it was already below.
- **An opaque handle.** `microphone::Converter` is a `Copy` wrapper around the register block, with a private constructor and one public method, `microphone()`. The driver takes the handle; the caller never sees the type inside it.

1.8.2 The Same Defect, One Layer Down

Once you have the question, it stops being about the PAC. Three examples in the same project reached for a *core* peripheral — SysTick for a delay, DWT for the cycle counter:

```rust
let mut delay = Delay::with_sysclk(unsafe { cortex_m::Peripherals::steal().SYST }, 19_000_000);
let mut core = unsafe { cortex_m::Peripherals::steal() };
core.DCB.enable_trace();
core.DWT.enable_cycle_counter();
```

Exactly the same shape: the layer that owns SysTick is the board, and it was not offering a door. Worse than a style problem — two owners of SysTick is a delay that lies, and a cycle counter that was never enabled reads as zero, which looks exactly like a measurement of a very fast loop. This project lost a full round of fall-detection data to that second one: the loop declared 400 Hz and ran at 158 Hz, and the only clue was that four different drops all showed the same 0.62 s of calm.

The fix, again, is doors rather than discipline:

```rust
let mut delay = board.delay();          // SysTick, handed out by its owner
let start = unsafe { dwt::start() };    // the cycle counter, switched on in one place
let elapsed = dwt::since(start);
```

1.8.3 A Layer That Exports the Layer Below

The most damaging version of the same mistake is a re-export, because it does not look like code at all:

```rust
// in a board support package
pub use efr32mg24_pac as pac;
```

One line, entirely reasonable-looking, and it means every application can reconfigure every register the board claimed to own. The board's API becomes advisory: the pins it configured, the power gates it enabled, the clock tree it chose — all of it can be undone by a caller that has the PAC. A BSP that re-exports its PAC is not a BSP; it is a suggestion.

The same crate carried a second, subtler version: *two facades at once.* A newer one (`Module`, the LED and the on-board sensors) and an older one (`Board` with `Pins` and a type-state GPIO layer) both existed, both public, and the examples had split between them. Half the code proved one, half proved the other, and nothing exercised the older one — which is the failure mode that matters: **an API with zero examples passes every check trivially**. It compiled, it was documented, and it had been dead for weeks.

1.8.4 Why It Kept Happening: The Table

Seven defects of this class were found in a single pass over a codebase that built cleanly. What they have in common is the column that says why nothing complained.

Defect	Why it compiled	What it cost
HAL constructors took a singleton or a register block	The signature is legal Rust, and idiomatic	examples had to `steal()` the PAC
BSP re-exported the PAC	A `pub use` is invisible in review	the board's promises became advisory
Two facades in one BSP crate	Both compiled; neither was unused by the compiler's standards	half the examples proved a dead API
`Board::gpio()` was public	Returning a register block is legal	callers reconfigured pins the board owned
Two GPIO implementations in one HAL	Different modules, both public, no overlap warning	605 unproven lines, and docs naming the wrong pin
Documentation outlived the code	Prose is not compiled	78 MHz, 256 KB, and an API that did not exist
Examples stole SysTick and DWT	`steal()` is `unsafe` but compiles	a delay that lied, a counter that read zero

Table 1.5: Seven layer violations in a codebase where cargo build, cargo test and cargo clippy were all green.

1.8.5 Writing the Boundary Down as a Check

A boundary that is only a paragraph is a boundary that will be crossed, and this project crossed all seven. The rules only started holding when each one became a line in the gate script — the same script that checks formatting and tests, so that it runs when nothing else does:

```sh
# 1. An example may only name the layer it demonstrates.
grep -rln 'efr32mg24_pac|pac::' crates/*/examples/ | grep -v '^crates/efr32mg24-pac/'
# 2. A BSP must not re-export the PAC.
grep -rln 'pub use efr32mg24_pac' crates/*/src/ | grep -v '^crates/efr32mg24-hal/'
```

```
# 3. The HAL must not put a register block in a public signature.
grep -rqE 'pub (fn|const fn) [^\n]*::RegisterBlock' crates/efr32mg24-hal/src/
# 4. No example may steal a core peripheral.
grep -rln 'Peripherals::steal' crates/*/examples/
# 5. Every crate carries at least one example of its own level.
for dir in crates/*/ libraries/*/; do find "$dir/examples" -maxdepth 1 -name '*.rs'; done
```

Each of those five checks failed at least once during the pass that wrote them. That is the argument for having them: not that the code was wrong, but that the code was wrong in ways that had already survived review.

1.8.6 What Is Not a Violation

The distinction that took the longest to state clearly is between a *token* and a *register block*:

```
// Legitimate: the singleton token proves exclusivity at compile time.
pub fn new(usart: pac::Usart0S, config: Config, clocks: &FrozenClocks) -> Self

// A violation: the caller receives the registers themselves.
pub fn new(adc: &pac::iadc0_s::RegisterBlock) -> Result<Self, MicError>
```

Both mention `pac`. The first is how every Rust HAL on the market works — `dp.USART0` in `stm32f4xx-hal`, `peripherals.USART0` in `rp2040-hal` — and it is the mechanism that makes it impossible to create two USART drivers for one USART. The second hands over the hardware itself, with no ownership at all: any number of callers may now configure the same converter, and the driver's invariants are whatever the last writer decided.

The rule, then, is not "never mention the PAC in the HAL". It is: **the HAL may take a token, and may never give out registers**.

1.8.7 The Procedure, for the Next Project

The pass that found seven defects was mechanical, and it transfers:

1. **List every public signature** in every crate, oldest and least used first. The dead API is where the violations accumulate, because nothing compiles against it.
2. **Ask the question** of each one — what does this force the caller to own? — and write the answer down next to the signature.
3. **Close each leak with a door on the owner**: a method on the struct that already holds the thing, or an opaque handle for what must be passed through.
4. **Turn each rule into a check** in the gate that already runs, and confirm the check fails on the old code before accepting it. A check that has never been red has not been tested.

The last step is the one that decides whether the work survives. This project had already written the layering down — in the README, in this book, in the crate documentation — and had violated it seven times anyway. What changed the outcome was not a better description of the layers. It was five greps that fail the build.

- A layer boundary is invisible to the compiler: the code compiles either way, which is why this class of defect survives review.
- The question that finds them is *what does this force the caller to own?*
- A HAL may take a peripheral *token* to prove exclusivity, and may never hand out *registers* — the difference decides whether a driver's invariants mean anything.

- A BSP that re-exports the PAC makes its own configuration advisory.
- An API with zero examples passes every check trivially; it is dead and nobody notices.
- A rule that is only prose will be broken. A rule that is a line in the gate script is a rule that holds.

1.9 The Silicon Labs Toolchain, From the Outside

Everything in this book is built with open tools: `cargo`, `rustc`, `probe-rs`, `openocd`, `llvm-objdump`. And yet every binary in it would be impossible without a vendor: the register definitions come from Silicon Labs' SVD, and the radio stacks are their binary blobs. The vendor's tools, therefore, are not optional — but most of them are not needed either, and knowing which is which saves weeks.

This section is about that boundary: what Silicon Labs ships, what each piece is actually for, and what this project measured about the three that touch a Rust workflow.

1.9.1 The Pieces, and What They Are For

Piece	What it is	Does a Rust path need it?
Simplicity Studio 5	IDE, part database, board files, configurators	No for building; **yes** for the Radio Configurator
Gecko SDK (now Simplicity SDK)	headers, emlib C sources, prebuilt stack libraries	**Yes**: SVD, and the blobs
Simplicity Commander	vendor CLI for flashing, locking, reading flash	No, and it is actively dangerous here (see below)
SEGGER J-Link	external debug probe, licensed	No: the board has its own debugger
CMSIS-DAP on the board	the SAMD11 chip that *is* the debugger	**Yes**, via `probe-rs` or `openocd`
Gecko Bootloader	the factory image at the base of flash	**Yes**, and it constrains every linker script
AppLoader UART API	OTA over a serial line	Only for OTA; it is a protocol, not a tool
Radio stack blobs	`libbluetooth_*`, `librail_*`	**Yes**, under MSLA licensing

Table 1.6: The vendor surface. Two things are unavoidable — the SVD and the radio blobs — and everything else is a convenience or a hazard.

1.9.2 JTAG, SWD, and the Debugger Already on the Board

There are three ways to reach a Cortex-M core, and the naming confuses two of them:

- **JTAG** — the five-wire standard (TCK, TMS, TDI, TDO, nTRST). Everything supports it, nothing needs it on a modern single core.
- **SWD** — two wires (SWCLK, SWDIO). It is what the EFR32MG24 uses, and what every board-level debug connector exposes.
- **cJTAG** — two-wire *JTAG*, a different protocol that happens to use two pins. Some Silabs documentation mentions it; it is not what `probe-rs` speaks.

The XIAO MG24 Sense has no debug header. It has a second microcontroller — a SAMD11 — wired between the USB connector and the EFR32's SWD pins, and that chip implements the CMSIS-DAP protocol (USB `2886:0062`). Two consequences, both measured:

- `probe-rs` sees the EFR32 through that bridge and can program it. So can `openocd` with `interface/cmsis-dap.cfg`. Neither needs a J-Link.
- The bridge is *flaky* in a specific way: the first attach works, and a later one dies with `Command ID in response (0x14) does not match sent command ID`. A USB reset does not clear it; unplugging the cable does. That is a fact about the SAMD11, not about your program, and it will waste an afternoon if you assume otherwise.

1.9.3 The Bootloader Is Not Optional, and It Is Why Linker Scripts Matter

Every XIAO MG24 Sense leaves the factory with a Gecko Bootloader at the base of flash. It occupies the first 24 KiB, and on this part the application therefore starts at **0x08006000**, not at `0x08000000`.

That single fact decides the linker script, and getting it wrong does not produce an error message — it produces a board that no longer boots, because the image was written over the bootloader. The symptom is subtle enough to be worth stating in full: the write succeeds, the read-back matches, and the board is silent.

Hence the two requirements this project imposes on itself, both mechanical:

```
# 1. never let the linker put anything below the bootloader
FLASH (rx) : ORIGIN = 0x08006000, LENGTH = 1536K - 24K

# 2. refuse to flash an image that would
python3 scripts/elf-load-range.py "$ELF"     # load_min must be >= 0x08006000
```

1.9.4 Simplicity Commander: Convenient, and the Wrong Tool Here

`commander` is the vendor's CLI: it flashes, reads flash, locks and unlocks parts, and writes the bootloader. It is well documented and it does not check where your image says it goes. A `commander flash image.hex` writes whatever the hex file describes — including, if the linker script is wrong, 0x08000000.

For a workflow that must not brick a board, that is the wrong property. The alternative this project uses is a script that (1) inspects the ELF's load range, (2) refuses anything starting below 0x08006000, (3) writes with `probe-rs download`, and (4) verifies with `probe-rs verify`. The guard is the point; the tool is replaceable.

1.9.5 The Radio Stack: Two Licences in One Directory

This is the part of the vendor surface that a Rust project cannot avoid and cannot reimplement.

- **Headers and emlib sources** are Zlib licensed: usable, redistributable, and readable. `sl_bt_api.h` is where the BGAPI command set is defined, and `rail.h` is where RAIL is.
- **The stacks themselves** are binary: `libbluetooth_host`, `libbluetooth_controller`, `libbgcommon` for Bluetooth, and `librail` for the proprietary/802.15.4 side. They are MSLA licensed, which is why they are not in this book's repository and why a clean checkout does not link.

Two traps follow from that split, and both cost real time in this project:

The header and the blob must be the same SDK version. When they are not, the blob exports symbols the bindings do not declare — `sl_bt_advertiser_create_set` was in the blob and absent from the generated bindings — and the failure looks like a typo in your code. It is not.

A blob references code that is not in the blob. The Bluetooth stack calls the CMSIS `CORE_*` helpers; RAIL calls four emlib functions (`CMU_ClockEnable`, `EMU_DCDCSetPFMXTimeoutMaxCtrl`,

`SYSTEM_ChipRevisionGet`, `SystemSYSCLKGet`). The application is expected to provide them, because in a vendor project the linker pulls them from emlib. In a Rust project they have to be provided deliberately — and the CMSIS ones must be provided *once*, for both stacks, because two definitions of `CORE_EnterCritical` either collide or, worse, disagree about which interrupt mask they set.

1.9.6 The Radio Configurator, and Why RAIL Needs a Generated File

RAIL is a library, not a radio. Before it can transmit it needs a *radio configuration*: channel numbers and their modem settings, the PA and front-end tables for this board's matching network, and the calibration data for the crystals actually fitted. Nothing in the header contains those numbers, and no amount of reading the reference manual produces them.

They are generated. The Radio Configurator inside Simplicity Studio emits `rail_config.c` from a description of the board and the desired protocol, and that file is the difference between "the library initialised" and "a packet left the antenna". This is the one place where the vendor IDE is genuinely required, and it is worth saying plainly: a Rust project can avoid Studio for editing, building, flashing and debugging, and cannot avoid it for radio.

1.9.7 What To Take Away

- The vendor tools split into: *data you need* (SVD, headers, emlib, blobs), *a file you cannot write yourself* (the radio configuration), and *conveniences that fight your safety rules* (Commander).
- The board's own debugger is a CMSIS-DAP bridge; SWD is the two-wire protocol; J-Link is unnecessary, and the bridge needs a physical replug when it desynchronises.
- The bootloader owns the bottom of flash. A linker script that ignores it bricks the board silently, so the guard belongs in a script that runs before the write.
- Zlib headers plus MSLA blobs is a normal arrangement, and it has two failure modes worth remembering: version mismatch, and symbols the blob expects the application to supply.

- Rust provides memory safety without sacrificing performance through its ownership system
- Embedded Rust programs use `no_std` and `no_main` for bare-metal operation
- The PAC/HAL/BSP architecture provides clean separation of concerns and promotes code reuse
- Cross-compilation requires the correct target specification (thumbv8m.main-none-eabihf for EFR32MG24)
- Building embedded binaries involves multiple steps: compile, link, and optionally convert to binary/hex format
- All necessary tools (rustup, probe-rs, cargo-binutils) are available cross-platform
- Our minimal program compiles to just 80 bytes in release mode
- A layer boundary is invisible to the compiler; it holds only when a check fails the build
- The vendor surface splits into data you need, one file you cannot write yourself, and conveniences that fight your safety rules

2. Understanding SVD Files and the PAC Generation Process

> *"The SVD is the only document about this chip that cannot be out of date, because the vendor generates the same file the debugger uses."*

Before a single line of HAL can be written, something has to know what registers exist, where they live, which bits mean what and what the reset values are. On ARM that something is an SVD file, and the tool that turns it into Rust is `svd2rust`.

This chapter is not a tutorial on running `svd2rust`. It is about the two things that actually matter: what the generated code looks like when you have to read it, and what the SVD knows that no other document does — including the register that explained why a week of work had produced zero results.

2.1 What an SVD Actually Contains

An SVD (System View Description) is an XML file — CMSIS-SVD, a standard — that describes a microcontroller's memory-mapped surface. It exists because every debugger needs the same information and nobody should re-derive it per vendor.

2.1.1 The Five Things That Matter

Element	What it says	Where you meet it in Rust
`peripheral`	base address and register layout	`Periph<T, ADDR>`
`register`	offset, size, access (r/w/rw)	`registers.read()`, `.write()`
`field`	bit offset, width, access	`w.mvp().set\_bit()`
`enumeratedValue`	named values, and their descriptions	the enum in the field writer
`resetValue`	what the hardware holds after reset	`reset()`

Table 2.1: The five elements that produce all the code you will actually call. Everything else in an SVD — interrupts, dim arrays, clusters — is scaffolding around them.

2.1.2 A Register, in the SVD and in Rust

The MVP's command register, as the SVD describes it:

```
<register>
  <name>CMD</name>
  <addressOffset>0x174</addressOffset>
  <fields>
    <field><name>START</name><bitOffset>0</bitOffset><bitWidth>1</bitWidth>
           <description>Start Command</description></field>
    <field><name>INIT</name><bitOffset>3</bitOffset><bitWidth>1</bitWidth>
           <description>Initialization Command/Qualifier</description></field>
    ...
```

and what that becomes, which is what you type:

```
// One write, two fields: the qualifier and the command together.
mvp.cmd().write(|w| w.init().set_bit().start().set_bit());
```

The word *Qualifier* in the description is the whole interface contract for that register: it says INIT modifies the meaning of the command in the same write rather than being a separate command of its own. Reading the generated code is reading the SVD.

2.1.3 Where the SVD Comes From, and Why It Is Trustworthy

Silicon Labs publishes SVD files per part, and their own tools build register views from the same file. That is the reason to trust it more than a PDF: a manual is written once and then slowly drifts, while an SVD that disagrees with the hardware breaks the vendor's debugger on the same day.

It is still not a specification of the *board*. Three of the errors documented in section 4.5 are board facts — which pin the LED is on, which way the I2C lines go — and no SVD contains them. Chapter 4's rule applies: chip facts from the SVD, board facts from a measurement.

2.1.4 The Practical Consequence: One File, One Crate

The project's PAC is generated once and checked in, because a build that downloads an SVD from the network is a build that stops working when the network does. Regeneration is a deliberate act:

```
# the SVD lives beside the crate it generates
ls vendor/EFR32MG24B220F1536IM48.svd
svd2rust -i vendor/EFR32MG24B220F1536IM48.svd --target cortex-m
# then split the generated lib.rs into one module per peripheral, which is what
# makes `cargo doc` usable and keeps a 40k-line file reviewable
```

That split is why this repository's PAC is a directory of small modules rather than one enormous file: `mvp\_s/`, `gpio\_s/`, `cmu\_s/`, and so on. It costs a script and saves every future diff.

2.2 What svd2rust Emits, and How to Read It

The generated PAC looks impenetrable because it is generated: names are long, types nest three deep, and every field has a writer type. It is worth ten minutes to learn the four shapes, because after that the generated code is the best documentation the chip has.

2.2.1 Shape 1: The Peripheral

```
pub type MvpS = crate::Periph<mvp_s::RegisterBlock, 0x4d00_0000>;
```

A peripheral is a zero-sized handle carrying its base address as a const generic. That is what makes `Peripherals::take()` sound: the singleton exists once, and each field is a handle with a compile-time address.

2.2.2 Shape 2: The Register Block

```
#[repr(C)]
pub struct RegisterBlock {
    ipversion: Ipversion,     // 0x00
    en: En,                   // 0x04
    swrst: Swrst,             // 0x08
    cfg: Cfg,                 // 0x0c
    status: Status,           // 0x10
    ...
}
```

The field order is the address map, and the doc comment on each accessor carries the offset — the one piece of information you need when comparing code against a manual page.

2.2.3 Shape 3: Read, Write, Modify

Every register has three call styles, and choosing the right one is a safety decision, not a style preference:

```
// read: no side effects on the hardware
let bits = mvp.status().read().bits();

// write: sets every field in the register, the others to their writer default
mvp.en().write(|w| w.en().set_bit());

// modify: read-modify-write, preserving fields you did not name
gpio.portc_model().modify(|_r, w| w.mode3().pushpull());
```

Chapter 4's central defect — a `write` where a `modify` was needed, which disabled fifteen other pins — is visible in these three signatures and invisible in the code that calls them. That is why the rule is stated at the level of the register, not of the driver.

2.2.4 Shape 4: The Field Writer

The most useful generated code is the field writer, because it is where the SVD's enumerated values become a Rust enum:

```
// From the SVD: <field><name>CMOA</name> with values None/Toggle/Clear/Set
mvp.cc0_ctrl().modify(|_r, w| w.cmoa().toggle());

// And when the SVD says a register has no such field, the compiler says so:
// error[E0599]: no method named `pwm` found for `FieldWriter<.. Cmoa, ..>`
```

That second snippet is from this project. It is a compile error that saved a silent misconfiguration: the timer's compare-action field genuinely has four values, and PWM is not one of them — the PWM waveform in this part comes from the register's *mode*, not from the action field. A HAL written against a guess would have written a valid-looking bit pattern and configured the wrong waveform.

2.2.5 Unsafe, and Where It Is Honest

```
mvp.instr0cfg0().write(|w| unsafe { w.bits(0x0102_0000) });
```

`bits()` is unsafe because the type system cannot know whether the value you assembled means what you think. That `unsafe` is the correct amount of friction: it appears exactly where a field-level API does not exist, and its presence in a diff is a signal that the code is at the register level on purpose.

2.3 The SVD as an Oracle

This section is the reason to read generated register accessors before writing a driver. In this project it turned two dead ends into two solved ones, in minutes, after days of guessing.

2.3.1 Case 1: The Accelerator That Never Ran

The matrix-vector processor (MVP) was configured exactly as the public material suggested: arrays filled, dimensions written, one loop, one instruction, then `CMD = START`. It never ran. The result array stayed zero, the status never returned to idle, and nothing faulted — the most expensive kind of failure, because there is nothing to read.

Two things came from the SVD, in this order:

1. **The block has its own enable register.** The generated code contains an `EN` register at offset `0x04`, described as *Block Enable*, with bit 1 as `DISABLING`. The firmware had enabled the bus clock (`CMU\_CLKEN1.MVP`) and never set `EN`. A clock gate is not an enable: with the block disabled, configuration writes are dropped and the command goes nowhere.

2. **The command register's INIT is a qualifier.** The field description says "Initialization Command/Qualifier", which means it modifies the command written in the same access — not a separate initialisation pass, which is what the firmware assumed it had to do.

Neither fact is in the tutorial material this project started from. Both are in the file the vendor's debugger uses.

2.3.2 Case 2: A Register That Is Not There

The GPIO modes taught the same lesson in the other direction. The generated `portb\_model` and `porta\_model` are *different types*:

```
// PORTA and PORTC: sixteen pins, so MODEL and MODEH.
// PORTB and PORTD: eight pins on this package, so MODEL only.
// error[E0609]: no field `portb_modeh` on type `&pac::gpio_s::RegisterBlock`
```

A hand-written header would have declared `MODEH` for all four ports, because that is what the family's documentation describes. The SVD describes *this part*, and the compile error it produces is a fact about the package rather than a limitation of the tool.

2.3.3 The Habit

1. Before writing a driver, read the generated module for the peripheral. The doc comments carry the offsets and the field names as the silicon team wrote them.
2. Grep the generated code for the register you think you need. If it is not there, the peripheral does not have it on this part — not "the PAC is incomplete".
3. When behaviour disagrees with expectation, read the *neighbouring* registers. In the MVP case the answer was four bytes below `CFG`: a register that existed, had a name, and was never written.
4. Write what you learn into the crate's docs, not into a chat window: the next person reads the accessor's doc comment, not your transcript.

2.4 Why PAC-Level Examples Exist Anyway

An example that pokes registers directly is an antipattern in a project that has a HAL. It duplicates knowledge, it names pins inline, and it is exactly the code a reader should not copy into a product. This project keeps nineteen of them, deliberately, and this section is the argument.

2.4.1 The Ladder

The same demo exists at four levels, and the differences are the whole curriculum:

Example	Lines	What a reader has to know to change it
`pac\_led\_blink`	90	the port's MODEL and DOUT registers, and its clock gate
`hal\_led\_blink`	50	that a pin has a port, a bit and a polarity
`module\_led\_blink`	35	that the board has an LED
`base\_led\_blink`	35	that the module is on a carrier

Table 2.2: One program, four rungs. Each rung removes a fact the reader must hold in their head — and each rung is only trustworthy if the rung below it is still testable.

2.4.2 What the PAC Level Proves

The bottom rung is not a tutorial. It is an instrument:

- **It proves the PAC matches the silicon.** If `pac\_led\_blink` blinks, the SVD, the generator and the part agree. If it does not, no amount of HAL work will help, and the failure has exactly one suspect.

- **It localises a regression.** When a HAL example stops working, the PAC example says whether the hardware path or the abstraction broke. That question has cost this project a full session at least once.

- **It documents facts the HAL deliberately hides.** `pac\_pullup\_probe` reads `PORTB\_DIN` with every pin in input mode and proves the external pull-ups exist; `pac\_i2c\_scan` proves the on-chip controller cannot complete a read. Neither is expressible through the HAL, because the HAL exists to make those facts invisible.

2.4.3 The Rule That Keeps It Honest

Proof-of-concept code rots faster than any other code, so it is held to a stricter rule than the HAL: **a PAC example may only name the layer it demonstrates**.

```
# from this project's gate: no example outside the PAC crate may mention the PAC
grep -rln 'efr32mg24_pac\|pac::' crates/*/examples/ \
  | grep -v '^crates/efr32mg24-pac/'
```

The inverse is enforced too: a PAC example that imports the HAL is not a PAC example, and would quietly stop testing what it claims to test. Between the two checks, each example belongs to exactly one rung of the ladder, and a reader can trust which rung they are standing on.

2.4.4 When to Delete One

- When its fact is now proved elsewhere *and* the example no longer adds a distinct failure signal. Two examples that fail for the same reason are one example.

- Never merely because it looks old. This project retired six HAL examples that had rotted into non-compiling code, and kept every PAC example that still builds and still tells you something the layer above cannot.

2.5 What the SVD Cannot Tell You

The SVD is the best document about this chip and it is still not enough. Three classes of fact are outside it, and every one of them cost this project real time.

2.5.1 Class 1: The Board

An SVD describes a *part*, and a board is a part plus wires:

- which pin the LED is on, and whether it is active low;

- which two pins carry a bus, and in which order;

- what external pull-ups exist;

- which pin gates power to a sensor.

None of that is in XML, which is why chapter 4 ends with a table of contradictions between the vendor's board documents and the board. The general rule: **chip facts from the SVD, board facts from a measurement, and the measurement written next to the code that depends on it.**

2.5.2 Class 2: The Aliases

This part's peripheral map has two windows: a secure alias at `0x4D00\_0000` and a non-secure one at `0x5D00\_0000`, for every peripheral that is not fixed to one domain. The SVD lists both, and it cannot list which one *your image* may use — that depends on the security attribution the bootloader and the linker applied.

Measured, on the MVP, from a non-secure image:

```
0x4D00_0000 (secure alias, from non-secure code)  -> reads as zero
0x5D00_0000 (non-secure alias) FAULTSTATUS (0x24) -> bus error
0x5D00_0000 (non-secure alias) the rest           -> works
```

Three different behaviours from one block, none of them described by the register file. The practical consequence is a rule this project now follows for any peripheral with two aliases: **probe reachability with a read-back of a harmless register before configuring anything**, and report which alias answered.

2.5.3 Class 3: Timing and Sequencing

An SVD says which bits exist. It does not say:

- that `EN` must be written and `DISABLING` polled before configuring;
- that a converter's `SYNCBUSY` must clear between writes;
- that a timer's compare register is buffered and takes effect on the next period;
- that an oscillator must be forced on and waited for.

Those are in the reference manual's prose, and the only reliable way to learn them is to watch a peripheral misbehave and then find the paragraph. What helps is knowing they exist: a peripheral that ignores a correct-looking configuration is almost always waiting for a sequence, not a bit.

2.5.4 The Order to Read Things In

1. **Generated register accessors** — for what exists, where, and with which fields.
2. **Field descriptions** — for what a value means; they are the SVD's own text.
3. **Reference manual prose** — for sequencing and timing, searched by register name.
4. **The board, with a probe** — for every wire, polarity and address.

Skipping to step 4 is expensive but sometimes correct; skipping step 1 is never correct, and it is the most common mistake in a project that started from someone's blog post.

- An SVD is XML describing every peripheral, register, field, enumerated value and reset value. It is the same file the vendor's debugger uses.
- `svd2rust` turns it into a PAC: a `Periph<T, ADDR>` per peripheral, a register block, and type-safe readers and writers per field.
- The generated field descriptions are documentation you can trust more than the manual, because they travel with the register accessors you are already calling.
- A register the SVD names *Block Enable* is not a suggestion. It was the missing write in an accelerator that had never executed.
- PAC-level examples are an antipattern for applications and indispensable as proof: they are the only place a HAL can be checked against the hardware.

3. Build Systems and Cross-Compilation Setup

"A build that succeeds is not evidence that it built what you wrote."

Cross-compiling for a microcontroller means four things have to agree: the target triple, the memory layout, the linker's idea of what a loadable segment is, and the image the programmer writes. When they disagree, the build still succeeds — that is the nature of linker scripts — and the board either does nothing or, in the worst case, no longer boots.

This chapter is the whole chain for this board, with the two build defects that produced a green log and a wrong artefact.

Contents

1.1 Rust's Value Proposition for Embedded 4
1.1.1 Memory Safety Without Garbage Collection 4
1.1.2 Zero-Cost Abstractions 4
1.1.3 Fearless Concurrency 4
1.1.4 Comparison with Traditional Embedded Languages 5
1.1.5 Rust vs C 5
1.1.6 Real-World Impact 5
1.1.7 Rust's Embedded Ecosystem 5
1.1.8 Core Crates 5
1.1.9 Tooling Integration 5
1.1.10 Addressing Embedded Development Challenges 6
1.1.11 Resource Constraints 6
1.1.12 Deterministic Behavior 6
1.1.13 Hardware Interface Safety 6
1.1.14 The EFR32MG24: Ideal for Rust 6
1.1.15 Summary 7
1.2 Your First Bare-Metal Rust Program 8
1.2.1 Creating a Minimal Embedded Project 8
1.2.2 Configuring for bare_metal: `no_std` 8
1.2.3 What is no_std? 8
1.2.4 The Entry Point 8
1.2.5 Cross-Compilation 9
1.2.6 Building the Project 9
1.2.7 Analyzing the Binary 9
1.2.8 What This Demonstrates 9
1.3 The PAC/HAL/BSP Architecture Pattern 11
1.3.1 The Three-Layer Architecture 11
1.3.2 Peripheral Access Crate (PAC) 11
1.3.3 Hardware Abstraction Layer (HAL) 12
1.3.4 Board Support Package (BSP) 12

1.3.5 The Flow of Abstractions . . . 13
1.3.6 Why This Separation Matters . . . 13
1.3.7 PAC Regeneration . . . 13
1.3.8 Multiple HAL Implementations . . . 13
1.3.9 Ecosystem Compatibility . . . 13
1.3.10 Our Approach in This Book . . . 14
1.4 Project Organization and Repository Structure . . . 15
1.4.1 Top-Level Directory Structure . . . 15
1.4.2 Crate Organization . . . 16
1.4.3 Peripheral Access Crate (efr32mg24-pac) . . . 16
1.4.4 Hardware Abstraction Layer (efr32mg24-hal) . . . 16
1.4.5 Radio System Crates (efr32mg24-rail-sys) . . . 16
1.4.6 Protocol System Crates (bluetooth-sys, etc.) . . . 16
1.4.7 Board Support Package (xiao-mg24-bsp) . . . 16
1.4.8 Why This Structure Works . . . 16
1.4.9 Clear Separation of Concerns . . . 16
1.4.10 Testing Strategy . . . 17
1.4.11 Version Management . . . 17
1.5 Development Prerequisites and Toolchain Setup . . . 18
1.5.1 Required Tools . . . 18
1.5.2 Installation on Linux (Ubuntu/Debian) . . . 18
1.5.3 Installation on macOS . . . 18
1.5.4 Installation on Windows . . . 18
1.5.5 Hardware Debug Probe Setup . . . 18
1.5.6 Installing probe-rs . . . 18
1.5.7 Flashing: one sanctioned path . . . 19
1.5.8 Updating Debug Probe Firmware . . . 19
1.5.9 Development Environment Setup . . . 19
1.5.10 VS Code Configuration . . . 19
1.5.11 Cargo Configuration for Embedded . . . 19
1.5.12 Git Configuration . . . 19
1.5.13 Verifying Your Setup . . . 19
1.6 Building Your First Embedded Binary . . . 21
1.6.1 Preparing a Complete Example . . . 21
1.6.2 Reference: Complete Project Example . . . 21
1.6.3 Understanding the Build Process . . . 21
1.6.4 Compile Step . . . 21
1.6.5 Binary Analysis . . . 21
1.6.6 Size Analysis . . . 21
1.6.7 Disassembly . . . 22
1.6.8 Listing Sections . . . 22
1.6.9 Build Optimization Comparison . . . 22
1.6.10 Linking and Linker Scripts . . . 23
1.6.11 Memory Layout . . . 23
1.6.12 Two Linker Flags That Are Not Optional . . . 23
1.6.13 Examining the Linker Output . . . 23
1.6.14 Creating a Flashable Binary . . . 23

1.6.15 Convert to Binary Format . . . 23
1.6.16 Hex Format for Some Tools . . . 24
1.6.17 Automated Build Script . . . 24
1.6.18 Build Automation with Justfile . . . 24
1.6.19 Key Takeaways . . . 24
1.7 What Goes In Each Crate, and Why . . . 25
1.7.1 The Question, Stated Sharply . . . 25
1.7.2 Three Candidate Layouts . . . 25
1.7.3 What Four Ecosystems Actually Do . . . 26
1.7.4 The Decision, and the Rule Behind Each Part . . . 26
1.7.5 Where the Algorithm Goes . . . 27
1.7.6 And Where the Examples Go . . . 27
1.7.7 A Decision Procedure for Hardware Not In This Book . . . 27
1.7.8 Adding Support for a New Chip or Board: the Order of Work . . . 28
1.7.9 What This Costs . . . 29
1.8 The Boundaries the Compiler Cannot See . . . 29
1.8.1 The Question That Finds Them . . . 29
1.8.2 The Same Defect, One Layer Down . . . 30
1.8.3 A Layer That Exports the Layer Below . . . 30
1.8.4 Why It Kept Happening: The Table . . . 30
1.8.5 Writing the Boundary Down as a Check . . . 30
1.8.6 What Is Not a Violation . . . 31
1.8.7 The Procedure, for the Next Project . . . 31
1.9 The Silicon Labs Toolchain, From the Outside . . . 32
1.9.1 The Pieces, and What They Are For . . . 32
1.9.2 JTAG, SWD, and the Debugger Already on the Board . . . 32
1.9.3 The Bootloader Is Not Optional, and It Is Why Linker Scripts Matter . . . 33
1.9.4 Simplicity Commander: Convenient, and the Wrong Tool Here . . . 33
1.9.5 The Radio Stack: Two Licences in One Directory . . . 33
1.9.6 The Radio Configurator, and Why RAIL Needs a Generated File . . . 34
1.9.7 What To Take Away . . . 34

3.1 The Target Triple, Decoded

`thumbv8m.main-none-eabihf` is four decisions in one string, and three of them can be wrong in ways that still compile.

Part	Means	If you get it wrong
`thumbv8m`	ARMv8-M | ARMv8-M Baseline	`armv7m` links, then traps on an instruction the core has
`main`	mainline, not baseline	baseline drops the DSP and FPU instructions you may be using
`none`	no OS, no libc	`std` becomes available and fails deep in linking
`eabihf`	hard-float ABI	`eabi` links and quietly passes floats in the wrong registers

Table 3.1: The four decisions. Only the last one fails *quietly*.

3.1.1 Hard Float, and Why It Matters Here

The EFR32MG24 has a single-precision FPU (FPv5-SP). `eabihf` tells the compiler it may pass floating-point arguments in `s0`–`s15` and may emit FPU instructions directly. With `eabi` the same source compiles to calls into soft-float helpers, links perfectly, and runs several times slower on exactly the arithmetic this project's classifier uses.

There is one trap on the way in: `-mfpu` and `-mfloat-abi` flags that *conflict* with the target's own choice produce a warning on every build — "switch `-mcpu=cortex-m33` conflicts with switch `-march=armv8-m.main+fp`". That warning is not harmless: it appeared once per C file in this project and buried the real warnings under it. The flags belong to the target; a build script adding its own is duplicating a decision it does not own.

3.1.2 The Toolchain, in Files

```
# the target must be installed once
rustup target add thumbv8m.main-none-eabihf

# .cargo/config.toml decides the default target and the runner
[build]
target = "thumbv8m.main-none-eabihf"

[target.thumbv8m.main-none-eabihf]
rustflags = ["-C", "link-arg=-Tlink.x"]
```

`link.x` comes from `cortex-m-rt`, and it is the file that reads `memory.x` and lays the image out. That indirection is why a wrong `memory.x` produces no error: `link.x` simply places the sections where it was told to.

3.1.3 Per-Crate Features, Not Per-Crate Flags

A workspace with several crates that must not duplicate the vector table uses a feature to decide which *one* of them pulls `cortex-m-rt`:

```
# the HAL exposes the runtime but does not enable it by default
[features]
rt = ["dep:cortex-m-rt"]

# examples turn it on; there is exactly one #[entry] per binary
efr32mg24-hal = { path = "../efr32mg24-hal", features = ["rt"] }
```

Without that split, two crates in one binary each define `Reset` and the link fails with a duplicate symbol that names neither crate. With it, the rule is explicit: a library may know about the runtime; only a final binary may install it.

3.2 The Bootloader Decides Your Memory Layout

The single most dangerous line in an embedded project is the `ORIGIN` of the flash region, and it is usually copied from a tutorial for a board with no bootloader.

3.2.1 What Is at the Base of Flash

This board ships with a Gecko Bootloader occupying the first 24 KiB. On the EFR32MG24 flash begins at `0x08000000`, so:

Region	Addresses	Owner
Bootloader	`0x08000000`–`0x08005FFF`	factory; never written by this project
Application	`0x08006000`–end	every image in this book

Table 3.2: 24 KiB that a linker script must not touch.

3.2.2 The Wrong memory.x, and What It Does

The version that appeared in this project's first draft of the book — copied from a generic `cortex-m` example — read:

```
MEMORY
{
  FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 1536K
  RAM  (rwx) : ORIGIN = 0x20000000, LENGTH = 256K
}
```

Every line is wrong for this board, and none of them produces an error. The linker happily lays the image out from address zero, the programmer writes it exactly there, the write verifies — and the board no longer boots, because the bootloader that was at zero has been replaced by an application that expects to be at `0x08006000`.

3.2.3 The Version in Use

```
MEMORY
{
  /* The Gecko bootloader owns 0x08000000..0x08005FFF. An application that starts
     below 0x08006000 overwrites it. */
  FLASH (rx)  : ORIGIN = 0x08006000, LENGTH = 1536K - 24K
  /* The mailbox at 0x2003F000 is a convention shared by every example; the stack
     is placed below it so a deep call cannot scribble over the report. */
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 252K
}

_stack_start = ORIGIN(RAM) + LENGTH(RAM) - 4096;
```

Both numbers are deliberate: 252K rather than 256K because the top 4 KiB are reserved, and `\_stack\_start` is below the mailbox so a stack overflow corrupts a report rather than the data being reported.

3.2.4 The Rule, Enforced in a Script

A comment is not protection. The project has two mechanical guards:

```
# 1. where would this image land?
python3 scripts/elf-load-range.py target/thumbv8m.main-none-eabihf/release/examples/pac_led_blink
# load_min=0x08006000 load_max=0x08006523 size=1316 segments=3

# 2. refuse anything below the bootloader boundary, before writing
bash scripts/flash.sh "$ELF"          # dry run: prints the plan, refuses load_min < 0x08006000
bash scripts/flash.sh "$ELF" --confirm
```

The first command is the one worth internalising: **read the load addresses before every flash**, not after a board stops answering. It takes one second and it is the difference between a mistake and a brick.

3.3 –nmagic, or the Linker's Idea of a Segment

This section documents the least intuitive flag in the whole chain, because without it the image is built, verified, and written to the wrong place.

3.3.1 What the Linker Does by Default

ELF loadable segments carry two addresses: a virtual address (`VMA`, where the code expects to run) and a physical address (`LMA`, where the bytes must land in memory). For flash that is always 1:1 — you cannot relocate code that runs from flash.

A linker aiming at an operating system still aligns segments to page boundaries, and to do that it gives the segment an *offset* that no longer matches its load address. An OS loader compensates; a bare-metal programmer does not, and writes the file's bytes at the segment's file offset while the code expects them at the segment's address.

3.3.2 The Symptom, Measured

```
# with a default link: the loadable segment's file offset is 0x1000, its address is 0x08006000
probe-rs download image.elf        # writes bytes at the wrong offset
probe-rs verify image.elf          # verifies the same wrong offsets: it agrees with itself
                                   # the board does not run
```

This is the failure that teaches the difference between "verified" and "correct". The programmer compares the flash against the file it just wrote. If the file was written at the wrong offset by a consistent rule, both sides agree and the check passes.

3.3.3 The Fix, and Why It Is One Flag

```
# in .cargo/config.toml, or as a link argument
rustflags = ["-C", "link-arg=--nmagic"]
```

`--nmagic` tells the linker not to align segments and not to pad them — the image is flat, VMA equals LMA for every loadable segment, and the programmer's naive write is correct. Every bare metal Rust project needs it and most tutorials omit it, because their toolchain's runner happens to compensate.

3.3.4 Checking the Invariant Instead of Trusting the Flag

The project's inspection script prints exactly what matters, so the flag is verified rather than believed:

```
python3 scripts/elf-load-range.py "$ELF"
# for each PT_LOAD: paddr == vaddr, and load_min >= 0x08006000
#   seg paddr=0x08006158 vaddr=0x08006158 filesz=5400
#   seg paddr=0x08007670 vaddr=0x08007670 filesz=108
#   seg paddr=0x080076DC vaddr=0x20000000 filesz=4     <- the .data initialiser
```

Three lines, and two facts read directly off them: every code segment has `paddr == vaddr` (the flag worked), and `load\_min` is above the bootloader (the memory map is right). The third segment is the initialiser for `.data`, which correctly has a flash address and a RAM address — that one is supposed to differ.

3.4 Builds That Lie

Two defects in this project had the same shape, and it is the most dangerous shape a build system can have: **the build succeeded, and the artefact was not what the sources said.**

3.4.1 Case 1: A build.rs That Did Not Watch Its Own C File

A C file compiled by the `cc` crate, used to provide symbols the vendor blobs reference:

```
// build.rs --- the version that lied
cc::Build::new().file("src/cmsis_stubs.c").compile("cmsis_stubs");
println!("cargo:rerun-if-changed=blobs/");
println!("cargo:rerun-if-changed=vendor/");
```

Editing `src/cmsis_stubs.c` changed nothing. Cargo had no reason to re-run the script, the old object file stayed, and the link failed on a symbol that was visibly present in the source — with a message that sends you looking at the blob instead of the build.

```
// the version that tells the truth
println!("cargo:rerun-if-changed=src/cmsis_stubs.c");
```

The general rule: **any file a build script reads or compiles is an input, and an input it does not declare is an input the build ignores.** Cargo does not scan your script for file paths.

3.4.2 Case 2: A Book That Compiled Yesterday's Sources

This book is built in a container on a different machine, from a tarball:

```
# the version that lied: extracts to efr32book/book/latex/
tar cf - book/latex | ssh host 'cd ~/efr32book && tar xf -'

# ...while the container compiles efr32book/latex/, so every build used the previous copy
podman exec --workdir /workspace/efr32book/latex tex xelatex main.tex
```

The path `book/latex` is preserved by `tar`, the compile directory is not, and the two never met. A whole section was written, "built" successfully, and did not appear in the PDF.

The fix is one flag plus one check:

```
tar cf - book/latex | ssh host 'cd ~/efr32book && tar xf - --strip-components=1'

# and then prove it, before compiling
md5sum book/latex/chapters/.../01.08-layer-boundaries.tex
ssh host 'md5sum ~/efr32book/latex/chapters/.../01.08-layer-boundaries.tex'   # must match
```

3.4.3 The Pattern Behind Both

Lie	What you see	What catches it
unwatched input	link error naming a symbol you wrote	`rerun-if-changed` on every input
wrong extraction path	success, no new content	compare a hash before compiling
cached artefact	old behaviour at new code	clean the package, or watch the log for the crate

Table 3.3: Three ways to build the wrong thing, and the one-line check for each.

The common answer is the same in all three cases: **verify that the thing being built is the thing you wrote.** It is a one-line check, and it is cheaper than any of the afternoons it saves.

3.5 The Gate: What Runs Without a Board

A project with hardware has a scheduling problem: the compiler is available constantly, the board is available sometimes. Anything that needs the board gets deferred, and deferred checks do not happen. The answer this project uses is a single script that runs everything that does not need hardware.

3.5.1 What It Checks

```
$ bash scripts/gate.sh
PASS  cargo fmt --all --check
PASS  efr32mg24-hal clippy, 0 warnings
PASS  efr32mg24-pac clippy, 0 warnings
PASS  xiao-expansion-base-bsp clippy, 0 warnings
PASS  xiao-mg24-bsp clippy, 0 warnings
PASS  motion-detection clippy, 0 warnings
PASS  efr32mg24-hal: 5 examples of its own level
PASS  efr32mg24-pac: 19 examples of its own level
PASS  xiao-expansion-base-bsp: 9 examples of its own level
PASS  motion-detection: host tests (9 passed)
PASS  motion-detection: no hardware dependency
PASS  examples outside the PAC crate name no PAC type
PASS  no BSP re-exports the PAC
PASS  the HAL exposes no register block publicly
PASS  no example steals a core peripheral
VERDICT: PASS
```

Every line is a property, not a command. That distinction matters: a script that runs `cargo clippy` tells you a tool ran; a line that says *zero warnings in five crates* tells you what is true.

3.5.2 The Discovered-vs-Listed Rule

The first version of this gate had a hard-coded list of crates, and the crate that most needed checking was the one that was not on it. The version in use *discovers* crates:

```
for dir in crates/*/ libraries/*/; do
  [ -f "${dir}Cargo.toml" ] || continue
  name=$(basename "$dir")
  ...
done
```

A list is a snapshot of what existed when someone wrote it. Discovery cannot fall behind — which is exactly what a check is for.

3.5.3 The Rule Nobody Wrote Down

The most valuable check in that output is the one that sounds odd: **every crate must have at least one example of its own level**.

- An API with zero examples passes every other check trivially — it compiles, it lints, and nothing calls it.
- It then rots silently: in this project a whole board module spent weeks documented and unbuildable, and the only symptom was that nobody used it.
- The check is cheap and it fails loudly, which is the entire point of a gate.

3.5.4 What a Gate Cannot Do

It cannot tell you the LED lit. This project's gate was green on a firmware that flashed successfully and did not blink — four green steps, no light. That is not a failure of the gate; it is the boundary of what a build-time check can know, and it is why the hardware checks in chapter 14 exist as a separate, written procedure with its own measurements.

- The triple is `thumbv8m.main-none-eabihf`: ARMv8-M mainline, no OS, hardware float. Getting the float ABI wrong links but produces a slow, subtly different binary.
- The bootloader owns the first 24 KiB. `memory.x` starts flash at `0x08006000`, and a script refuses any image that would start lower.
- `--nmagic` is not optional: without it the linker emits a segment whose file offset disagrees with its load address, and the programmer writes to the wrong place.
- Two builds lied in this project: a `build.rs` that did not watch its own C file, and a book build that compiled yesterday's sources because of a missing `--strip-components`.
- A gate that runs without the board is worth more than a checklist, because it runs every time.

Part II

Hardware Abstraction Layer (HAL)

4. Phase A: GPIO Implementation

> *"A pin is not an abstraction problem. It is a register problem that everybody thinks is an abstraction problem."*

A GPIO is the simplest peripheral on any microcontroller and the one that produces the most over-engineered code. This chapter takes the opposite route: it starts from the registers on this part, derives the smallest abstraction that is still safe, and shows the three defects that the bigger abstraction — 605 lines of type-state pins — did not prevent.

Everything here is measured on the XIAO MG24 Sense (EFR32MG24B220F1536IM48-B), and the numbers are read back over SWD rather than assumed.

Contents

2.1 What an SVD Actually Contains . . . 38
2.1.1 The Five Things That Matter . . . 38
2.1.2 A Register, in the SVD and in Rust . . . 38
2.1.3 Where the SVD Comes From, and Why It Is Trustworthy . . . 38
2.1.4 The Practical Consequence: One File, One Crate . . . 38
2.2 What svd2rust Emits, and How to Read It . . . 38
2.2.1 Shape 1: The Peripheral . . . 39
2.2.2 Shape 2: The Register Block . . . 39
2.2.3 Shape 3: Read, Write, Modify . . . 39
2.2.4 Shape 4: The Field Writer . . . 39
2.2.5 Unsafe, and Where It Is Honest . . . 39
2.3 The SVD as an Oracle . . . 39
2.3.1 Case 1: The Accelerator That Never Ran . . . 39
2.3.2 Case 2: A Register That Is Not There . . . 40
2.3.3 The Habit . . . 40
2.4 Why PAC-Level Examples Exist Anyway . . . 40
2.4.1 The Ladder . . . 40
2.4.2 What the PAC Level Proves . . . 41
2.4.3 The Rule That Keeps It Honest . . . 41
2.4.4 When to Delete One . . . 41
2.5 What the SVD Cannot Tell You . . . 41
2.5.1 Class 1: The Board . . . 42
2.5.2 Class 2: The Aliases . . . 42
2.5.3 Class 3: Timing and Sequencing . . . 42
2.5.4 The Order to Read Things In . . . 42

4.1 The Registers, and How Few There Are

A GPIO port on the EFR32MG24 is not a black box. It is five things:

Register	What it decides	Access
`MODEL`	mode of pins 0–7	read-modify-write, per pin
`MODEH`	mode of pins 8–15	only on PORTA and PORTC here
`DOUT`	the output latch, one bit per pin	read-modify-write
`DIN`	the pin as the outside world holds it	read only
`CMU\_CLKEN0`	whether the port answers at all	gate before the first access

Table 4.1: The GPIO port, in full. Five registers, and only one of them is optional.

The modes are numbers, not types:

```
// Measured from the PAC's field encodings, and confirmed by the effect on the pin.
const DISABLED: u32 = 0;   // the pad is off; an analog peripheral may claim it
const INPUT:    u32 = 1;   // input buffer on, no pull
const PUSH_PULL:u32 = 4;   // driven high or low
const WIRED_AND: u32 = 8;  // open drain: 0 pulls low, 1 lets the pull-up win
const WIRED_OR:  u32 = 9;  // open drain, inverted sense
```

Two of these decide most of the code in this book. `PUSH_PULL` drives an LED and a piezo. `WIRED_AND` is what makes a bit-banged I2C bus possible, because "output 1" means "release the line" and the external pull-up makes it a real 1 — which is exactly what an open-drain bus requires and exactly what a push-pull pin must never do.

4.1.1 The Clock Gate Comes First

The most expensive mistake in this whole project was not a wrong bit. It was touching a register whose bus clock was off:

```
// Before: the write faulted, and the fault handler did not exist yet.
gpio.porta_dout().modify(|_, w| w.dout7().set_bit());

// After: gate the port, then talk to it.
dp.cmu_s.clken0().modify(|_r, w| w.gpio().set_bit());
gpio.porta_dout().modify(|_, w| w.dout7().set_bit());
```

The measurement, taken from the fault status register after the fact: `CFSR = 0x8200` (a precise bus fault) with `BFAR` pointing at the GPIO address that was written. A GPIO register with no clock is not "reads as zero" — it is a bus fault, and if the vector table still points at a default handler the board simply stops.

4.1.2 Not Every Port Has Every Register

```
// This package is a 48-pin QFN: PORTB and PORTD expose eight pins, not sixteen.
// MODEH covers 8..15, so on those two ports it does not exist at all.
match port {
    Port::A | Port::C => { /* MODEL and MODEH */ }
    Port::B | Port::D => { /* MODEL only */ }
}
```

That is not a simplification invented for this book; it is what the SVD says, and a HAL that pretends otherwise fails to compile *after* someone has written the code. Two of the eight compile errors during this project's pin primitive came from exactly that: the generated register blocks differ per port, and a macro written for one does not typecheck for the other.

4.2 A Pin Is Three Numbers

The type-state approach to GPIO is seductive: make an output pin a different *type* from an input pin, make a pin claimed by one driver unclaimable by another, and let the compiler refuse the mistakes. This book's repository had exactly that — 605 lines of it — and it is now retired, for a reason worth stating precisely.

4.2.1 What the Small Version Looks Like

The whole abstraction is a port, a bit and a polarity:

```
pub enum Port { A, B, C, D }

pub struct Output<'a> {
    gpio: &'a pac::gpio_s::RegisterBlock,
    port: Port,
    bit: u8,
    active_low: bool,
}

impl<'a> Output<'a> {
    pub fn on(&self)  { self.write(!self.active_low) }
    pub fn off(&self) { self.write(self.active_low) }
    pub fn toggle(&self) {
        let current = read_dout(self.gpio, self.port);
        self.write(current & (1 << self.bit) == 0);
    }
}
```

Everything else is a register write:

```
fn write_dout(gpio: &pac::gpio_s::RegisterBlock, port: Port, bit: u8, high: bool) {
    // Read-modify-write, always: DOUT is one register for sixteen pins.
    let set = |r: u32, w: &mut pac::W<_>| {
        let mask = 1u32 << bit;
        let value = if high { r | mask } else { r & !mask };
        unsafe { w.bits(value) }
    };
    match port {
        Port::A => gpio.porta_dout().modify(set),
        Port::B => gpio.portb_dout().modify(set),
        Port::C => gpio.portc_dout().modify(set),
        Port::D => gpio.portd_dout().modify(set),
    }
}
```

That is the entire output path: 40 lines including the four-way match. The input path is shorter still — `DIN` needs no mask, only a shift — and the mode setter is the read-modify-write of `MODEL` or `MODEH` shown in the previous section.

4.2.2 Why the Larger Version Was Retired

The type-state layer was not wrong in principle. It was wrong here, in three ways that only show up in a real project:

1. **Nothing used it.** Its only consumer was a board module that a newer facade replaced. An API with zero users compiles, lints clean, and rots: at the time it was retired, its README documented the user LED as `PB2` active-*high*, when the board's LED is `PA7` active-*low*.

2. **It moved the wiring into the HAL.** To hand out a typed `LED` the HAL had to know which pin the LED was on — a board fact in the wrong layer. The layered rule from section 1.8 is that the HAL names no board wiring, and the typed pins were the mechanism by which it did.

3. **It cost more than it prevented.** The three real defects in this project's GPIO code — a blind `MODEL` write, a mode value with the wrong width, and an open-drain pin configured push-pull — were all invisible to the type system, because all three are legal code with the wrong *number* in it. Types catch the mistakes you can name; registers catch the rest.

4.2.3 What Replaced the Type System

An ownership rule that is checkable without generics: the board takes the peripherals once (`Board::take()`), and every pin comes out of it with a port and a bit named at the call site:

```
let board = Board::take().expect("peripherals taken twice");
let led     = board.output_on(Port::A, 7, true);      // active low
let button  = board.input_on(Port::C, 1);             // pull-up fitted on the base
let buzzer  = board.output_on(Port::C, 3, false);
```

The exclusivity that the type system used to provide is now the PAC singleton's job — one `take()` per program — and the wiring that the HAL used to hide lives in whichever crate owns the board. Fewer lines, fewer layers, and the two facts a reader needs (which pin, which polarity) are in the code they are reading.

4.3 Read-Modify-Write, or Why the Bus Stopped Working

One register, sixteen pins. That sentence is the whole section, and it caused one of the most confusing failures in this project.

4.3.1 The Defect

Setting a pin's mode looks like a write. It is a write to a register that also holds fifteen other pins:

```
// Wrong: this configures *every* pin on the low half of PORTC, not just PC3.
gpio.portc_model().write(|w| w.mode3().pushpull());

// Right: read what is there, change one field, write it back.
gpio.portc_model().modify(|_r, w| w.mode3().pushpull());
```

The wrong version does not fail. It reconfigures the fifteen other pins to their reset value — and on this board the other pins on that half-port include the two that carry an I2C bus. A bus that had just been proved working stops answering, with no error anywhere, because the pins that carried it went back to `DISABLED`.

4.3.2 How It Was Found

The failure looked like an I2C problem: the device stopped acknowledging, a scan returned nothing, and reading the bus's own registers showed the peripheral configured exactly as intended. The registers that mattered were the *pin* ones:

```
PORTC_MODEL = 0x00000000      # every pin on bits 0..7 disabled
PORTC_DOUT  = 0x00000030      # the bus lines still latched high
```

Both lines at once: the latch said high, the pads were off. That combination can only come from a write that reset the mode register, and a search for `.write(` on that register found it in one place — the function that was setting a buzzer pin.

4.3.3 The Rule

Every write to a GPIO configuration register in this project now goes through `.modify()`. The only `.write()` calls are to registers that genuinely hold one field and no other pin — and even then, the code says so in a comment, because the next person will copy the pattern somewhere it does not hold.

The same rule applies to `DOUT`: a blind `write` there sets every pin on the port high or low at once. It is the reason the pin primitives in this repository all take a port and a bit, and never a whole register value:

```
// The signature is the rule: there is no way to write DOUT without saying which bit.
fn write_dout(gpio: &RegisterBlock, port: Port, bit: u8, high: bool)
```

4.3.4 A Second, Quieter Version of the Same Bug

`modify()` on a register whose clock is off is a bus fault, but `modify()` on a register with a *write-only field* is worse: it reads back a value the field does not contribute to, so the read-modify-write silently clears it. This is why the mode values in this project are written as an explicit numeric field of the right width rather than as a chain of boolean writers, and why the IADC's configuration registers (chapter 7's sibling problem) are written once, in one function, in a documented order — conditional writes into a synchronising peripheral lose the field you did not intend to touch.

4.4 Open Drain, and the Line That Has Two Masters

An I2C bus has two wires and, on each of them, two possible masters, each of which may want to pull the line low while the other wants it high. The electrical answer is the reason the mode list has a `WIRED\_AND` in it.

4.4.1 What WIRED_AND Does

```
mode = WIRED_AND   (8)
  DOUT bit = 0  ->  the pad pulls the line to ground
  DOUT bit = 1  ->  the pad releases the line (high impedance)
```

The line is held high by an external resistor. So "1" is not something the pin produces — it is something the pin *stops preventing*. Two devices can then share the wire: any one of them can pull it low, and the only way it goes high is if nobody does. That is what makes arbitration possible, and it is why a push-pull pin must never be connected to a shared bus: driving it high against another device driving low is a short circuit through two transistors.

4.4.2 The Measurement That Proved the Pull-Ups

The IMU could not be found for two sessions. Before blaming the pin map, this project measured whether the bus lines had pull-ups at all:

```
# every pin on PORTB set to input, then read DIN
PORTB_MODEL = 0x11111111     # all inputs
PORTB_DIN   = 0x0000000C     # bits 2 and 3 high
```

Bits 2 and 3 are `PB2` and `PB3`: the two pins of the IMU's bus, reading high with nothing driving them. That is an external pull-up, present and working — which retired the "the board has no pull-ups" hypothesis in one instruction and moved the investigation to the pin *order*.

4.4.3 Bit-Banging an Open-Drain Bus

With `WIRED\_AND` the whole I2C protocol reduces to two writes and one read:

```
fn sda(&self, high: bool) { write_dout(self.gpio, self.bus.port, self.bus.sda_bit, high) }
fn scl(&self, high: bool) { write_dout(self.gpio, self.bus.port, self.bus.scl_bit, high) }
fn read_sda(&self) -> bool { read_din(self.gpio, self.bus.port) & (1 << self.bus.sda_bit) != 0 }
```

There is no direction switch. On an open-drain bus a pin used as an input with the output latch high is the same electrical state as a released output — which is why a bit-banged I2C master can be four functions and no state machine, and why the same pin can be written and read in the same transfer.

4.4.4 Half a Bit Period, and Why It Is a Number in Cycles

Timing on a bit-banged bus is a delay per half period, and the delay is not a duration — it is a count of loop iterations whose real length depends on the compiler:

```
// ~5 us at 19 MHz, measured: 60 iterations of this loop on this compiler.
const HALF_BIT_CYCLES: u32 = 60;
```

That constant is a measurement, not a datasheet value, and it is the reason chapter 14 exists: the same class of mistake — `asm::delay(n)` assumed to be `n` cycles when it is about three per iteration — turned a 2.7 kHz tone into something else entirely.

4.5 What the Vendor's Own Documents Get Wrong

Three documents describe this board's pins, and two of them disagree with the board. This section is short and uncomfortable, and it exists because a wrong pin in a comment is worse than no comment: it is believed.

4.5.1 The User LED

Source	Says	What it is
The HAL's retired GPIO README	`PB2`, active high	wrong pin, wrong polarity
An earlier board module doc	`PB02/D10`	wrong pin
Register read-back (measured)	`PA7`, active low	**this one**

Table 4.2: The user LED, as documented and as measured. The measurement is a write to PORTA_DOUT bit 7 and a read of PORTA_DOUT to confirm the latch moved.

Why the error survived: nothing in the project ever *used* the wrong pin in a way that failed. A pin that does not exist toggles a latch nobody looks at, and the code still returns `Ok`. Only the blink example — the one that must be seen — exposed it.

4.5.2 The IMU's Two Wires

The Arduino variant for this board declares:

```
PIN_WIRE_SDA1 = PB3
PIN_WIRE_SCL1 = PB2
```

The board is wired the other way round. Measured, in this order:

1. a bit-banged scan of all 128 addresses on the pair `(PB2, PB3)` with SDA=`PB2` found a device at `0x6A`;

2. the same scan with the roles swapped found nothing;
3. the device at `0x6A` answered `WHO\_AM\_I = 0x6A`, which is the LSM6DS3TR-C's identity register.

Three independent facts — the scan, the negative control, and the identity register — agree. The variant file does not, and it is the file a reader would trust first.

4.5.3 The Lesson, Which Is Not "the Vendor Is Wrong"

The vendor's documents are mostly right and are the only reason this works at all: the SVD, the reference manual, the bootloader layout and the radio headers are all correct and all essential. But the pin map of a *board* is not the same document as the pin map of a *chip*, and it is maintained by people assembling a product, not by the silicon team.

So the rule this project adopted: **every pin fact is measured on the board, and the measurement is written next to the claim**. Two of the three corrections above cost an afternoon each. Both would have been found in ten minutes by reading a register instead of a comment.

4.6 Proving a Pin Without Trusting Your Eyes

The blink is the first program every embedded developer writes, and it is the first one that lies. A blink that does not blink has four different causes that look identical: the code never ran, the pin is wrong, the polarity is wrong, or the LED is not connected. This section is about telling them apart with instruments that are already there.

4.6.1 The Case That Made This Necessary

This project had a blink that flashed successfully and did not blink. The write verified, the read-back matched, the board was silent. Every step of the toolchain reported success:

```
cargo build            Finished `release` profile [optimized] target(s)
python3 scripts/elf-load-range.py   load_min=0x08006000 load_max=0x08006813
probe-rs download      Finished in 0.41s
probe-rs verify        Finished in 0.28s
```

Four green steps and no light. The only way forward was to stop asking the board to show and start asking the register to tell.

4.6.2 Reading the Latch Over SWD

The probe can read any address while the program runs. The output latch is one word, and it toggles or it does not:

```
# halt, read DOUT twice, resume
openocd -c "init; halt" -c "mdw 0x5003C040 1" -c "sleep 500" -c "mdw 0x5003C040 1" -c "resume;
↪  shutdown"

# 0x5003c040: 00000080      <- bit 7 high
# 0x5003c040: 00000000      <- bit 7 low, 500 ms later
```

That pair of readings is a proof of the *program*: the latch is moving at the rate the code intended. What it is not is a proof of the *board*: a latch that toggles and an LED that stays dark means the pin is not the LED's pin, or the LED is active-low and the code drives it backwards.

The second reading of `PORTA\_MODEL` closes the other half:

```
PORTA_MODEL = 0x00000040   # mode 4 (push-pull) in the field for pin 7
```

Latch moving, mode correct, and still no light: that combination is now a board question, not a software one — which is exactly the division of labour a measurement is for.

4.6.3 Three Instruments, Three Questions

Instrument	Question it answers	How it is read
The LED	is the whole chain alive?	eyes, and only after the two below
`DOUT`/`MODEL` over SWD	is the program right?	`mdw` on the port
A mailbox in RAM	did the code reach this point?	`mdw` on a magic word

Table 4.3: Order matters: the LED answers last, because it answers slowly and ambiguously.

4.6.4 The Mailbox, Which Is the Real Lesson

The instrument that made the rest of this project possible is not a pin at all. It is a small block of RAM — a magic word, a sequence number, a status and a message — that a program writes before it does anything it might not survive:

```
// 0x2003F000: magic "MBOX", seq, status (0 info / 1 pass / 2 fail), len, message bytes
mailbox::write(Status::Pass, b"cap=0x20000000 4000@400Hz real=clean");
```

Read with one `mdw` over SWD, it answers questions that no LED can: did the classifier load its calibration, how many samples did the ring hold, which of the two I2C aliases answered. It costs 300 bytes of RAM and it replaced an entire class of guesswork — which is why every example in this book reports through it rather than through a serial port that may not exist.

- A GPIO port on this part is five registers: `MODEL`, `MODEH`, `DOUT`, `DIN` and the port's clock gate.
- `MODEH` does not exist on every port: on PORTB and PORTD this package has eight pins, so those pins live in `MODEL` alone.
- A pin is a port, a bit and a polarity. That is enough to drive an LED, read a button and open-drain a bus, and it is 189 lines — not 605.
- Writing `MODEL` or `MODEH` without reading first reconfigures every other pin on the same half-port, which is how a working bus stops working.
- The LED on this board is `PA7`, active *low*. Two of the vendor's own documents disagree with that.
- A blink is proven by the register toggling, not by the LED looking on.

5. Phase A: Clock Management and System Configuration

"Every timing bug is a clock bug wearing a disguise."

Nothing on this part runs without a clock, and almost nothing that goes wrong is blamed on one. This chapter is the record of four separate defects in one project that were all clocks: a delay that was three times too short because a loop is not an instruction count, a peripheral that never started because its oscillator was selected but not enabled, a converter that hung because it was clocked from the wrong source, and a sampling loop that claimed 400 Hz and ran at 158.

The numbers here are measured with the cycle counter, not read from a datasheet.

Contents

3.1 The Target Triple, Decoded 46
3.1.1 Hard Float, and Why It Matters Here 46
3.1.2 The Toolchain, in Files 46
3.1.3 Per-Crate Features, Not Per-Crate Flags 46
3.2 The Bootloader Decides Your Memory Layout 46
3.2.1 What Is at the Base of Flash 46
3.2.2 The Wrong memory.x, and What It Does 47
3.2.3 The Version in Use 47
3.2.4 The Rule, Enforced in a Script 47
3.3 –nmagic, or the Linker's Idea of a Segment 47
3.3.1 What the Linker Does by Default 47
3.3.2 The Symptom, Measured 47
3.3.3 The Fix, and Why It Is One Flag 48
3.3.4 Checking the Invariant Instead of Trusting the Flag 48
3.4 Builds That Lie 48
3.4.1 Case 1: A build.rs That Did Not Watch Its Own C File 48
3.4.2 Case 2: A Book That Compiled Yesterday's Sources 48
3.4.3 The Pattern Behind Both 48
3.5 The Gate: What Runs Without a Board 48
3.5.1 What It Checks 49
3.5.2 The Discovered-vs-Listed Rule 49
3.5.3 The Rule Nobody Wrote Down 49
3.5.4 What a Gate Cannot Do 49

5.1 The Tree, in the Order You Meet It

A Series 2 clock tree is not complicated once it is drawn as three questions. This section answers them for this board only, because the full diagram in the reference manual has forty boxes and no project needs forty.

5.1.1 Question 1: Which Oscillator

Oscillator	Rate	When it is the right answer
`HFRCO0`	19 MHz default	the reset state; enough for GPIO, timers and I2C
`HFXO0`	38.4 MHz crystal	radio, USB, anything needing ppm accuracy
`FSRCO`	20 MHz fast RC	cheap and fast, but see the IADC below
`HFRCOEM23`	19 MHz	low-energy domain; the IADC and TIMER0 like it
`DPLL0`	up to 78 MHz	the CPU's fast mode, locked to HFRCO or HFXO
`LFRCO/LFXO`	32.768 kHz	RTC, sleep, wake-up

Table 5.1: Six oscillators, and the four that matter for a board with an I2C bus, a buzzer and a microphone.

5.1.2 Question 2: Which Branch

Each oscillator feeds branches, and each branch has a selector and a divider:

```
// The low-energy peripherals (IADC, TIMER0 in this project) are clocked from EM01GRPACLK.
dp.cmu_s.em01grpaclkctrl().modify(|_r, w| w.clksel().hfrcoem23());
```

The name matters when reading the manual: `EM01GRPACLK` is one branch, `EM23GRPACLK` another, and a peripheral belongs to exactly one. Choosing the right oscillator on the wrong branch gives a peripheral clocked at zero — which looks identical to a peripheral whose registers are not being written.

5.1.3 Question 3: Which Gate

Every peripheral has a bus-clock gate in `CMU\_CLKEN0` or `CMU\_CLKEN1`. On this part the GPIO bank is bit 0 of `CLKEN0` and the MVP's is bit 30 of `CLKEN1`. A register access with the gate closed is a bus fault, precisely reported:

```
CFSR = 0x00008200   # PRECISERR set: a precise data bus fault
BFAR = 0x5003C040   # ...at PORTA_DOUT, whose clock was off
```

That pair of registers is the fastest diagnosis in this book. If a peripheral reads as zero, check the gate; if a write faults, read `CFSR` and `BFAR` before anything else.

5.1.4 The Three Questions, Together

```
// Oscillator: force HFRCOEM23 on and wait for it.
dp.cmu_s.clken0().modify(|_r, w| w.hfrcoem23().set_bit());
dp.hfrcoem23_s.ctrl().modify(|_r, w| w.forceen().set_bit());
while !dp.hfrcoem23_s.status().read().rdy().bit_is_set() {}

// Branch: the low-energy domain comes from it.
dp.cmu_s.em01grpaclkctrl().modify(|_r, w| w.clksel().hfrcoem23());

// Gate: the peripheral's own bus clock.
dp.cmu_s.clken0().modify(|_r, w| w.timer0().set_bit());
```

Three decisions, three writes, and one wait. The wait is the subject of the next-but-one section: it is the only one of the four steps that can hang.

5.2 The Clock You Forgot You Were Using

Every HAL has a constant called something like `DEFAULT\_SYSCLK\_HZ`, and it is almost always wrong for the board it runs on. This section is about the one on this board, because it silently defines the length of every delay in the project.

5.2.1 19 MHz, Not 78

The EFR32MG24 can run at 78 MHz. It does not, out of reset, and this board's firmware never raises it. The default is `HFRCO0` at 19 MHz, and the measurement that settled it was a stopwatch rather than a datasheet:

```
# a blink with a 500 ms half-period, counted against a phone stopwatch
# 20 blinks took 20.0 s  ->  1.000 s per cycle  ->  the delay was right at 19 MHz
# the same code timed as if SYSCLK were 78 MHz produced a 4x fast blink
```

That is a crude instrument, and it is the right one: a stopwatch over twenty cycles has an error far below the 4× discrepancy being tested.

5.2.2 Why the Error Is Invisible

A wrong `SYSCLK\_HZ` does not produce a wrong-looking program. It produces a program whose delays are all scaled by the same factor — blinks that are too fast, tones that are too high, buses that are too slow or too fast. Each of those looks like a bug in that peripheral:

Symptom	Blamed on	Actually
LED blinks at 4× the rate	the delay loop	`SYSCLK\_HZ` too high
Tone is 4× the intended pitch	the timer's TOP	same constant
I2C bus works on one board, not another	the devices	same constant, other clock
Buzzer sounds, then the beep pattern is wrong	the pattern code	same constant

Table 5.2: One wrong constant, four different suspects. This is why it is measured once and written down next to the definition.

5.2.3 The Right Shape for the Constant

The version that survives contact with a second board is not a number in the HAL. It is a number the board passes in:

```
// The HAL knows how to delay; it does not know how fast this board's clock runs.
let mut delay = Delay::with_sysclk(syst, 19_000_000);
```

and in this project's board layer, one place says it out loud:

```
pub const DEFAULT_SYSCLK_HZ: u32 = 19_000_000;   // measured, see 05.02
```

A second board with a 38.4 MHz crystal changes one line, and every delay, tone and bus timing follows it. A HAL with the constant baked in cannot be reused, and — worse — fails in a way that looks like the new board's fault.

5.2.4 Measuring It Properly

The stopwatch is enough to catch a factor of four. For anything finer, use the cycle counter, which counts CPU cycles and therefore cannot disagree with the CPU:

```
let start = dwt::cycles();
delay.delay_ms(1000);
let elapsed = dwt::since(start);
// elapsed / 19_000_000 must be ~1.0 s; a factor of 3.0 here means the delay's
// internal arithmetic is wrong, not the constant.
```

That distinction — constant wrong *versus* arithmetic wrong — is the difference between recalibrating and debugging, and the cycle counter is what separates them.

5.3 Selecting an Oscillator Is Not Starting It

The clock tree has a trap that costs an afternoon the first time and ten minutes every time after: choosing a source does not turn it on, and a peripheral clocked by a stopped source behaves like a peripheral that ignores you.

5.3.1 The Two-Step That Is Actually Three

```
// 1. Ask for it.
dp.cmu_s.clken0().modify(|_r, w| w.hfrcoem23().set_bit());
// 2. Force it on: without this the oscillator idles when nothing demands it,
//    and "nothing" includes the branch you are about to select.
dp.hfrcoem23_s.ctrl().modify(|_r, w| w.forceen().set_bit());
// 3. Wait. This is the step that hangs if the oscillator never comes up.
while !dp.hfrcoem23_s.status().read().rdy().bit_is_set() { /* spin */ }
```

Step 2 is the one people miss. It is also the one that makes step 3 a real wait instead of an instant pass — and an instant pass is worse than a hang, because it looks like success.

5.3.2 The Converter That Hung on FSRCO

The microphone's converter (IADC0) had two separate problems in this project, and one of them was purely a clock choice. Clocked from `FSRCO`, the converter's disable sequence never completed:

```
# after writing EN = 0 and waiting for the disabling bit to clear
IADC0_STATUS  = ...DISABLING = 1     (still, after 100 ms)
# the next configuration write then faulted, because the peripheral was mid-synchronisation
```

The same code, clocked from `HFRCOEM23`, disabled in microseconds and never faulted. No register value changed; only `em01grpaclkctrl`. That is the entire difference between a working microphone driver and an afternoon of bus faults, and it was found by trying the other oscillator rather than by reading harder.

5.3.3 The Rule That Came Out of It

Before blaming a peripheral, confirm that it is *clocked*, and confirm it in the way the hardware reports it:

1. read the gate bit back — not the value you wrote, the value the register holds;
2. read the oscillator's ready status, not just its enable bit;
3. if the peripheral has a synchronisation status, read it: `DISABLING`, `SYNCBUSY` and `RESETTING` are the hardware telling you the difference between "configured" and "ready".

A peripheral that reads as zero after a correct-looking write has failed one of those three, and the three have different fixes. Without the read-backs they are indistinguishable — which is why every driver in this repository returns an error instead of a value when a readiness wait expires.

5.4 Delay Without Lying

There are three ways to wait on a Cortex-M and this project has been wrong about all three.

5.4.1 asm::delay Is Not a Cycle Count

```
// "1000 cycles", according to the name and to every example on the internet.
cortex_m::asm::delay(1000);
```

Measured against the cycle counter, one iteration of the loop that `asm::delay` expands to takes about **three** cycles on this part with this compiler. So `delay(1\_000\_000)` waits three million cycles, not one — a 3× error, and the reason a 2.7 kHz tone computed with it came out as something else entirely.

Two consequences, both practical:

- **Never derive a period from it.** A tone, a baud rate or a bus half-period computed with `asm::delay` is wrong by a factor nobody wrote down.
- **Do not multiply by three and call it fixed.** The factor is a property of the compiler's code generation, not of the hardware; it changed between optimisation levels in this project.

5.4.2 SysTick: Real Milliseconds

SysTick counts CPU cycles and can therefore produce real time:

```
pub fn delay_us(&mut self, us: u32) {
    let cycles = (u64::from(us) * u64::from(self.sysclk_hz)) / 1_000_000;
    // SysTick's reload is 24 bits: split anything larger into chunks.
    let mut remaining = cycles;
    while remaining > 0 {
        let chunk = remaining.min(0x00FF_FFFF);
        self.syst.set_reload(chunk as u32);
        self.syst.clear_current();
        self.syst.enable_counter();
        while !self.syst.has_wrapped() {}
        self.syst.disable_counter();
        remaining -= chunk;
    }
}
```

It has one owner — the board — and it must never be shared with a scheduler that also wants it. In this project that rule is enforced by the type system in the smallest possible way: `SYST` is taken once by `Board::take()`, and `board.delay()` hands it out.

5.4.3 Ever the Millisecond: Do Not Divide Early

A defect worth its own paragraph, because it looks like arithmetic hygiene:

```
// The sample interval arrives as microseconds: 2500 us, i.e. 416 Hz.
let dt_ms = dt_us / 1000;        // 2 ms
// Every timeout in the state machine is now 20% short.
```

Integer division by 1000 truncated 2500 to 2, and every threshold derived from it inherited a 20error. The classifier's free-fall window, its landing window and its settle timer were all wrong at once, and every one of them was measured against real drops before anyone suspected the unit.

The rule: **carry the finest unit the hardware gives you and divide only where a human reads it**. This project's classifier now takes microseconds everywhere and converts once, at the boundary, for display.

5.4.4 The Cycle Counter: Measurement, Not Delay

The third instrument is not for waiting at all:

```
let start = dwt::cycles();
// ... work ...
let elapsed_us = dwt::cycles_to_us(dwt::since(start), 19_000_000);
```

Two registers and no ownership questions: enable `TRCENA` in `DEMCR`, then `CYCCNTENA` in `DWT->CTRL`, then read `CYCCNT`. It is how every timing number in this book was obtained, and it is the only instrument here that cannot lie about the clock because the CPU *is* the clock.

One trap, measured: a counter that was never enabled reads as zero, and zero elapsed cycles looks exactly like a loop that ran impossibly fast. Compare two consecutive reads before trusting either.

5.5 What to Write Down, and Where

A clock fact has three places it can live, and only one of them survives the project.

5.5.1 The Three Places

Place	Lifetime	What happens to it
In the chat where you found it	minutes	lost
In a commit message	months, if you search	nobody searches
In the source, next to the constant it explains	the life of the code	**read by the next person**

Table 5.3: The only durable location is the one somebody will read while editing the thing it describes.

5.5.2 How the Numbers Here Are Written

Every measured constant in this project carries three things: the number, the unit, and how it was obtained. For example:

```
/// Half a bit period on the bit-banged I2C bus, in loop iterations.
///
/// Measured: 60 iterations take ~5 us at SYSCLK = 19 MHz on this board, so ~95 kHz.
/// This is a measurement, not a datasheet value -- `asm::delay` is about three cycles
/// per iteration here, so the divisor is not the clock.
const HALF_BIT_CYCLES: u32 = 60;
```

and for the value that everything else depends on:

```
/// SYSCLK as measured with a stopwatch over 20 blink cycles: 19 MHz (the HFRCO default).
/// The part supports 78 MHz; this board's firmware never selects it.
pub const DEFAULT_SYSCLK_HZ: u32 = 19_000_000;
```

Quantity	Value here	How it was measured
SYSCLK	19 MHz	stopwatch over 20 blink cycles
`asm::delay` cost	3 cycles/iteration	cycle counter, two build profiles
I2C half period	5 μs (60 iterations)	scope-free: 95 kHz against a known device
Buzzer tone	2.7 kHz, TOP = 3518	TIMER0 TOP read back over SWD
IMU sample rate	416 Hz configured	`CTRL1\_XL` register + measured intervals
Sampling loop	158 Hz when claiming 400	cycle counter, per-sample interval recorded

Table 5.4: The clock facts of this project. The last line is the one that mattered: a loop that reports a rate it does not achieve poisons every timing test downstream.

5.5.3 The Table That Belongs in Every Project

5.5.4 The One That Generalises

The last row of that table is the lesson of this chapter. The firmware said 400 Hz because the *intended* interval was 2500 μs. It ran at 158 Hz because the display flush blocked for tens of milliseconds inside the loop. Every threshold in the state machine was therefore 2.5× fast, and three rounds of drop tests were interpreted against a wrong time base before anyone noticed that four different drops showed the same 0.62 s of calm.

The fix was not a better delay. It was to **measure the interval between samples and pass the measurement on**:

```
let now = dwt::cycles();
let dt_us = dwt::cycles_to_us(dwt::since(self.last), sysclk_hz);
self.last = now;
// The classifier is told what actually happened, not what was planned.
detector.update(&accel, dt_us);
```

Every timing defect in this project — and there were four — would have been caught on the day it was introduced by that three-line habit.

- A clock tree is three separate decisions: which oscillator, which branch, and which peripheral gate. Getting one of the three wrong produces the same symptom — a peripheral that reads as zero.
- SYSCLK on this board is the HFRCO default, measured at 19 MHz. The part can do 78 MHz; the board does not, until someone configures it.
- Selecting an oscillator is not starting it. Force it on, then wait for the ready bit.
- `asm::delay(n)` is not `n` cycles — it measured about three per iteration here, which is a 3× error in every derived period.
- A loop's *intended* rate is not its actual rate. Measure the interval between samples and pass the measurement on, never the intention.

6. Phase B: Communication Peripherals - USART

This chapter content will be added

7. I2C Master Mode: Two Buses, One That Did Not Work, and Two Lines That Were Swapped

"The datasheet is a description of the hardware. It is not the hardware."
– the lesson of this chapter

I2C looks like the easiest peripheral in the machine: two open-drain wires, an address, a byte, an acknowledge. It is also the peripheral where this project lost the most time, and every hour was lost for the same reason — trusting a document over a measurement.

This chapter builds an I2C master from the register level up, and then does something most texts skip: it shows what happens when the on-chip controller does not work, how to prove that it is the controller and not your code, and how to build a transport that does work instead. Along the way it solves a real failure: a sensor that never answered because the two bus lines were the other way round, and every scan of the "correct" pins returned an empty bus.

Contents

5.1 The Tree, in the Order You Meet It . . . 62
5.1.1 Question 1: Which Oscillator . . . 62
5.1.2 Question 2: Which Branch . . . 62
5.1.3 Question 3: Which Gate . . . 62
5.1.4 The Three Questions, Together . . . 62
5.2 The Clock You Forgot You Were Using . . . 62
5.2.1 19 MHz, Not 78 . . . 63
5.2.2 Why the Error Is Invisible . . . 63
5.2.3 The Right Shape for the Constant . . . 63
5.2.4 Measuring It Properly . . . 63
5.3 Selecting an Oscillator Is Not Starting It . . . 63
5.3.1 The Two-Step That Is Actually Three . . . 64
5.3.2 The Converter That Hung on FSRCO . . . 64
5.3.3 The Rule That Came Out of It . . . 64
5.4 Delay Without Lying . . . 64
5.4.1 asm::delay Is Not a Cycle Count . . . 64
5.4.2 SysTick: Real Milliseconds . . . 65
5.4.3 Ever the Millisecond: Do Not Divide Early . . . 65
5.4.4 The Cycle Counter: Measurement, Not Delay . . . 65
5.5 What to Write Down, and Where . . . 65
5.5.1 The Three Places . . . 65
5.5.2 How the Numbers Here Are Written . . . 65
5.5.3 The Table That Belongs in Every Project . . . 66
5.5.4 The One That Generalises . . . 66

7.1 I2C Protocol Fundamentals

I2C is two wires and a clock, and everything else follows from one electrical decision: the lines are *open drain*. A device never drives a line high; it releases it and a pull-up resistor does the work. Two consequences drive every design decision that follows.

- A device can only pull a line *low*. "High" is the absence of anybody pulling.
- Because releasing is not a conflict, two devices can never damage each other by talking at once — the bus resolves it as a wired-AND.
- But because a "1" comes from a resistor, the rise time is set by $R \times C$: the bus capacitance is what limits the clock rate and the wire length.

7.1.1 The Transaction, in Voltage

A transfer is a sequence of edges, and each of them means something specific:

1. **START**: SDA falls while SCL is high. Nothing else on the bus produces that pattern, which is why it can be used as a marker without a separate wire.
2. **Address**: seven bits, most significant first, then a direction bit (0 = write, 1 = read).
3. **ACK**: the addressed device pulls SDA low for the ninth clock. If nobody pulls, the byte is unacknowledged — the single most useful diagnostic on the bus.
4. **Data**: bytes, each followed by an ACK, in the direction agreed.
5. **STOP**: SDA rises while SCL is high.

The detail that catches everybody the first time: **SDA may only change while SCL is low**. Both the START and the STOP are defined as the violation of that rule, which is exactly why they are unambiguous.

7.1.2 The Electrical Reality on This Board

The EFR32MG24 does not have a dedicated I2C pin pair in the sense a beginner expects. The peripheral observes ordinary GPIO pins, so the same pin can be used as a GPIO, as an I2C line, or as something else — and the routing is a separate configuration step.

That gives two ways to implement a bus on this part:

The on-chip controller (I2C0/I2C1). The hardware generates the edges, handles addressing, ACK and clock stretching, and interrupts the core when done.

A bit-banged master. Software drives the pins. Slower, more code, and completely under your control — which matters enormously when the first option misbehaves.

This chapter uses both, and the reason is not stylistic. On this part the on-chip controller could not complete a read, and the proof of that is the subject of the next section.

7.1.3 Open Drain in the Registers

An open-drain pin is not configured "as output and set to 1". It is configured in a mode where the output driver can only sink current, and the value written has the opposite meaning to the one intuition suggests:

```
// The line is released (reads high through the pull-up) when the output latch is 1,
// and pulled low when it is 0. This is not a convention we chose: it is what WIREDAND
// means, and it is why a bit-banged master writes `1` to let go.
porta_dout().write(|w| unsafe { w.dout(1 << pin) }); // release
porta_dout().write(|w| unsafe { w.dout(0) });        // drive low
```

Getting this backwards produces the most confusing failure in the whole chapter: a bus that appears dead, while the code looks correct.

The mistake that cost the most time in this project. The GPIO was configured for the bus, the address was right, the device was powered, and every scan returned nothing — because the pin was in a mode where writing `1` *drove the line high* instead of releasing it. Two open-drain lines both driven high is a short through the drivers, and the resulting waveform is not an I2C bus at all.

7.2 The Controller That Could Not Read

This section is a worked example of a specific and unglamorous engineering skill: proving that a peripheral, and not your code, is at fault. It matters because the alternative — guessing — is unbounded. You can spend days rewriting a driver that was never the problem.

7.2.1 The Symptom

An I2C0 master was configured with the PAC: bus clock enabled, pins routed, clock divider set, address written, and a read requested. Writes completed. Reads did not: the transfer never finished, and the state register reported the controller waiting in the transmit path with the bus already acknowledged as finished for the write half.

The measurement that matters, taken from `I2C0->STATE` after a write-then-read sequence:

```
I2C0 STATE = 0xd7
    BUSY         = 1
    TRANSMITTER  = 1
    NACKED       = 1
    RXDATAV      = 0     <- no receive data will ever arrive
```

`RXDATAV = 0` is the whole story: the controller accepted the read command and never received a byte into its buffer. There is nothing to read out, so any polling loop written against `RXDATAV` waits forever.

7.2.2 What Was Ruled Out, and How

A diagnosis is only worth reading if it says what it eliminated and on what evidence. The table below is the actual list, in the order it was worked through.

Hypothesis	Evidence against
Wrong address	The same address acknowledged during the write half of the transfer.
Wrong pins	A bit-banged master on the same two pins read the device immediately.
Missing pull-ups	The pins read high with every GPIO configured as input, no internal pull: the pull-ups are on the board.
Missing bus clock	`CMU\_CLKEN0.I2C0` read back set; with it cleared the peripheral faults instead of running.
Bad clock divider	Two dividers, including the slowest available, produced the same `STATE`.
The device itself	The device answered a bit-banged transaction on the same bus, at the same address, in the same second.

Table 7.1: Hypotheses eliminated before concluding that the on-chip controller was at fault.

Fourteen measurements across these hypotheses produced the same `STATE` every time. That repetition is the point: a single failing attempt is consistent with a bug, a dozen identical failures along different paths are consistent with the hardware.

The control experiment is the one that decided it. The bit-banged master used the same pins, the same pull-ups and the same device. It read the correct identifier on the first attempt. That single result moved the fault from "our driver" to "this controller, on this part, in this configuration", and it took two minutes to run.

7.2.3 What To Do About It

There are three honest responses to a peripheral that does not work, and only one of them is acceptable in a project that has to ship.

1. **Rewrite the driver.** Cheap to try, unbounded to continue. Only justified while the control experiment has not been run.
2. **Build a transport that works.** Slower, but fully understood, and it turns the bus into something you can debug byte by byte. This is what this project did, and it is the subject of the next section.
3. **Find the missing configuration.** The correct answer when it exists. It does not always: this chapter does not claim the controller is broken in general, only that it could not complete a read under every configuration that was tried, and that a working alternative was available.

Write down which one you chose, and why. A project that silently switches to a workaround loses the knowledge that it did. This project keeps the fourteen measurements in `docs/04-reference/I2C0-READ-DEFECT.md`, so that a later engineer — or a later silicon revision — can pick up the investigation instead of repeating it.

7.2.4 Why This Belongs in a Book

Because it is the normal experience of driving new hardware, and textbooks rarely show it. The registers are documented, the sequence is correct, and the thing still does not work. The skill being taught here is not I2C: it is how to reduce a hardware failure to a statement you can defend, and then how to keep shipping while it is unresolved.

7.3 A Transport That Works: The Bit-Banged Master

Once the on-chip controller is out of the picture, the bus becomes software, and the software can be made small enough to hold in your head.

7.3.1 The Whole Protocol in Four Primitives

Every I2C transfer — a register write, a 12-byte burst, a scan of 128 addresses — reduces to four operations on the two lines:

```
/// Let SDA float (the pull-up makes it a 1) or pull it to ground.
fn sda(&mut self, high: bool);
/// Same for the clock.
fn scl(&mut self, high: bool);
/// Let the bus settle for half a bit period, then sample SDA back.
fn read_sda(&self) -> bool;
/// Half a bit period of wall time.
fn half_bit(&self);
```

With those, a byte is a loop and a START is two writes:

```
fn start(&mut self) -> bool {
    self.sda(true);
    self.scl(true);
    self.half_bit();
    self.sda(false);        // SDA falls while SCL is high: START
    self.half_bit();
    self.scl(false);
    self.half_bit();
    true
}

fn write_byte(&mut self, byte: u8) -> bool {
    for bit in (0..8).rev() {
        self.sda(byte & (1 << bit) != 0);
        self.half_bit();
        self.scl(true);
        self.half_bit();
        self.scl(false);    // data is sampled by the device on the rising edge
    }
    // Ninth clock: release SDA and see whether anybody pulls it down.
    self.sda(true);
    self.half_bit();
    self.scl(true);
    self.half_bit();
    let acked = !self.read_sda();
    self.scl(false);
    acked
}
```

The acknowledge is the only place where the master *reads*. Reading SDA during a write is what allows the driver to report "nobody is home" instead of silently succeeding — and that distinction is what makes a bus scan possible at all.

7.3.2 Timing Without a Timer

The half-bit period does not need an interrupt or a timer peripheral. A counted loop is enough, and its length is calibrated against something known: the CPU clock.

```
/// ~5 us at the 19 MHz this board runs at, which is a 100 kHz bus.
const HALF_BIT_CYCLES: u32 = 60;

fn half_bit(&self) {
    cortex_m::asm::delay(HALF_BIT_CYCLES);
}
```

`cortex\_m::asm::delay(n)` does **not** execute exactly n CPU cycles. On this part it ran about three times faster than the name suggests, which turned a nominal 100 kHz bus into a 300 kHz one — and turned a buzzer calibration into a 3× error elsewhere in the same project. Anything that claims a frequency should be measured with the cycle counter (`DWT->CYCCNT`), not assumed from a loop count.

7.3.3 The `embedded-hal` Face

The transport is more useful if the rest of the ecosystem can use it. `embedded-hal` v1.0 defines the trait, and the implementation is a thin wrapper over the primitives above:

```
impl<B: I2cBus> embedded_hal::i2c::I2c for SharedBus<'_, B> {
    fn transaction(&mut self, address: u8, operations: &mut [Operation<'_>])
        -> Result<(), Error>
    {
        // One START, the operations in order, one STOP. A register read is a write of the
        // register number followed by a repeated START and a read: getting the repeated
        // START wrong is the second most common cause of a silent bus.
        ...
    }
}
```

Designing the driver against the trait rather than against the pin-level functions is what allowed the same SSD1306, BME68x, PCF8563 and LSM6DS3TR-C drivers to be written once and used on two different buses — one on PC04/PC05, another on PB2/PB3 — without changing a line of driver code.

7.3.4 Two Buses on One Board

The board this book was written against has two I2C buses, and the reason is worth recording:

Bus	Pins	Devices	Notes
I2C on the base	PC04 (SDA), PC05 (SCL)	OLED 0x3C, BME68x 0x76, RTC 0x51	pull-ups on the bas
IMU bus	PB2 (SDA), PB3 (SCL)	LSM6DS3TR-C 0x6A	external pull-ups, c

Table 7.2: The two I2C buses of the XIAO MG24 Sense on the XIAO Expansion Base.

That second row is where the chapter's hardest lesson lives, and it is the subject of the next section.

7.4 Two Lines, Swapped: A Case Study in Not Trusting a Pin Map

A six-axis IMU was supposed to be on this board at address `0x6A`. The vendor's documentation said so, the vendor's example code said so, and the board's own pin map said which two pins carried the bus. Every scan of those pins returned an empty bus: not a mismatch, not garbage — nothing at all, as though no device existed.

It existed. The two lines were the other way round.

7.4.1 What the Documentation Said

Three sources, all consistent and all wrong in the same direction:

- The vendor's product page: the IMU answers at `0x6A`.
- The vendor's example: `LSM6DS3 myIMU(I2C\_MODE, 0x6A)`, and a library header that selects `Wire1` rather than the default bus.

- The board support package's pin map: `SDA1 = PB3`, `SCL1 = PB2`.

The third line is the one that matters, and it is the one that is wrong. On this board the data line is **PB2** and the clock is **PB3**.

Why a swapped pair produces silence rather than an error. Both lines idle high through their pull-ups. If the master drives what it believes to be the clock while the device is watching it as data, the START condition — SDA falling while SCL is high — never occurs. The device never wakes up, never acknowledges, and never reports anything, because no transaction ever began. Every failure mode of a wrong pin assignment looks like an empty bus, not like a bad byte.

7.4.2 How It Was Found

The useful part of this story is not the answer; it is the method, which is reusable on any board with an unknown bus.

Step one: prove the pins are a bus at all. Before talking to anything, configure every GPIO as an input with no internal pull, and read the port. Two pins that read high with no internal help are held up by resistances on the board — which is what an I2C bus looks like, and nothing else on a general GPIO header does.

```
PORTB_DIN = 0x0C    ->  bit 2 (PB2) and bit 3 (PB3) are both high, no internal pull
```

The first version of this probe was useless: it configured the pins as *outputs* (`mode 0`), which disables the input buffer, so the result carried no information at all. `mode 1` — input, no pull — is the measurement.

Step two: stop assuming which pin is which. Instead of testing one assignment, test the matrix. Every plausible pin pair, in both orders, against both addresses the device could answer on:

```
8 pin pairs x 2 orders x 2 addresses = 32 probes
result: sda=p1.2 scl=p1.3 addr=0x6a id=0x6a
```

That single line is the whole fix. The device had been there the entire time, on the pair the documentation listed — with the two wires the other way round.

Step three: make the driver's configuration a value, not a hardcoded write. The first attempt after the fix still failed, for a second and much more embarrassing reason: the pin-abstraction's write function always addressed the same port, so changing "which bus" changed nothing about the electrical output while appearing to work. The lesson is small and general:

```rust
// Wrong: the selector looked configurable but every branch wrote PORTB.
fn set(&mut self, pin: u8, high: bool) {
    self.portb_dout().write(|w| unsafe { w.dout(if high { 1 << pin } else { 0 }) });
}

// Right: the port is part of the state, so choosing a bus chooses the registers.
match self.pins {
    Pins::Base => self.portc_dout().write(...),
    Pins::Imu  => self.portb_dout().write(...),
}
```

The pattern to notice. A parameter that does not reach the hardware is worse than a missing parameter: it produces a plausible-looking experiment whose result is meaningless, and you will believe the result.

7.4.3 Verification

The fix was accepted only when two independent implementations agreed:

Implementation	Reading	Expected
PAC example	`id=6a a=985,81,139 mg`	WHO_AM_I = 0x6A, $\lvert a\rvert \approx 1$ g
HAL example	`id=6a a=970,79,142 mg`	same device, same registers

Table 7.3: Two layers, one device: the numbers differ only by sensor noise and the physical orientation at the moment of the read.

A magnitude of about 1000 mg is not a detail to skip: a three-axis accelerometer at rest measures gravity, so the vector length is a free self-check on both the bus and the scale-factor arithmetic. A dead sensor reports zeros; a mis-scaled one reports something that is not 1 g.

7.4.4 What This Teaches

- **A pin map is a claim, not a measurement.** Check the electrical behaviour of the pins before believing a table in a repository.
- **Test the matrix, not the hypothesis.** Eight pairs in two orders took less time than a day of reasoning about which document was right.
- **Silence is informative.** An empty bus narrows the problem to "no transaction started", which points at wiring, mode or timing — never at the protocol layer.
- **Verify with a physical invariant.** Gravity, a chip identifier, a checksum: something that would be wrong if the bus were returning plausible garbage.

7.5 What To Take Away

This chapter covered one peripheral, but three of its lessons apply to every peripheral in this book.

- **The order of diagnosis is electrical, then protocol, then driver.** Prove the pins, prove the timing, prove the bytes *in that order*. Skipping to the driver is how a project loses a week to a mode bit.
- **Controls turn opinions into statements.** "The controller does not complete reads" is an opinion until the same pins, pull-ups and device are read successfully by a bit-banged master. Then it is a finding.
- **A workaround is a decision, and decisions get written down.** The bit-banged transport exists because the on-chip one could not be made to work in any configuration that was tried. That is a legitimate engineering outcome, and it is only legitimate while the measurements behind it are kept.
- **Every abstraction must be tested against the hardware it claims to abstract.** The pin abstraction whose "which bus" parameter never reached a register is the canonical example: it compiled, it ran, and it measured nothing.

7.5.1 Exercises

1. **(Easy)** Write a bus scan that reports, for every address that acknowledges, the byte read back from register `0x00`. On a bus with three devices, how do you tell a device that answered from one that is not there?

2. **(Easy)** Configure a GPIO as an input with no internal pull and read a pin that has an external pull-up, then one that does not. Compare the port values. What does the difference prove about the board?

3. **(Medium)** Add a repeated-START to the bit-banged master and use it to read a register in a single transaction. Then implement the same read as two transactions and explain, with the device's datasheet, which devices tolerate the difference.

4. **(Medium)** Calibrate `HALF\_BIT\_CYCLES` with the DWT cycle counter instead of trusting the constant, and report the bus frequency you actually get. Compare with the value the name suggests.

5. **(Hard)** Reproduce the fault-isolation table of this chapter for a peripheral of your choice: list every hypothesis, the measurement that eliminates it, and the control experiment that decides between "our code" and "the hardware".

Where this leads. The transport built here is what every other device driver in this book sits on: the OLED, the environmental sensor and the real-time clock all use the same bit-banged master, on their own bus, through the same `embedded-hal` trait. The next chapter takes the first of them — the display — and shows what changes when the payload is a framebuffer rather than a register.

8. Phase B: SPI Master Mode (Three Variants)

This chapter content will be added

9. Phase C: Advanced Peripherals - DMA, ADC, and Power Management

This chapter content will be added

Part III

Wireless Protocol Stacks

10. Phase D: Wireless Protocol Stacks - RAIL Foundation

This chapter content will be added

11. Phase D2: Bluetooth 5.3 Low Energy

"A radio stack is the one part of an embedded project you do not write."

Bluetooth on this part is not a peripheral. It is a second processor's worth of protocol logic, delivered as three binary libraries with a C API, and the job of a Rust project is to make that API usable without pretending to reimplement it.

This chapter is that job, end to end: what the three blobs are, how the bindings are generated and where they lie, the symbols nobody but you can provide, and what a beacon and a GATT peripheral actually cost in code. Every claim here was measured on this board's SDK, and the places where it does not work yet are marked as such.

11.1 What the Stack Actually Is

Before writing a line of binding code, three questions: what is in the binary, how do you talk to it, and what does it expect you to do.

11.1.1 Three Libraries, One Protocol

Library	What it is	Size on this part
`libbluetooth\_controller`	link layer: radio, timing, packets	600 KB
`libbluetooth\_host`	GAP, GATT, SM: the protocol logic	700 KB
`libbgcommon`	shared buffers, BGAPI parsing	40 KB

Table 11.1: The three blobs of a Gecko SDK 4.x Bluetooth build. There is no fourth: the profiles and services you need are your code.

Together they are larger than most applications on this part, which is why a project chooses Bluetooth deliberately rather than by default. All three are MSLA licensed and therefore not in this book's repository; a clean checkout compiles the bindings and fails at link with a list of missing symbols, which is the correct behaviour.

11.1.2 BGAPI: Commands In, Events Out

The API is asynchronous and message-based, and both halves matter:

```
command:  sl_bt_advertiser_set_data(handle, ...)   ->  sl_status_t (immediate)
reply:    sl_bt_evt_advertiser_set_data_complete   (later, in the event queue)
```

Every command returns a status that says "accepted" or "refused" — never "done". The completion arrives as an event, and an application that does not drain the event queue eventually stops being able to send commands, because the queue is finite and the stack does not grow it.

```
loop {
    if let Some(event) = ble.poll_event() {
        match event { /* Connected, Disconnected, GattWrite, ... */ }
    }
}
```

That loop is not optional and it is not decoration: it is the other half of every call.

11.1.3 What the Stack Assumes You Have Done

The blobs are written for a vendor project, and they assume the surrounding C world exists:

- **Clocks.** The radio needs the HFXO; the stack enables it, but if the crystal never settles the boot event never arrives.
- **Critical sections.** The CMSIS `CORE\_*` helpers, which in a vendor project come from headers as `static inline`.
- **An event loop.** Yours.
- **Memory.** Fixed buffers inside the blobs, sized by the stack configuration in the header. A Rust project cannot change them without regenerating that configuration.

- **Crypto** — for any connectable, bondable device, a PSA implementation the blob calls but does not contain.

None of that is in the API documentation, and all of it appears as a link error or a timeout on first use. Section 11.3 is about the link errors; section 11.4 is about the timeouts.

11.2 The Bridge: bindgen Over a Vendor Header

Generating Rust bindings from `sl_bt_api.h` is one command and three surprises. All three of this project's were compile errors in the wrapper, and all three are the kind that look like mistakes in your own code.

11.2.1 Surprise 1: The Union Is Emulated

The header declares its event payload as an *anonymous* union inside a struct. bindgen represents anonymous unions with `\_\_BindgenUnionField<T>`, regardless of the Rust target requested — measured: adding `.rust\_target(Stable\_1\_71)` left the generated file byte-identical.

The consequence is 18 errors of the form:

```
error[E0609]: no field `connection` on type
              `&__BindgenUnionField<sl_bt_evt_connection_opened_s>`
```

The fix is not a bindgen flag. It is to read the payload the way the emulation requires:

```
// before: reads the wrapper as if it were the union
let data = &evt.data.evt_connection_opened;

// after: says out loud that the active member is the caller's responsibility
let data = unsafe { evt.data.evt_connection_opened.as_ref() };
```

That `unsafe` is the correct amount of friction. Reading a union is unsafe because nothing checks that the member you chose is the one the sender wrote — and the event ID is the only thing that tells you.

11.2.2 Surprise 2: A Flexible Array Member Is Not a Slice

```
pub struct uint8array {
    pub len: u8,                        // "< Number of bytes stored in data"
    pub data: __IncompleteArrayField<u8>,
}
```

`data` has no length and no `Index`, so `data.value.data[i]` does not compile. The bytes are read through the pointer, with the length taken from the header's own field:

```
let byte = unsafe { data.value.data.as_ptr().add(i).read() };
```

Two facts worth keeping: the length is authoritative (it was written by the sender), and the pointer arithmetic is the only way to reach a flexible array — which is exactly why C APIs that return them need a total-length parameter somewhere.

11.2.3 Surprise 3: One Field, Two Names

```
sl_bt_evt_gatt_server_attribute_value_t      ->  attribute: u16
sl_bt_evt_gatt_server_user_read_request_t    ->  characteristic: u16
```

The same 16-bit attribute handle, named differently in two events of the same SDK version. A wrapper that assumes one name fails to compile on the other, and a wrapper that "fixes" it by renaming the Rust type breaks the one that was right.

The project's rule: convert at the boundary, once, with a comment naming both spellings. The application-facing type has one field, `attribute`, and the FFI layer knows about the vendor's inconsistency so nothing above it has to.

11.2.4 What the Generated Code Is Good For

With those three handled, the bindings are genuinely useful in the way the SVD's are: every field carries the header's doc comment, so `cargo doc` shows the vendor's own description of the value you are about to write. That is the strongest argument for generating bindings rather than hand-writing them — not the function signatures, which are mechanical, but the documentation that travels with them.

11.3 The Symbols Nobody Provides

The bindings compile, the blobs are in the link path, and the linker still fails — on functions that exist nowhere in the SDK's public surface. This section is the complete answer for this board, because it cost a full session to assemble.

11.3.1 The CMSIS Helpers

```
undefined symbol: CORE_EnterAtomic
  >>> referenced by bgbuf.c.obj:(bgbuf_free)
undefined symbol: CORE_ExitAtomic
  >>> referenced by ll_hci.c.obj:(ll_hciDataFromHost)
```

In a vendor project these are `static inline` in the CMSIS headers, so the C compiler inlines them and no symbol is ever needed. The blob was compiled against those headers, so its object code contains *references*, and the application is expected to provide the definitions.

The definitions are four lines each, and the two that matter differ:

```c
uint32_t CORE_EnterCritical(void) { /* masks with BASEPRI */ }
uint32_t CORE_EnterAtomic(void)   { /* masks with PRIMASK */ }
```

This difference is not cosmetic. `BASEPRI` masks interrupts at or below a chosen priority and leaves priority 0 running; `PRIMASK` masks everything. A stack that enters a buffer-manipulating critical section with the wrong one still links, still works at idle, and corrupts its attribute buffer once a connection is active. Implement them from the CMSIS semantics, not from a guess about what "critical" means.

11.3.2 Provided Once, for Every Stack

This board links two blob families — Bluetooth and RAIL — and both want the same helpers. There are two ways to get that wrong:

- **Compile the C file in each crate.** Two archives with the same symbols: either a duplicate-symbol link error, or one silently wins and the other's semantics are lost.
- **Give each its own copy.** Works, until the two copies drift and only one is updated.

This project's answer is a build-support crate that compiles the file once, which both stacks depend on. A crate with no Rust API, whose entire contract is "linking me gives you these symbols":

```toml
[dependencies]
cmsis-stubs = { path = "../cmsis-stubs" }
```

11.3.3 The #[link] Directive That Is Not Obvious

Having the crate as a dependency is not enough: nothing in Rust *calls* those symbols, so the linker never pulls the archive in, and every symbol comes back undefined while the archive sits in the search path. The missing piece is an explicit directive:

```
#[link(name = "cmsis_stubs", kind = "static")]
extern "C" {}
```

Five lines that do nothing at runtime and everything at link time. They belong in the crate whose objects need the symbols — the bindings crate — not in the helper crate.

11.3.4 The Rule That Generalises

1. Read the link errors in full, once, and list every symbol. The linker names the object file that wanted each one (`bgbuf.c.obj`, `ll_hci.c.obj`), which tells you which layer is asking.

2. Decide, per symbol, whether it is *data you must supply* (CMSIS, emlib) or *a library you must vendor* (PSA crypto). They have completely different fixes.

3. Supply data once for the whole binary; vendor libraries with their licence in mind.

4. Declare the link directive where the reference is, and comment why — because the next reader will see an empty `extern` block and delete it.

11.4 A Beacon and a GATT Peripheral

Two examples, two rungs. The beacon proves the stack boots and the antenna works; the GATT peripheral proves a connection, a subscription and a notification path. Everything else in Bluetooth is a variation on those.

11.4.1 The Beacon: Nine Lines and a Phone

```
let mut ble = Bluetooth::beacon(&mut delay, "xiao-mg24")?;
loop {
    if ble.poll_event().is_some() {
        led.on();  delay.delay_ms(20);  led.off();
    }
}
```

What has to be true for a phone to see the name:

1. **The stack boots.** `Bluetooth::new` resets the controller and waits for `system\_boot`. If the HFXO never settles, this times out — a board fault, not a radio one.

2. **The payload fits.** Legacy advertising carries **31 bytes** per PDU. A complete name costs `2 + len`; a 128-bit UUID costs 18; the flags cost 3. A name of 20 characters and a 16-bit service UUID is already 27 of the 31.

3. **Advertising actually starts.** `start\_advertising` returns a status; a non-zero status is a refusal, and the reason is in the event queue, not in the return value.

Step	What it does	The trap
advertise connectable	sets the connectable flag in the PDU	a beacon's PDU is non-connectable; a phone times out with no explanation
wait for `Connected`	gives you the connection handle	the handle is not the advertising set handle, and they are both small integers
wait for a write on the CCC	the client subscribed	a notification sent before this is dropped by the stack, not queued
notify	pushes a value	the transmit queue is finite: refusals must be counted, not ignored

Table 11.2: Four steps, four traps. Three of the four produce no error — the value simply does not arrive.

11.4.2 The GATT Peripheral: Four Steps and Four Traps

11.4.3 Counting Refusals Is the Whole Discipline

```
match ble.notify(handle, &payload) {
    Ok(()) => notified += 1,
    Err(_) => refused += 1,
}
```

A product that silently loses measurements is worse than one that reports them late. The counter is the difference between "the phone did not get them" and "the board could not send them", and it is the first thing to read when a client reports gaps.

11.4.4 The Beacon Is the Debugging Instrument

Before writing the GATT path, prove the beacon. It exercises the same stack, the same blob, the same antenna and the same boot sequence, and it fails for a strictly smaller set of reasons. A project that debugs connectable advertising and beacon advertising at the same time is debugging two things at once with one symptom.

11.4.5 What a Client Can Prove, and What It Cannot

A read-back from a client is not evidence that the value crossed the air: the host's Bluetooth daemon caches attribute values, and 'bluetoothctl' will happily answer a read from that cache. The proof that survives the objection is an *echo*: the client writes a characteristic, the firmware copies those bytes into a second characteristic and notifies them, and the client compares what it receives with what it wrote. Bytes that come back from inside the radio had to cross the link in both directions.

```
client: WriteValue 5a a5 d0 b8        ->  characteristic with write
board:  cv=5aa5d0b8  (recorded on the board)
board:  notify 5a a5 d0 b8 ... (24 times in 12 s)
client: Echo PASS: 24 notifications carried the bytes this client wrote
```

Publishing a sensor is then a question of which service to publish it in. A vendor-specific 128-bit UUID works, and a generic phone application will show it as bytes; the standard Environmental Sensing service (`0x181A` with `0x2A6E` temperature, `0x2A6F` humidity and `0x2A6D` pressure) is decoded and labelled by any such application, with no profile document from us. The cost is that the GATT database builder has to accept 16-bit UUIDs at all: an API that only exposes 128-bit UUIDs quietly forces every client to parse bytes.

11.4.6 A Connectable Advertiser Stops When Somebody Connects

The failure that costs an afternoon: the server works for the first client and never again. A connectable advertiser stops advertising when a central connects, and unless the `Disconnected`

event restarts it, the board is reachable exactly once. The symptom is indistinguishable from a crashed firmware — the mailbox still says "advertising" — so the restart belongs in the event handling, not in a comment.

11.5 What It Costs, and What Is Still Missing

A chapter about a radio stack that only reports successes would be useless. This one ends with the bill.

11.5.1 The Numbers

Example	Image size	What it needs
`ble\_beacon`	5.9 KB	three blobs, the stubs, a delay
`rail\_version`	1.3 KB	one blob, the stubs
`ble\_gatt`	does not link	the above plus PSA crypto

Table 11.3: Measured with elf-load-range.py. The application is small; the blobs are not, and they are what fills flash.

The three Bluetooth blobs are on the order of a megabyte together. On a part with 1.5 MB of flash and a 24 KiB bootloader, that is affordable once — and it is the reason a project chooses BLE for a product rather than adding it "just in case".

11.5.2 The Gap That Must Be Named: PSA Crypto

A connectable stack references the PSA crypto API, and this SDK build does not ship it in the blobs. The eight symbols are:

```
psa_crypto_init      psa_destroy_key      psa_import_key    psa_mac_abort
psa_mac_sign_finish  psa_mac_sign_setup   psa_mac_update    psa_reset_key_attributes
```

They come from Silicon Labs' `libpsa\_crypto` and `libmbedtls` builds, which are separate artefacts with their own licence terms. Until they are vendored, a GATT example is a staged file rather than a working one — and this book says so instead of shipping it as an example that "should work".

A project that does not need bonding can sometimes avoid the crypto path, but not by accident: the link tells you whether it has.

11.5.3 The Three Things to Verify Before Believing a BLE Build

1. **Version match.** The header and the blobs are from the same SDK. Check a symbol the blob exports (`arm-none-eabi-nm libbluetooth_host*.a | grep sl\_bt\_advertiser`) against the generated bindings.

2. **Link, not compile.** A `-sys` crate that compiles proves nothing about the blobs. Build one example that links.

3. **One real advertisement.** A phone that sees the name is the only proof that the antenna, the crystal and the stack configuration agree. Everything below that is a build check.

11.5.4 Where This Book Stands

```
bindings compile                    yes, with three documented workarounds
beacon links                        yes
advertising through RAIL            yes -- seen by a phone, by BlueZ, and by a connection attempt
connection                          established at the link layer, dropped: no link layer in this
↪  firmware
GATT links                          no: four service symbols still missing (below)
RAIL links for version only         yes; RAIL_Init needs emlib sources
```

The RAIL path now advertises, and `scripts/ble-scan-bluez.sh` is the check that says so at stack level rather than in a capture. GATT stays out of reach for a different reason than the rest of this chapter, and section 11.6 records what it cost to learn the difference.

11.5.5 The Four Symbols Between Here and GATT

Measured by linking, then by reading the call sites in the blobs:

```
sli_psa_context_get_size     r0 = 6|3|0 (context type) -> size passed to bg_zalloc
sli_aes_crypt_ctr_radio      (dst, 128, src, key, arg5, arg6) -- six arguments
sl_sleeptimer_restart_timer  vendored source, needs the SYSRTC interrupt wired
nvm3_open                    vendored source, needs a reserved flash region and
↪  nvm3_default_common_linker
```

None of them is honestly replaceable by a stub: the first returns an allocation size, so a wrong number corrupts the heap rather than failing; the second produces the access addresses, so a zero stub makes every connection predictable; the third drives the stack's timers; the fourth is the persistence the bonding database sits on.

Stating that plainly is the point. A chapter that claimed the last four were done would be worse than no chapter, because the next person would trust it.

11.6 The API Is Not the Air

This section could not have been written before the board was flashing, because everything in it is a measurement that contradicts a success message. It is the part of Bluetooth bring-up that no header documents, because the header is describing the API and the failure is between the API and the antenna.

11.6.1 Four Successes, One Silence

The first working attempt reported, on every link of the chain, that it was fine:

```
RAIL_Init                 handle returned
RAIL_BLE_Init             void
RAIL_BLE_ConfigPhy1Mbps   status 0
RAIL_ConfigTxPower        status 0
RAIL_SetTxPowerDbm(100)   status 0
RAIL_StartTx              status 0
RAIL_EVENT_TX_PACKET_SENT 39529 times   <- counted in the radio's own interrupt
```

and no receiver heard anything. The counter in the last line is the one that makes this worth writing down: it is not the program counting its own calls, it is the radio's interrupt handler reporting that the modem completed a packet. The packet was built correctly too, and read back off the chip as

```
00 14 | 00 11 22 33 44 55 | 02 01 06 | 0A 09 "xiao-mg24"
```

which is a well-formed `ADV\_IND`: public address, general-discoverable flags, complete local name. A scanner at one hundred percent duty cycle heard four other advertisers in the same room, up to 262 reports each, and this board zero times.

The API was not lying about what it was asked. It was reporting completion of a transmission that had no power behind it.

11.6.2 The Crystal Has To Be Selected, Not Merely Started

The radio is clocked from the 39 MHz crystal. Starting it is half the job:

```
HFXO0->CTRL |= HFXO_CTRL_FORCEEN;                       /* the oscillator runs */
CMU_ClockSelectSet(cmuClock_SYSCLK, cmuSelect_HFXO);   /* and now it is the clock */
```

With only the first line, `RAIL\_GetRadioClockFreqHz()` reports 19000 kHz — the internal oscillator — and the BLE channel configuration is refused with `RAIL\_ASSERT\_INVALID\_XTAL\_FREQUENCY`. That assert is not a complaint about the crystal; it is the library saying that the frequency it was given and the frequency it is running at are different numbers, and it is right.

Two smaller traps live in the same paragraph. `CMU\_OscillatorEnable` in this emlib is documented as a dummy function kept for backward compatibility: it compiles, it is callable, and it does nothing. And `CMU\_ClockFreqGet(cmuClock\_HFXO)` returns zero because the function's switch has no case for that clock, so a zero there says nothing at all about the oscillator.

11.6.3 The Amplifier Needs Its Curves, and the Call Is One Line

The gap between the modem and the antenna was a missing initialisation, and the library documents it in a comment inside its own conversion function: without `RAIL\_InitTxPowerCurves` the dBm-to-raw conversion finds no curve and returns 255, its "invalid" value. Silicon Labs' entry point for that is one call:

```
sl_rail_util_pa_init();                  /* loads the PA curves into RAIL */
RAIL_ConfigTxPower(handle, sl_rail_util_pa_get_tx_power_config_2p4ghz());
RAIL_SetTxPowerDbm(handle, 100);
```

The measurement that identifies this failure is a read-back, not a status:

```
                            before                      after sl_rail_util_pa_init()
RAIL_SetTxPowerDbm(100)     0 (success)                 0 (success)
RAIL_GetTxPowerDbm()        -32768  (invalid power)     99   (10 dBm)
RAIL_GetTxPower()           0       (no level)          0x24
```

A radio with no valid amplifier level completes every packet it is asked to send. The success in the first row is true and worthless; the two read-backs are the only rows that describe the air.

11.6.4 The Vector Table Had No Radio In It

Before any of that could be measured, the first transmission faulted: `HardFault`, `CFSR=0x00000100` (imprecise bus error), at `RAIL\_StartTx`. The cause was in the PAC, not in the radio: it declared 46 of the device's 74 interrupts, and every radio one was among the missing — `AGC`, `BUFC`, `FRC\_PRI`, `FRC`, `MODEM`, `PROTIMER`, `RAC\_RSM`, `RAC\_SEQ`, `SYNTH`, `RFECA0/1`. Those slots held zeroes in the vector table, so the first radio interrupt jumped to address zero.

Two further layers hide the same defect, and both were measured:

- The vendor names its handlers `<NAME>\_IRQHandler`. A vector entry saying `FRC` falls short of the blob's `FRC\_IRQHandler` by the suffix.
- A `PROVIDE(FRC = DefaultHandler)` *satisfies* the vector's reference, so the linker never pulls the archive member that defines the real handler. With the aliases in place the program halted inside `DefaultHandler`. Linking the RAIL blob with `+whole-archive` keeps both: a PAC example with no radio still links, and a binary with the radio gets RAIL's handlers.

11.6.5 The Instrument Matters More Than the Hypothesis

Two instruments were needed and neither was the obvious one.

`hcitool lescan` and `bluetoothctl` printed nothing on this host's adapter, including while the host itself was advertising, so both would have been read as "nothing on air". The scan that works is set up through raw HCI with the window equal to the interval — one hundred percent duty cycle — and the reports are read back out of a `btmon` capture, which sees other advertisers the command-line tools never printed.

`RAIL\_GetRadioState` returning idle after each transmission proves only that the state machine moved. The value that identified the fault was the power read-back; the value that proved the modem was working all along was the send-complete event, counted in an interrupt.

11.6.6 What the Working Sequence Is

```
emlib_hfxo_start(); emlib_clock_select_hfxo();
RAIL_Init(&cfg, NULL);                        /* with an events callback */
RAIL_BLE_Init(handle);
RAIL_BLE_ConfigPhy1MbpsViterbi(handle);       /* only the Viterbi 1M config exists here */
sl_rail_util_pa_init();                       /* the PA curves: without this, silence */
RAIL_ConfigTxPower(handle, sl_rail_util_pa_get_tx_power_config_2p4ghz());
RAIL_SetTxPowerDbm(handle, 100);
RAIL_ConfigData(&packetMode); RAIL_SetTxFifo(txBuf, 0, 64);
/* RAIL's channel argument is the PHYSICAL channel; the logical number belongs to the radio
   parameters. rail_ble.h's repeated-transmit example pairs railChannel 0 with logicalChannel 37,
   and physical N is 2402 + 2N MHz: the three advertising channels are (0,37), (12,38), (39,39).
   ↪  */
for (;;) {
  channel = pick(0, 12, 39);
  RAIL_BLE_ConfigChannelRadioParams(handle, 0x555555, 0x8E89BED6, logical(channel), false);
  RAIL_PrepareChannel(handle, channel);
  RAIL_WriteTxFifo(pdu, pduLen, true);        /* the full PDU, header included */
  RAIL_StartTx(handle, channel, RAIL_TX_OPTIONS_DEFAULT, NULL);
}
```

The radio parameters are re-applied per channel because the whitening seed is the channel number, and they take the *logical* number while `RAIL\_StartTx` and `RAIL\_PrepareChannel` take the physical one. Sending 37, 38 and 39 to all three calls — the obvious reading — transmits on 2476, 2478 and 2480 MHz: data channels except the last, so two thirds of the packets go where no scanner looks. That mistake was made here, measured, and it is why an earlier version was heard intermittently.

With that sequence the board advertises, and the proof is a capture from another machine:

```
Address: 55:44:33:22:11:00 (OUI 55-44-33)
Flags: 0x06   LE General Discoverable Mode, BR/EDR Not Supported
Name (complete): xiao-mg24
RSSI: -83 dBm
```

11.6.7 Proof Has Layers, and the Highest One Is the Stack

A capture proves the controller received a packet. It does not prove a host stack kept it, and "a phone will see it" is a claim about a stack. The same advertisement was therefore put through BlueZ:

```
Device C0:DE:00:00:00:01 xiao-mg24
        Name: xiao-mg24
        Alias: xiao-mg24
        RSSI: -73
```

and a connection attempt says where the RAIL path ends:

```
hci0 C0:DE:00:00:00:01 type LE Random connected eir_len 14
[CHG] Device C0:DE:00:00:00:01 Connected: yes
hci0 C0:DE:00:00:00:01 type LE Random disconnected with reason 0
Failed to connect: org.bluez.Error.Failed le-connection-abort-by-local
```

The advertisement is genuinely connectable and the link-layer connection is established; it is then dropped because this firmware advertises and never listens. That is the boundary: advertising is a transmitter, a connection is a link layer, and GATT is a host stack. Three different pieces of the protocol, three different amounts of work.

11.6.8 The Rule This Chapter Adds to the Book

A radio that reports success is not a radio that radiates. Every link in this chain — the channel configuration, the transmission, the modem, the interrupt count — can be green while the antenna is silent, and the only measurement that distinguishes them is one taken *outside the chip*: a receiver, and a read-back of the amplifier's own idea of its power. The build was correct for a long time before it was true.

11.7 One Radio, Three Protocols

Everything in this chapter so far treats the radio as a Bluetooth radio. It is not. It is a 2.4 GHz radio that happens to be running Bluetooth at the moment, and the same silicon is meant to run 802.15.4 — Zigbee and Thread — later. That is not an aspiration, it is measurable in the archive this project already links:

```
$ nm -g librail_efr32xg24_gcc_release.a | grep -c RAIL_IEEE802154
81
$ nm -g librail_efr32xg24_gcc_release.a | grep -c RAIL_BLE
54
```

Eighty-one 802.15.4 entry points and fifty-four Bluetooth ones, in the same binary. Nothing has to be downloaded for the link layer of Zigbee or Thread; the question is only what the layers above the radio are allowed to assume.

11.7.1 The rules, written while Bluetooth was being brought up

A constraint noticed later is a refactor. So the rules are stated here, at the point where the Bluetooth path was made to work, and they bind the code that follows.

The radio sits behind a protocol-neutral boundary. The HAL tunes, transmits and receives. Bluetooth and 802.15.4 are modules above it. A HAL function whose name contains `advertising`, `access address` or `connection` is in the wrong layer, however convenient it looks at the time.

Channel numbering belongs to the protocol. This chapter paid for that rule: RAIL's channel argument is the *physical* channel, Bluetooth's advertising channels are the pairs $(0, 37)$, $(12, 38)$, $(39, 39)$, and 802.15.4 numbers its channels on a different grid entirely. A caller that passes a number it heard from a specification is passing a number into the wrong namespace.

Services that all three protocols need are written once, and neutrally. The timer, the persistent store and the cipher are not Bluetooth features:

Service	Needed by	Where it belongs
timer over SYSRTC	every stack schedules events	the HAL
persistent store (nvm3)	pairing data, network keys, frame counters	one flash region, one API
cipher	Bluetooth CTR, 802.15.4 CCM*	a block cipher; CCM* is a mode on to

The timer is the immediate consequence. The next section of work is a timer, and writing it as a `sl\_sleeptimer` shim would mean Zigbee needs a second driver for the same peripheral. Writing it as a HAL timer over SYSRTC costs the same and serves three protocols.

One state buffer is a single-protocol assumption. `RAIL\_AddStateBuffer` takes a buffer per protocol instance. Code that hard-codes one global has already decided the product runs one protocol forever.

11.7.2 What the radio does not decide

Whether Bluetooth and an 802.15.4 stack run together, time-sliced, or in separate firmware images is a question this part answers with its 256 kB of RAM, and the answer can wait. What cannot wait is the boundary that keeps the question open. The test is one sentence long, and it can be asked of any new piece of code:

> *Would this be identical for 802.15.4?*

If yes, it belongs in the HAL or in a hardware-free library, and it must be written so that Zigbee and Thread can use it unchanged. If no, it is a protocol module, and neither the HAL, nor the BSP, nor the other protocol may reach into it. Applied consistently, the second radio protocol becomes an addition; applied carelessly, it becomes a rewrite of everything this chapter built.

- A Bluetooth stack arrives as binary blobs plus a C API. The Rust work is bindings, symbols and a safe surface — not a protocol implementation.
- The header and the blobs must be the same SDK version. A mismatch produces calls that exist in the blob and not in the bindings, which reads like a typo and is not.
- The blobs reference `CORE\_*` helpers that the application must provide, and they must be provided once for the whole binary, not once per stack.
- Legacy advertising carries 31 bytes; a name plus a service UUID uses more than half of it, and a builder that refuses an oversized payload is better than one that truncates.
- A connectable stack pulls the bonding path, which needs a crypto library. That is a license and a link problem, not an API problem.
- A radio can report every transmission complete and radiate nothing: the amplifier needs its power curves loaded, and the API's own success says nothing about the air.
- RAIL's channel argument is the physical channel, not the logical one, and the advertising channels are (0,37), (12,38), (39,39).

12. Phase D3: Zigbee 3.0 Protocol Stack

This chapter content will be added

13. Phase D4: Thread Protocol and Matter Integration

This chapter content will be added

Part IV

Testing, BSP, and Production

14. Test-Driven Development and Quality Assurance: Nine Real Drops

> *"A test that passes while the thing it tests is broken is worse than no test at all: it converts an open defect into a certified one."*
> – the rule this chapter was written to satisfy

Embedded software is usually tested in two places: on a host, where the compiler is friendly and the hardware is absent, and on the target, where the opposite is true. Most projects do only the second, call it "testing on hardware", and never learn that the numbers they trusted were produced by an instrument that was lying.

This chapter follows one feature from first line to validated behaviour: a fall detector for a wrist-worn or carried device, built on a six-axis IMU. It is a good case study because it cannot be judged by inspection — a fall is an event in time, and the only way to know whether you detect it is to drop the thing and look. Along the way, three of the measurements used to validate it turn out to be wrong in ways that would have certified a defect as fixed.

14.1 What Can Be Tested Off Target

The first design decision of a testable embedded feature is not a test at all: it is where the code lives.

14.1.1 Split the Policy From the Peripheral

A fall detector is two things wearing one name:

- **Acquisition** — power the IMU, configure its scale and output rate, read a 12-byte burst, convert to milligees. This only exists on the target.
- **Classification** — decide, from a stream of magnitudes and the time between them, that what just happened was a fall. This is arithmetic and state, and it does not know that a sensor exists.

If both live in the driver crate, the second becomes untestable: building the tests drags in a PAC, a linker script and a target triple, so nobody writes them. This project keeps the classifier in its own crate with **no dependencies at all**:

```toml
[package]
name = "motion-detection"
version = "0.1.0"
edition = "2021"

[dependencies]
# deliberately empty: no PAC, no HAL, no panic handler
```

> A dependency-free classifier crate is not an aesthetic choice. It is what makes `cargo test` work on the developer's machine in under a second, which is what makes writing the tenth test cheap enough to bother.

14.1.2 The API That Makes Testing Possible

The crate takes samples and returns events, with the sample interval as a parameter rather than something it looks up:

```rust
pub struct FallDetector { /* phase, peak, timers */ }

impl FallDetector {
    pub const fn new() -> Self;
    /// Feed one sample; the interval is microseconds.
    pub fn update_us(&mut self, accel_mg: [i32; 3], dt_us: u32) -> Option<Event>;
    /// True while the device is believed to be in the air.
    pub fn in_free_fall(&self) -> bool;
}

pub enum Event {
    FreeFall { ms: u32 },
    Impact { peak_mg: i32 },
    Fall { peak_mg: i32, freefall_ms: u32 },
}
```

Two details are worth copying into any similar design:

1. **Time is an argument.** A detector that calls a clock cannot be fed a recording at ten times real speed, and cannot be tested for how it behaves when a sample arrives late. Passing the interval makes both possible and costs nothing.
2. **Microseconds, not milliseconds.** At 400 Hz a sample is 2.5 ms; rounding to whole milliseconds is a 20 % error in every timeout. This is not a theoretical concern — see Section 14.4.

14.1.3 The Shape of the Whole Test Suite

Nine host tests cover the classifier. They are not nine variations of the same case; each one exists because a specific wrong implementation would pass the others:

Test	What it would catch
`quiet_board_reports_nothing`	an implementation that reports on any movement
`a_dip_too_short_is_not_a_fall`	counting a single low sample as free fall
`impact_without_free_fall_is_reported`	landing impact knock a fall
`a_fall_caught_before_landing_is_not_reported`	ignoring the landing entirely
`handling_is_not_a_fall`	thresholds tuned only to the positive case
`real_drop_with_impact_after_a_one_sample_interlude_is_detected`	a single interlude is not the transition
`real_drop_with_a_long_interlude_is_detected`	detection, with a longer gap
`a_soft_landing_after_a_short_free_fall_is_detected`	a landing detector is too strict
`isqrt_is_exact`	square roots that drift and shift every threshold

Table 14.1: The classifier's host tests, each justified by the wrong implementation it rejects.

Every one of those tests was added after a real failure, not before. The suite started with five and grew as the recordings exposed assumptions: the interlude tests came from two drops that were missed, the soft-landing test from a third. That is normal. What is not acceptable is deleting a failing test because the new code is "better".

14.2 Tests That Pin a Threshold to a Measurement

A threshold with no test is a guess that compiles. The discipline that changes it is simple: **every constant is justified by a number, and every number has a test on the other side of it.**

14.2.1 The Near-Miss Rule

Any rule that suppresses or classifies something needs a case that comes *close* and must still behave correctly. For a fall detector the near misses are the interesting ones:

Boundary	Value	Must trigger	Must not trigger
free fall	< 400 mg	real drops: 32–50 mg	shaking: never below 630 mg
free-fall length	≥ 40 ms	42, 70, 80, 100, 138, 145, 176–252 ms	dips of 10–25 ms
landing	≥ 1100 mg	1148–12823 mg	caught in the hand: ≈ 990 m
stillness	< 1500 mg for 600 ms	1.04–1.17 s observed	bouncing: >1500 mg

Table 14.2: Every constant, the measurements on both sides of it, and the control that must not fire.

Two of these margins are uncomfortably thin, and saying so is part of the test:

The landing threshold is 1100 mg with the softest real landing at 1148 mg — an 8 % margin, on nine drops, performed by one person on one desk. It is enough to ship a rule and not enough to claim the rule is right. The test encodes the measurements, not a belief about all landings everywhere.

14.2.2 Writing the Test From the Recording, Not From the Idea

The failing pattern to avoid is a test that restates the implementation:

```
// Useless: this asserts that the code does what the code does.
#[test]
fn detects_a_fall() {
    let mut det = FallDetector::new();
    assert!(det.update([0, 0, 0], 5).is_some());   // whatever the code says
}
```

The useful version uses a sequence of samples taken from a recording, and asserts on the number a person measured:

```
#[test]
fn a_soft_landing_after_a_short_free_fall_is_detected() {
    // Drop 3 of the 400 Hz round: 42 ms of free fall, landing of only 1493 mg,
    // then quiet. This is the case an 800 ms settle rule lost.
    let mut det = FallDetector::new();
    feed(&mut det, [990, 40, 60], 300);      // resting
    feed(&mut det, [30, 25, 20], 42);        // in the air
    feed(&mut det, [1200, 700, 400], 20);    // 1493 mg landing
    match feed(&mut det, [980, 40, 60], 700) {
        Some(Event::Fall { peak_mg, .. }) => {
            assert!((1200..1700).contains(&peak_mg), "peak {peak_mg}");
        }
        other => panic!("expected a fall, got {other:?}"),
    }
}
```

Three properties make this test worth its lines: the numbers come from hardware, the assertion range is wider than the measured value (sensor noise is real), and it fails loudly with the actual event when the behaviour changes.

14.2.3 Embed the Recording in the Image

Host tests prove the logic. They do not prove that the code on the device behaves the same, because the device may run a different build, a different clock, a different optimisation level. The cheap bridge between the two is to embed the recordings in the firmware and replay them at boot:

```
=== self test at boot, no host attached ===
real=clean  synth=ok  200Hz=2/2  400Hz=4/4
```

That line is the whole argument: the handling recording produces nothing, the synthetic fall is detected, and six of the real drops — replayed through the same binary that runs live — are all found. A regression in the shipped image shows up as a different line, on a device that needs no laptop.

> Embed the recordings as `const` arrays (a few kilobytes) and replay them *before* the main loop. It costs one flash write and turns every power-up into a regression test.

14.3 Nine Real Drops

The detector was validated by dropping the board, by hand, and reading what it recorded. Not because that is rigorous — it is not, and the chapter says so below — but because it is the only measurement that can distinguish a plausible rule from a working one.

14.3.1 The Protocol

Three rounds, each recorded by the board itself, plus one control:

1. Configure the IMU for a range that will not clip (±8 g) and the fastest output rate it offers (416 Hz).
2. Sample as fast as the loop allows, and record every sample *with its interval*.
3. Drop the board from about 40 cm, several times, two seconds apart.
4. Then perform the negative case: shake hard, set it down suddenly, shake again.
5. Read the log over the debug probe, with a script, and analyse it on the host.

The control is not optional. A detector that fires on everything detects every fall. Every round of drops in this chapter was recorded together with a handling round that had to stay silent, and the thresholds were chosen to satisfy both at once.

14.3.2 What the Recordings Showed

Round	Rate	Drops	Detected
1	200 Hz	2	2
2	200 Hz	5	3 (two lost their free fall between samples)
3	400 Hz	4 onto a bed	4
control	200 Hz	handling	0 false alarms

Table 14.3: Three rounds of drops and one control, all recorded on the board.

Round 3 is the one worth reading in full, because it is the round that corrected the design:

Free fall	Landing	Quiet before the verdict	Verdict
176 ms	11699 mg	1.04 s	fall
223 ms	13840 mg	1.05 s	fall
248 ms	13149 mg	1.14 s	fall
252 ms	13742 mg	1.17 s	fall

Table 14.4: Four drops onto a bed, measured. The landings are three to four times harder than the prediction that a mattress would absorb them.

14.3.3 The Three Assumptions the Data Destroyed

"A fall is a big impact." Real landings measured 1148 to 12823 mg. Handling peaked at 1801 mg. The ranges overlap: a board laid down flat lands more gently than a board shaken hard, so **no impact threshold can separate a fall from a knock**. What separates them is the free-fall phase. The impact threshold only has to distinguish "landed" from "returned to one g because it was caught".

"The impact follows the free fall immediately." In the first recordings it came 120 ms later, and in the second 400 ms later. A state machine that required the landing at the transition detected **zero of two** drops. Opening a 600 ms window took the same data to two of two.

"A bed is soft." Predicted: a mattress landing would fall below the 1100 mg threshold and be missed. Measured: 11.7 to 13.8 g. The prediction was wrong, the rule was right, and the honest conclusion is that nine drops on one bed say nothing about mattresses in general.

What this validation is not. Nine drops, one board, one desk, one bed, one person. It proves the detector works on this setup and that the thresholds are not arbitrary. It does not prove a fall-detection rate for a product, and a claim that it did would be the kind of overreach that gets someone hurt.

14.3.4 What Is Still Missing

Two things are absent from this validation and both are visible in the code:

- **The gyroscope is read and ignored.** Rotation would separate "dropped while flat" from "tipped over on a table", and the data is already in the same burst.
- **A person falling is not a board being dropped.** A carried device falls with its owner: shorter free fall, cushioned landing, and a body that moves afterwards. The classifier is tuned to the harsher case for the impact threshold and the easier one for the free-fall phase, which is a defensible starting point and not an answer.

14.4 The Instrument That Lied

Everything in the previous section was, at one point, wrong in the same way: the measurements were taken on a time axis that did not exist. This section is the most important one in the chapter, because the failure was silent, it survived a code review, and it certified a defect as fixed.

14.4.1 The Sampler That Claimed 400 Hz and Ran at 158

The loop was written to sample at 400 Hz and handed the detector a constant:

```
const SAMPLE_PERIOD_CYCLES: u32 = 47_500;  // 400 Hz at 19 MHz
const DT_US: u32 = 2_500;                  // 400 Hz, in microseconds
...
application.sample_us(motion.accel_mg, DT_US);
```

The loop was paced correctly — it never sampled *faster* than 400 Hz. But each iteration also did work: a 12-byte I2C burst, a detector update, a record write, and periodically a 1090-byte display flush over the same bit-banged bus. The true interval was set by the **slowest** part of the body, not by the period:

```
measured: 317 samples in 2 s  ->  158 Hz, mean interval 6.3 ms
```

Every timeout in the detector was therefore 2.5 times too short, and every duration in the first analysis — "40 ms of free fall" — was 2.5 times too small. The measurements were self-consistent, which is exactly why nobody noticed.

14.4.2 What Gave It Away

Not a test. A pattern in the output:

```
AIRBORNE  free 85ms   ->  FALL   after 0.62 s
AIRBORNE  free 52ms   ->  FALL   after 0.62 s
AIRBORNE  free 120ms  ->  FALL   after 0.62 s
AIRBORNE  free 40ms   ->  FALL   after 0.53 s
```

Four different drops, four different free falls, and the settle time is the same to the millisecond. **The number was describing the code, not the world.** That is the signature of a broken instrument: when an output is constant across inputs that should vary, you are reading your own parameter back.

14.4.3 Three Fixes, and One of Them Was Deleted by the Compiler

1. **Measure the interval.** The loop takes the difference of the DWT cycle counter between samples and passes that everywhere — detector, recorder, alarm.

```
let dt_us = now.wrapping_sub(last_sample_cycles) / CYCLES_PER_US;
last_sample_cycles = now;
application.sample_us(motion.accel_mg, dt_us);
```

2. **Record the interval with every sample.** A dataset whose timing is unknown can only be analysed dishonestly. One byte per sample, in 250 μs units.

3. **Get the display out of the critical path.** Refresh every 800 samples instead of 80, and never while the device is airborne or within 1.2 s of an event: a flush in the middle of a fall stalls the sampler exactly when the samples matter. Measured effect: 158 Hz to 235 Hz without touching the sensor.

And then the second-order failure, which is the reason this section exists:

The compiler deleted the per-sample timing array. The recording was written to "carry its own timing", and nothing in the crate ever read that array — so the optimiser removed it from the image entirely. The code compiled, the comment claimed a property, the symbol was absent from the binary, and the recording carried nothing. It was found by running `arm-none-eabi-nm` and looking for the symbol instead of trusting the comment. The writes are volatile now, and the check is part of the habit: *a property you cannot see in the artefact is a property you do not have.*

14.4.4 The Instrument Rules

- **Never pass the interval you asked for; pass the one you measured.** The intended rate is a wish. The DWT counter is the fact.
- **A constant output across varying inputs means you are measuring yourself.** The 0.62 s settle was a parameter echoed back.
- **A recording must carry its timing.** Otherwise every later analysis inherits an assumption nobody remembers making.
- **Verify the artefact, not the intent.** Check that the symbol is in the binary, that the counter advanced, that the file grew. Comments describe intentions; the binary is what runs.
- **Calibrate the instrument before calibrating the product.** The rate, the units and the resolution have to be right before any threshold on top of them means anything.

14.5 What To Take Away

- **Put the policy in a crate with no dependencies.** If the classifier needs a PAC to compile, its tests will not be written.
- **Pass time in as an argument.** It makes fast replay, late samples and deterministic tests free.

- **Every threshold is a measurement with a test on both sides.** A rule without a near-miss case is a guess with a comment.
- **Embed the recordings and replay them at boot.** Every power-up becomes a regression test on the exact binary that ships.
- **Calibrate the instrument first.** A wrong sample rate invalidates every number downstream, silently and consistently.
- **Say what the validation is not.** Nine drops prove a rule is not arbitrary; they do not prove a detection rate for a product.

14.5.1 Exercises

1. **(Easy)** Take the classifier out of the driver crate in a project of your own and make its tests run with `cargo test` on the host. Report how long the suite takes before and after.
2. **(Easy)** Add a test for the boundary on each side of one threshold: a sample exactly at the limit, and one a single unit below it. Which side does your implementation treat as inclusive, and is that what the documentation says?
3. **(Medium)** Instrument your own sampling loop with `DWT->CYCCNT` and report the real rate and the worst-case interval. Compare with the rate you intended.
4. **(Medium)** Add the gyroscope to the classifier: compute the rotation during the free fall and use it to separate "dropped flat" from "tipped over". What additional recording do you need to set the threshold, and can you get it without a second person?
5. **(Hard)** Design the validation for a detector of a different event of your choice (a door being forced, a machine starting, a container being opened) following the protocol of this chapter: rounds, a control that must stay silent, near-miss cases, and an explicit statement of what the sample does not prove.

Where this leads. The next chapter takes the same evidence-driven approach to the layer above the drivers: the board support package, where the question stops being "does this device work" and becomes "does this board work, and how do I prove that nobody has to know which pin it is on".

15. Board Support Packages and Hardware Abstraction

> *"A BSP is the layer that knows which wire is which — and it is the only layer allowed to."*

The PAC knows registers. The HAL knows protocols. Neither knows that a particular board puts an LED on `PA7` with the cathode to the pin, and that is the BSP's job: the last layer between the user's intent and the silicon, and the only one that should contain a pin number.

This chapter builds a BSP for hardware that arrives in two pieces — a module and a carrier — which is the arrangement that exposes every mistake in the design.

15.1 Two Pieces of Hardware, Not One

This board is not one board. It is a module — the XIAO MG24 Sense, which runs on its own — plus a carrier that the module plugs into. The distinction is physical, it decides who owns which feature, and a single BSP crate erases it.

15.1.1 What Each Piece Owns

Piece	Components	Can be used alone?
XIAO MG24	LED `PA7`, USB, radio	yes
XIAO MG24 *Sense*	the above + IMU `LSM6DS3TR-C`, analog mic	yes
Expansion Base	OLED, BME68x, RTC, buzzer, button, Grove, microSD	no; needs a XIAO

Table 15.1: Three hardware identities and one board on the desk. The Sense is a variant of the module, not a different module.

15.1.2 The Same PCB With More Parts

The Sense is a XIAO with two extra components fitted. That single fact decides the crate layout:

- **A feature, not a crate.** Two crates would each need the boot sequence, the clock tree, the LED and the pin map — four things to keep in step, for two products that are physically the same PCB.
- **A feature that is opt-in.** A user with a plain XIAO must not compile the IMU driver or its bus. `--features sense` adds exactly those.
- **The feature names the thing that is fitted,** not the price bracket or the marketing name: `sense` means "the two sensors are populated".

```toml
[features]
default = []
# The Sense variant is the same PCB with more components fitted: the IMU and the
# analog microphone. Same board, fewer parts -- so a feature, not a crate.
sense = []
```

15.1.3 The Carrier Is a Different Crate, and It Points One Way

The Expansion Base adds devices the module has never heard of, and a user may own neither, one or both. So it is its own crate, and the dependency has a direction:

```
xiao-expansion-base-bsp  ──depends on──▶  xiao-mg24-bsp  ──depends on──▶  efr32mg24-hal
```

The opposite arrow — the module crate knowing that a carrier exists — is the defect this book's section 1.8 describes: it makes the module unusable without the carrier's code, and it puts carrier pins in a crate whose whole claim is "the XIAO on its own".

15.1.4 What the Direction Buys

- A user with only the XIAO compiles one small crate, not the carrier's drivers.
- Carrier examples live where the carrier is, so the path of a file tells you the hardware it needs before you open it.

- The module crate cannot drift into knowing the carrier, because the manifest forbids it — and `cargo` enforces manifests even when reviewers do not.

15.2 One Crate per Board, and the Arrow Between Them

The crate layout is not a matter of taste. It follows from one question: *which of these things can be owned separately?*

15.2.1 The Two Schools

Project	Layout	Where the wiring lives
`stm32f4xx-hal`	one HAL per family, no BSP	in every example
`esp-hal`	one HAL, no BSP, chip by feature	in every example
`rp-hal` / `rp-hal-boards`	HAL per chip + a BSP crate per board	in the BSP crate

Table 15.2: Two schools. The difference is not sophistication: it is whether the board's wiring is a fact worth writing down once.

Both schools are legitimate. The first optimises for a HAL whose users all have a schematic in front of them; the second optimises for a board whose wiring was expensive to establish. This project is firmly in the second camp, for a reason that is in chapter 4's table of contradictions: the pin map of this board was measured, and a measurement that is repeated in every example is a measurement that will be repeated wrongly.

15.2.2 The Four Crate Questions

1. **How many chips?** One HAL per chip or per family. This part gets one.
2. **Do the boards stack?** If a carrier holds a module, one crate per piece, arrow carrier → module.
3. **Are there variants?** A variant of the *same* PCB is a feature. A different PCB is a crate.
4. **Was the wiring obvious or was it conquered?** Obvious wiring can live in examples; conquered wiring belongs in a BSP with the measurement next to it.

15.2.3 What Ends Up in the Workspace

```
crates/
  efr32mg24-pac/            registers
  efr32mg24-hal/            drivers + pin and bus primitives
  xiao-mg24-bsp/            Module: led(), and with --features sense: imu(), microphone()
  xiao-expansion-base-bsp/  ExpansionBase: oled(), environment(), rtc(), buzzer(), button()
libraries/
  motion-detection/         domain policy: no hardware at all
```

Two rules hold that shape together, and both are checks in this project's gate:

- **Nothing in libraries/ may depend on the hardware stack.** A classifier that needs a HAL is a classifier that cannot be tested on a laptop, and the test suite is the only thing that makes it trustworthy.
- **Every crate that exposes hardware carries at least one example of its own level.** An API with no examples compiles, lints, and rots — this project had a whole module do exactly that for weeks.

15.2.4 The Defect the Layout Prevented

Before the split, the HAL itself contained `board.rs` with the carrier's wiring — the OLED bus, the button pin, the buzzer channel — because it was convenient. Two consequences followed immediately:

1. The HAL became unusable without the carrier's assumptions, so a user with a bare XIAO had to read carrier pins to blink an LED.
2. Two BSPs existed at once: the HAL's board module and the new facade. Half the examples proved one, half the other, and the older one stopped compiling without anyone noticing.

The fix was not better documentation. It was deleting the wiring from the HAL and making the dependency direction a manifest fact.

15.3 What a BSP Should Expose

A BSP's public API is a vocabulary, and the test of a vocabulary is what a user can say with it without knowing anything else.

15.3.1 The Right Shape

```
let module = Module::take().expect("peripherals taken twice");
let mut led = module.led();

let base = ExpansionBase::take().expect("peripherals taken twice");
let mut oled = base.oled();
let env = base.environment()?;      // BME68x
let rtc = base.rtc()?;              // PCF8563
let mut buzzer = base.buzzer();
```

Every one of those names is a thing on the desk. None of them is an address, a bus, a pin, an alias or a register — and that is the whole specification.

15.3.2 What the Signature Hides, and What It Must Not

Hidden	Example	Why hiding it is right
addresses	`0x3C`, `0x76`, `0x51`	a user cannot check them and must not need to
pins	`PC04/PC05`, `PB2/PB3`	measured facts; wrong in half the documents
power gates	`PD5`, `PC8`	sequencing the user would get wrong
aliases	which peripheral window answered	a security-attribution detail
baud and clock	I2C half-period, PWM frequency	driver knowledge

Table 15.3: What a facade should hide. Note that every row is a *fact*, not a *choice* — the user has no decision to make about any of them.

15.3.3 What It Must Not Hide: Structure

The temptation is to hide the two buses, because "two I2C buses" is an implementation detail. It is not — it is a fact about the hardware that changes what the user can do:

- The panel, the BME68x and the RTC share `PC04/PC05`.

- The IMU has its own pair, `PB2`/`PB3`, because the module's sensors are wired to the module's pins.

- A single `i2c()` would have to take a "which bus" argument anyway, so hiding it only moves the choice to a place with less information.

The rule that came out of this: **hide the addresses, keep the structure**. A facade that hides structure has to guess, and guessing is what the layer below exists to prevent.

15.3.4 Conflicts Should Be Refused, Not Resolved

```
// The microSD card's chip select shares a pin on this board. When that driver lands,
// this facade is where the conflict is refused -- not silently arbitrated.
pub fn sd(&mut self) -> Result<SdCard<'_>, BoardError>
```

A facade that resolves a real conflict silently is a facade that produces a bug the user cannot see. Returning an error — or not offering the operation at all — is honest, and it is the difference between an API that documents the hardware and one that pretends the hardware is simpler than it is.

15.4 Examples Belong to the Hardware They Need

The most practical BSP rule in this book is about where example files live, because that is where a layout either documents itself or lies.

15.4.1 The Rule

An example lives in the crate whose **smallest** hardware requirement it satisfies.

Example	Lives in	Needs
`module\_led\_blink`	`xiao-mg24-bsp`	the module
`module\_imu`	`xiao-mg24-bsp` (`sense`)	the module, Sense variant
`base\_env\_clock`	`xiao-expansion-base-bsp`	module + carrier
`base\_fall\_detector`	`xiao-expansion-base-bsp`	module + carrier (panel, buzzer)

Table 15.4: The path of the file is the bill of materials.

15.4.2 The Three Ways to Get It Wrong

1. **Everything in the carrier crate.** Convenient, and it makes the module crate's API unexercised: this project's module facade had no examples for weeks, and its documentation described a pin that did not exist.

2. **Everything in the module crate.** Impossible without the carrier's devices, so the examples would need `cfg` guards and would prove neither level.

3. **A flat examples/ directory at the workspace root.** The path stops telling you anything, and `cargo` stops checking that the crate still builds its own examples.

15.4.3 The Check That Keeps It True

```
# every crate that exposes hardware must carry at least one example of its own level
for dir in crates/*/ libraries/*/; do
  [ -f "${dir}Cargo.toml" ] || continue
  n=$(find "$dir/examples" -maxdepth 1 -name '*.rs' | wc -l)
  [ "$n" -gt 0 ] || fail "$(basename "$dir"): no examples -- its API is not built by anything"
done
```

That check is why this project's module facade has a `module\_led\_blink` that uses only the module, and why the carrier's examples are in the carrier. A rule enforced by the file system is a rule nobody has to remember.

15.4.4 The Half-Example That Taught the Rule

A fall detector was written in the HAL's examples directory. It used the IMU, the panel and the buzzer: two boards' worth of hardware, in the crate that knows neither. It compiled, because the HAL happened to contain the wiring at the time — which was itself the defect the split fixed.

After the split, the same program had an obvious home: the carrier crate, which depends on the module and therefore has both the IMU (through the module) and the panel (its own). Moving it changed no code. That is the sign of a layout that was right: the program did not have to change to be correctly placed.

- Hardware that can be owned in pieces needs a crate per piece, with the dependency arrow pointing one way: carrier depends on module, module depends on HAL.
- A variant of the same PCB is a feature, not a crate. Two crates would duplicate the boot, the clock tree and the LED, and one of them would drift.
- A BSP's API is what a user asks for — `led()`, `imu()`, `oled()` — never an address, an alias or a register.
- A facade that hides a real conflict (two buses, a shared pin) hides it until the user hits it. Hide the addresses; keep the structure visible.
- Every example belongs to the crate whose hardware it needs, and the gate fails a crate that has hardware and no example.

16. Documentation Strategies and Project Management

This chapter content will be added

17. Production Deployment and Ecosystem Integration

This chapter content will be added

18. Complete Project Walkthrough: From Zero to Production

This chapter content will be added

A. EFR32MG24 Reference Manual Quick Guide

This appendix will be added.

B. Rust Embedded Ecosystem Overview

This appendix will be added.

C. Troubleshooting Common Issues

This appendix will be added.

D. Complete Code Examples Repository

This appendix will be added.

E. Wireless Protocol Comparison

This appendix will be added.

End of Book

For more information and updates:
https://github.com/bitscrafts/EFR32MG2X-RS

Document built: November 14, 2026
Build version: 1.0.0